

final-project-v5.R

liula

2025-10-16

```
## QUESTION 1 ##  
# GOAL: Exploratory Data Analysis  
# check for missing values, replace if needed  
# check data types of all the columns in the data frame  
# determine insights
```

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —  
## ✓ dplyr      1.1.4      ✓ readr      2.1.5  
## ✓ forcats   1.0.0      ✓ stringr   1.5.1  
## ✓ ggplot2   3.5.2      ✓ tibble    3.3.0  
## ✓ lubridate 1.9.4      ✓ tidyr     1.3.1  
## ✓ purrr     1.1.0  
## — Conflicts — tidyverse_conflicts() —  
## ✗ dplyr::filter() masks stats::filter()  
## ✗ dplyr::lag()    masks stats::lag()  
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
Listings = read.csv("Listings.csv")
Reviews = read.csv("Reviews.csv")
# names(Listings)
# names(Reviews)
# glimpse(Listings)
# glimpse(Reviews)

## use a left join to make sure that the data frame has all the rows from Listing and the additional info from Reviews
df = left_join( Listings , Reviews , by = c("id" = "listing_id"))
## check to see if there are any NaNs in the data set

# apply( df, 2, FUN = anyNA)

## find the index for where the NaNs are located and store in missing_beds vector
missing_beds = which(is.na(df$beds))
## using missing_beds vector to replace the NaNs with the mean
## beds should not double -> use ceiling to round 1.92 up to 2
df$beds[missing_beds] = ceiling(mean(df$beds, na.rm = TRUE))
## do the same thing with avg rating
missing_rating = which(is.na(df$avg_rating))
# df$avg_rating[missing_rating] = which(is.na(df$avg_rating))
df$avg_rating[missing_rating] = mean(df$avg_rating, na.rm = TRUE)

## since there are blank values in room type decided to replace these blanks with "unknown"
df$room_type[df$room_type == ""] <- "Unknown"

## now that the data frame has been cleaned of NaNs and Blanks
df = df %>%
  select(-id) %>%
  mutate(neighborhood = as.factor(neighborhood) , host_since = as.Date(host_since, format = "%m/%d/%Y")
         , room_type = as.factor(room_type) , bathrooms = as.factor(bathrooms),
         bedrooms_f = as.factor(bedrooms))
glimpse(df)
```

```
## Rows: 1,931
## Columns: 15
## $ neighborhood      <fct> Shaw, Capitol Hill, Edgewood, Edgewood, Shaw, Col...
## $ host_since         <date> 2009-10-31, 2011-09-01, 2012-06-27, 2010-12-10, ...
## $ host_acceptance_rate <dbl> 0.89, 0.80, 0.99, 0.76, 0.83, 0.98, 0.83, 0.40, 0...
## $ superhost          <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, ...
## $ host_total_listings <int> 7, 4, 11, 91, 2, 2, 2, 1, 1376, 91, 13, 1, 1, 4, ...
## $ room_type          <fct> Entire home/apt, Entire home/apt, Entire home/apt...
## $ accommodates       <int> 4, 3, 6, 1, 3, 2, 6, 2, 3, 1, 6, 2, 2, 2, 5, 2...
## $ bathrooms          <fct> 1 bath, 1 bath, 1 bath, 2 shared baths, 1 bath, 1...
## $ bedrooms           <int> 2, 1, 1, 0, 1, 1, 3, 1, 1, 1, 2, 1, 1, 1, 3, 0...
## $ beds              <dbl> 2, 1, 4, 1, 2, 2, 3, 1, 2, 1, 3, 1, 2, 1, 1, 3, 1...
## $ price              <dbl> 72, 205, 93, 30, 200, 96, 224, 130, 338, 55, 272,...
## $ min_nights         <int> 31, 2, 2, 31, 3, 2, 4, 31, 1, 31, 3, 2, 2, 2, 2, ...
## $ total_reviews      <int> 191, 156, 377, 4, 14, 268, 22, 26, 35, 12, 206, 5...
## $ avg_rating         <dbl> 4.31, 4.80, 4.75, 3.25, 4.86, 4.85, 4.95, 4.76, 4...
## $ bedrooms_f         <fct> 2, 1, 1, 0, 1, 1, 3, 1, 1, 1, 2, 1, 1, 1, 3, 0...
```

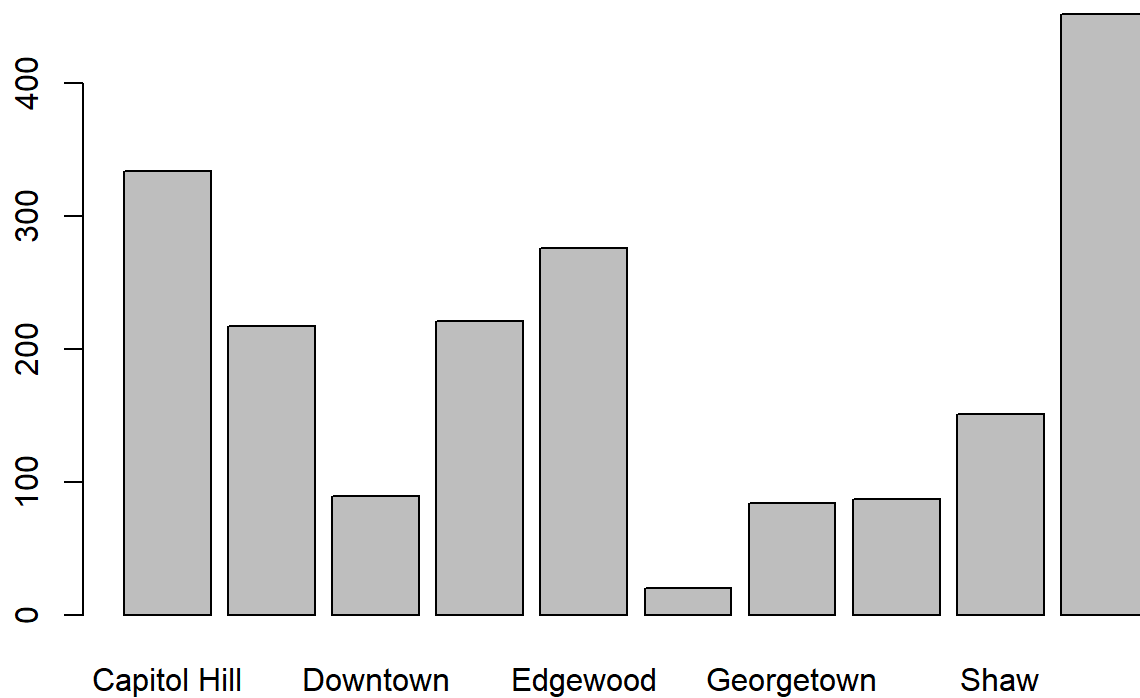
Let us explore each column

1) Neighborhood: Character data type -> convert to factor to make more robust for categorical analysis

```
neigh_freq=table(df$neighborhood)
```

it was found that Union Station, Capital Hill, Columbia heights, dupont, and edgewood were the most popular

```
barplot(neigh_freq)
```



```
# 2) host_since is a character data type -> convert to a date
## decided to use the IQR method of removing outliers since the data was skewed with an extreme outlier

interquartilerange= IQR(df$price)
upper = quantile(df$price, 0.75)
lower = quantile(df$price , 0.25)
upper_outlier = upper + 1.5 * interquartilerange
lower_outlier = lower - 1.5 * interquartilerange
df_price= df %>%
  filter ( price > lower_outlier & price < upper_outlier)

# ggplot(df, aes(x=host_since, y = price)) + geom_line() +labs(y = "Price" , x = "Host_since",
# title = "Line Plot of Raw Price V Host_since")
# ggplot(df_price, aes(x=host_since, y = price)) + geom_line() +labs(y = "Price" , x = "Host_si
# nce",
# title = "Line Plot of Filtered Price V Host_since")

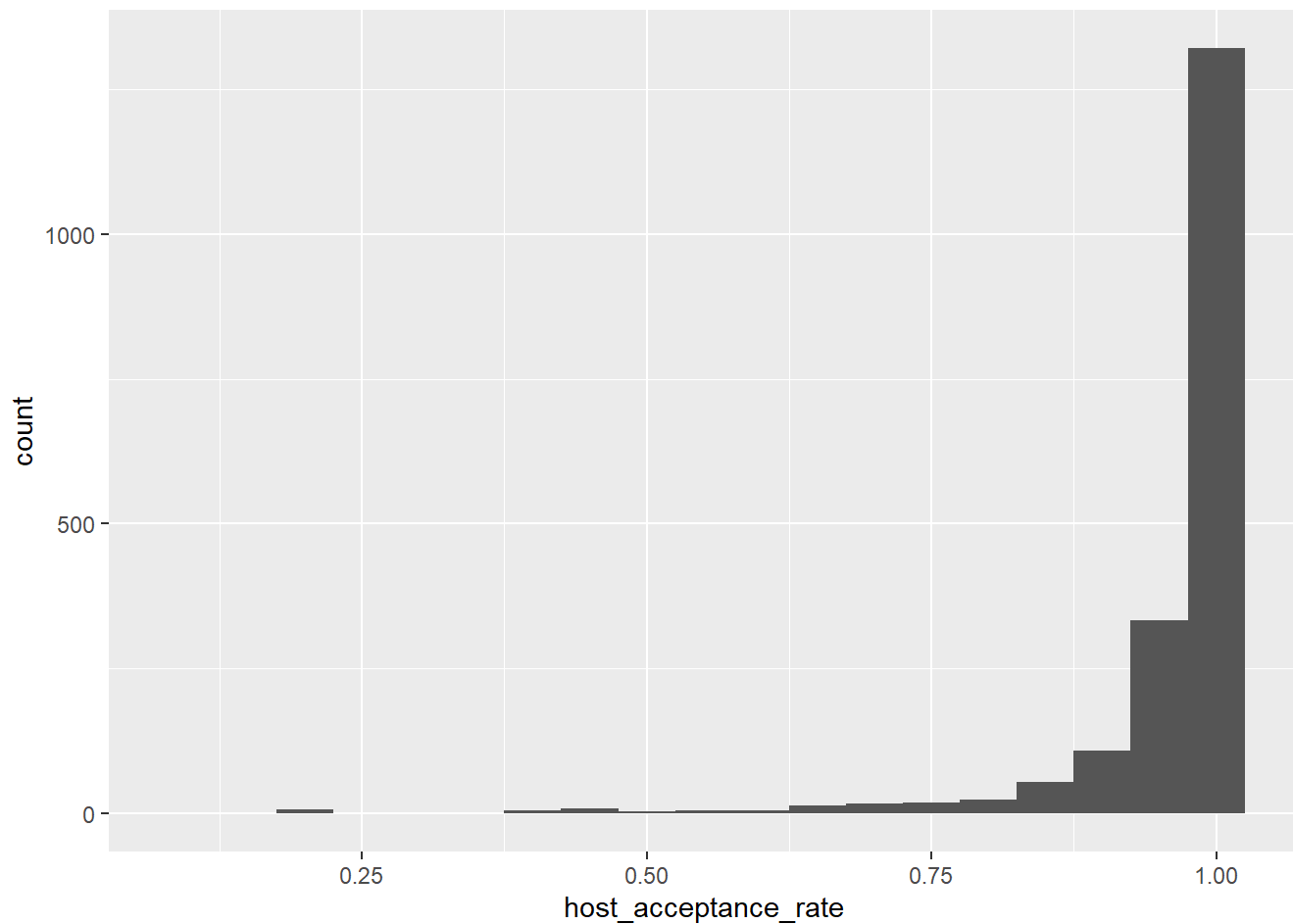
# 3) host_acceptance_rate is a double
## data type makes sense, no need to change . explore values

summary(df$host_acceptance_rate)
```

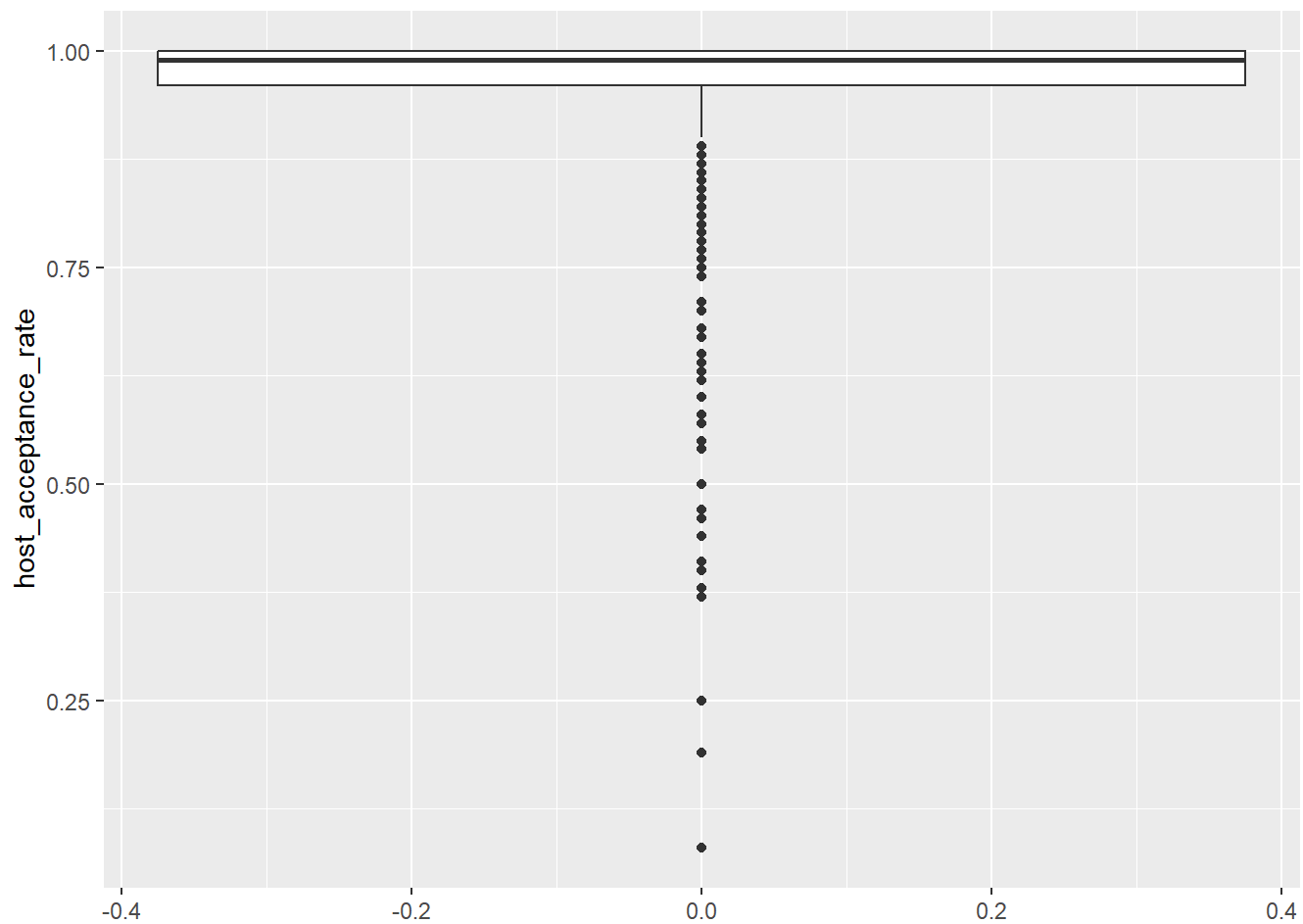
```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.0800  0.9600  0.9900  0.9563  1.0000  1.0000
```

```
## summary stats show possible left skewness
```

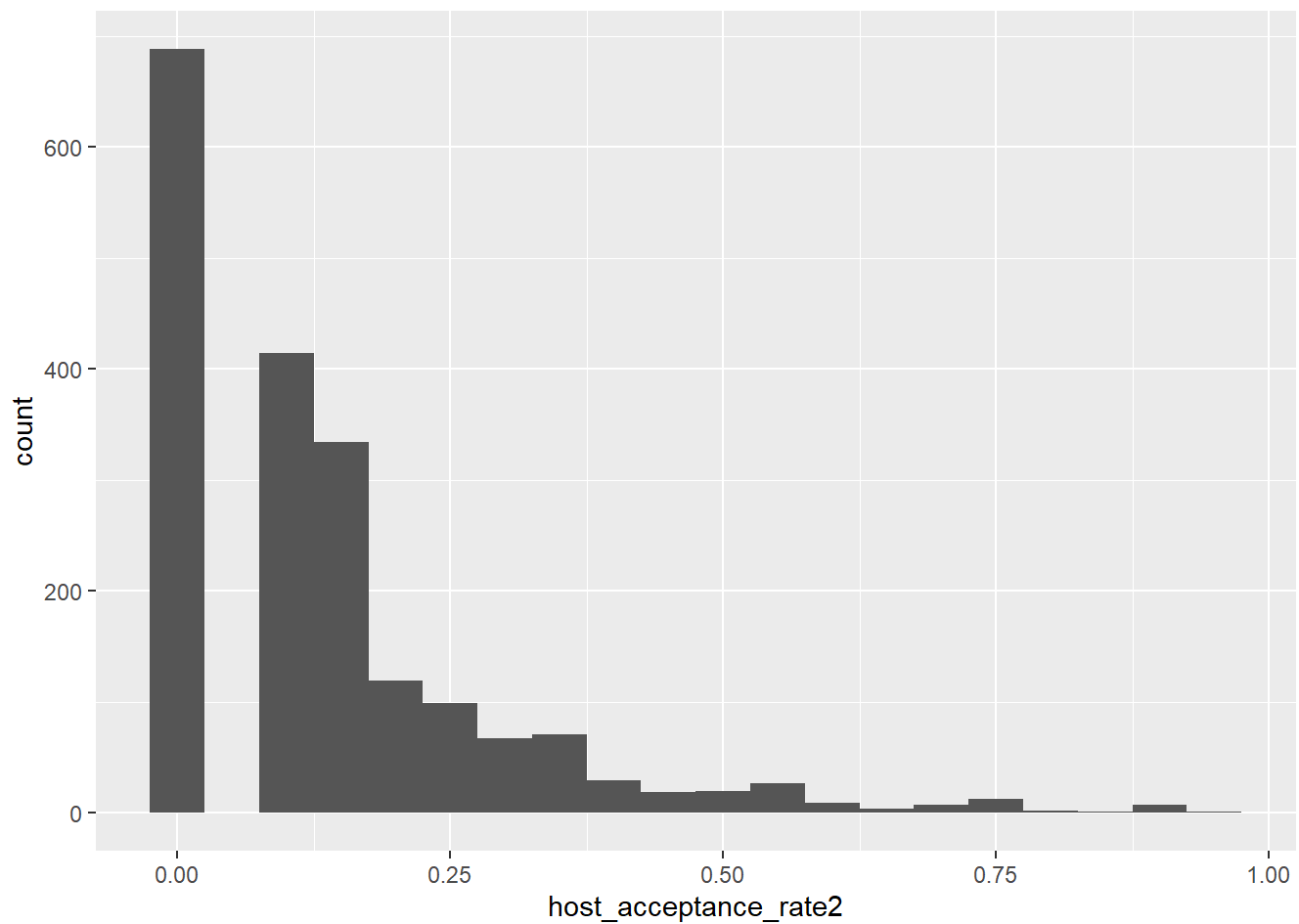
```
ggplot(df, aes(host_acceptance_rate)) + geom_histogram(binwidth = 0.05)
```



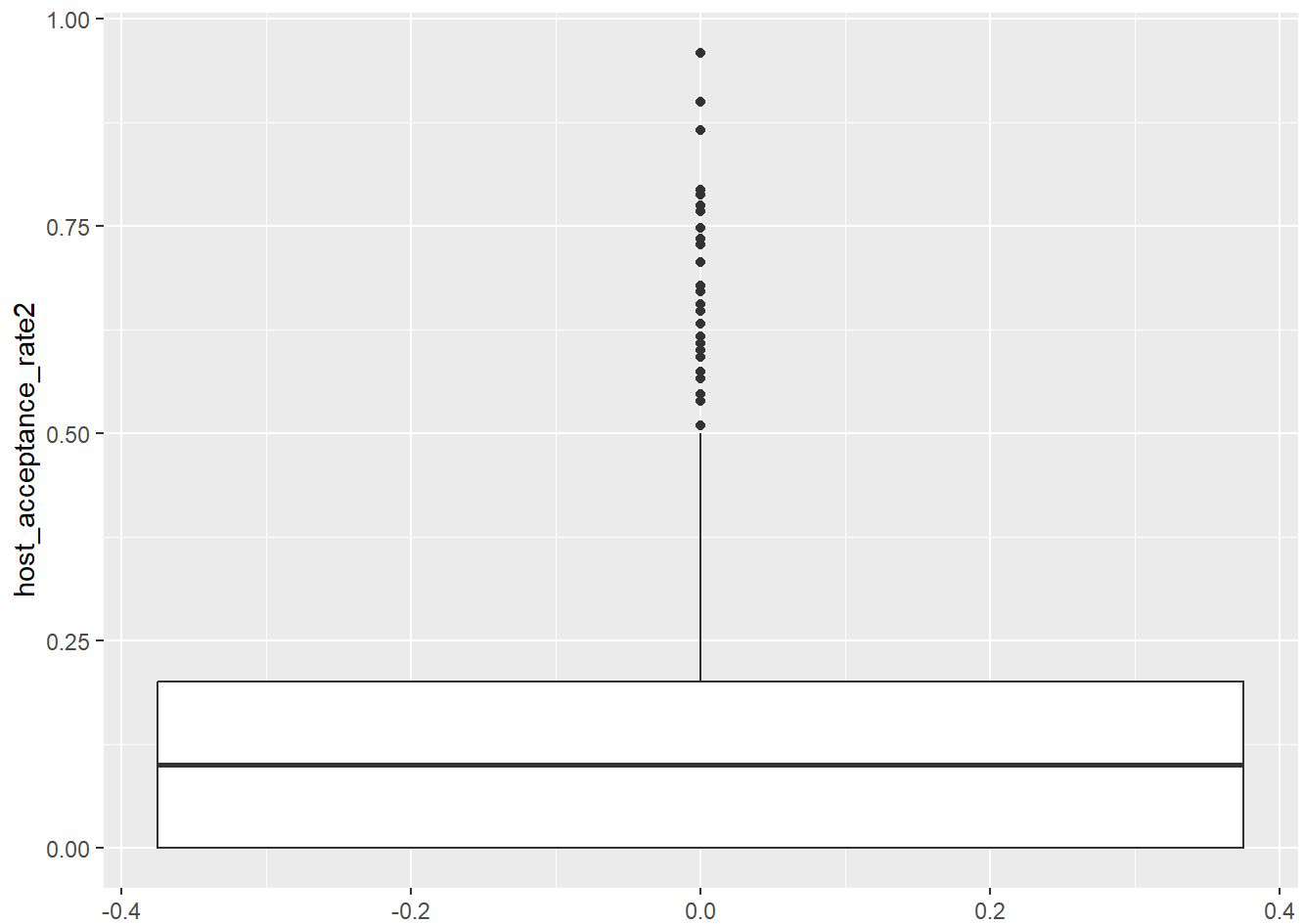
```
ggplot(df, aes(host_acceptance_rate)) + geom_boxplot() + coord_flip()
```



```
## box plot and histogram confirm left skewness
df_host_test = df %>%
  mutate( host_acceptance_rate2 = ((1-(host_acceptance_rate))^(1/2)))
## since data is left-skewed, can manipulate
ggplot(df_host_test, aes(host_acceptance_rate2)) + geom_histogram( binwidth = 0.05)
```



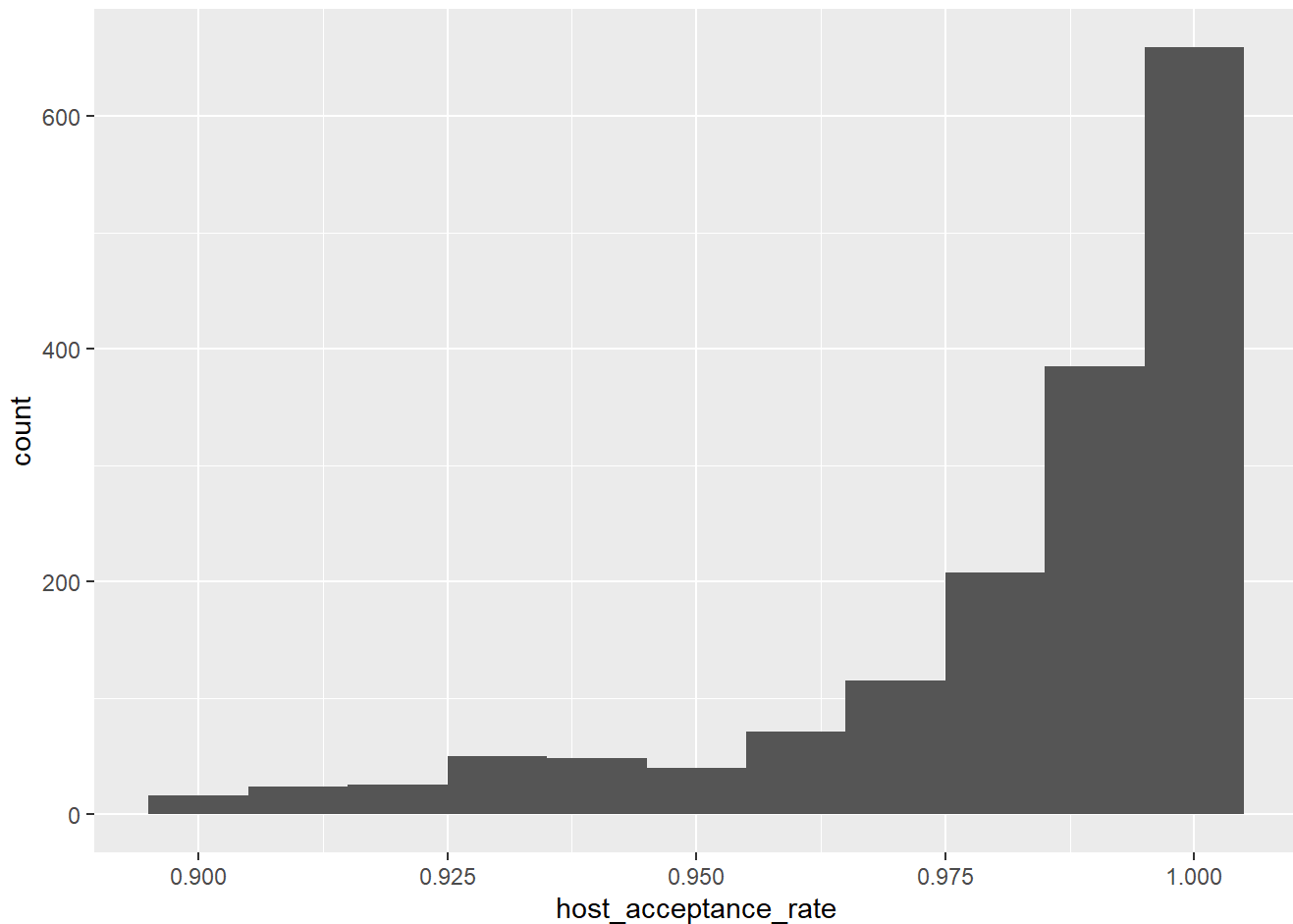
```
ggplot(df_host_test, aes(host_acceptance_rate2)) + geom_boxplot() + coord_flip()
```



```
## Transform was not great, Outlier analysis performed
## data is very skewed so IQR outlier analysis is the best approach
IQR_host_rate = IQR(df_price$host_acceptance_rate)
upper_host_price = quantile(df_price$host_acceptance_rate , 0.75)
lower_host_price = quantile(df_price$host_acceptance_rate , 0.25)
upper_outlier_host_price = upper_host_price + 1.5 * IQR_host_rate
lower_outlier_host_price = lower_host_price - 1.5 * IQR_host_rate

df_host_price = df_price %>%
  filter(host_acceptance_rate > lower_outlier_host_price & host_acceptance_rate < upper_outlier_
host_price)

ggplot(df_host_price , aes(host_acceptance_rate)) + geom_histogram(binwidth = 0.01)
```

4) Superhost is a logical data type -> this works but for analysis might be better to have 0 and 1

use if else for a vectorized approach of replacing all values

```
superhost2=ifelse(df_price$superhost == FALSE, 0 , 1)
```

append the new vector to the existing data frame df

```
df_price = data.frame(df_price, superhost2)
```

```
superhost_freq = table(df_price$superhost)
```

```
superhost2_freq = table(df_price$superhost2)
```

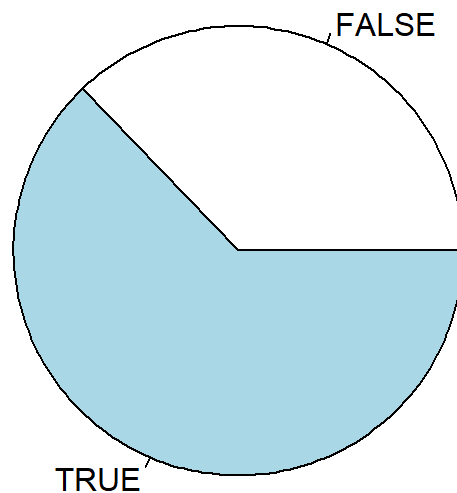
```
prop.table(superhost_freq)
```

```
##
```

```
##      FALSE      TRUE
```

```
## 0.3718784 0.6281216
```

```
pie(superhost_freq)
```

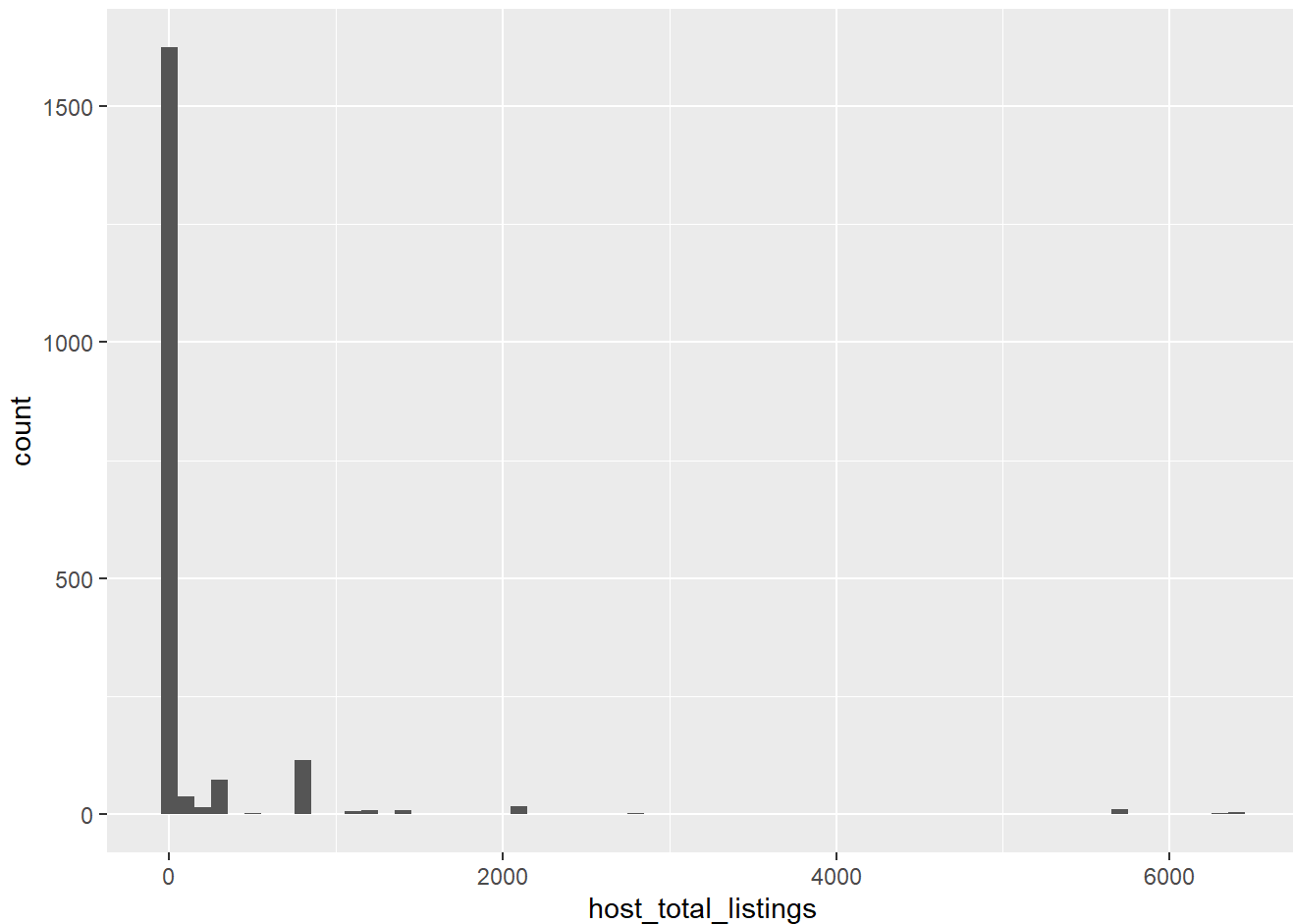


```
# pie(super host2_freq)
## approximately 2/3 of users are super hosts
## makes sense given that the host acceptance rate is so high
```

```
# 5) host_total_listing is an integer data type
## first find summary statistics
summary(df$host_total_listings)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##       1.0     1.0     4.0   158.1   15.0  6397.0
```

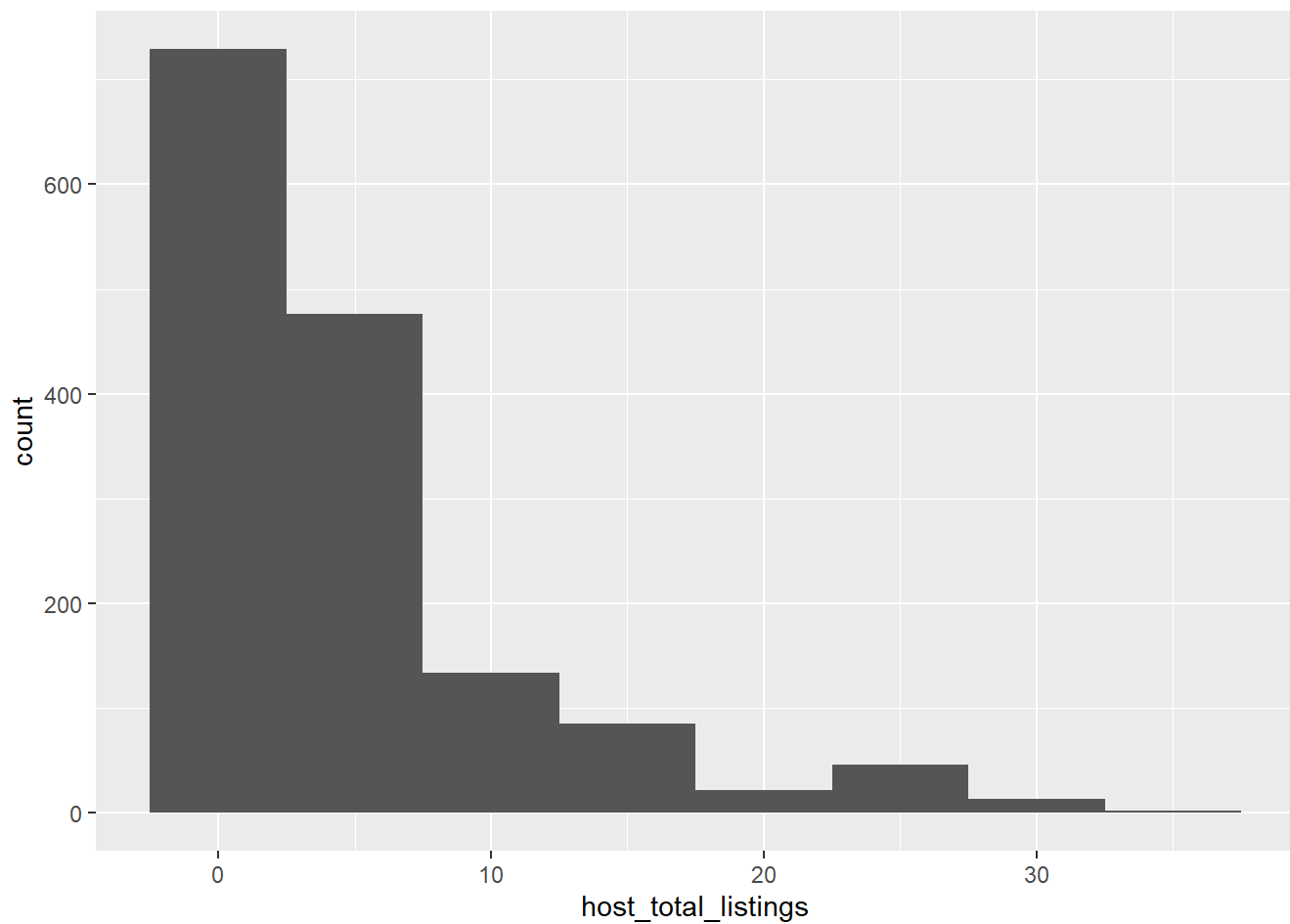
```
ggplot(df, aes(host_total_listings)) + geom_histogram(binwidth = 100)
```



```
iqr_total_listings = IQR(df_price$host_total_listings)
upper_total_listing = quantile(df_price$host_total_listings, 0.75)
lower_total_listing = quantile(df_price$host_total_listings, 0.25)
upper_outlier_total_listings = upper_total_listing + 1.5 * iqr_total_listings
lower_outlier_total_listings = lower_total_listing - 1.5 * iqr_total_listings

df_listings_price = df_price %>%
  filter ( host_total_listings > lower_outlier_total_listings & host_total_listings < upper_outl
ier_total_listings)

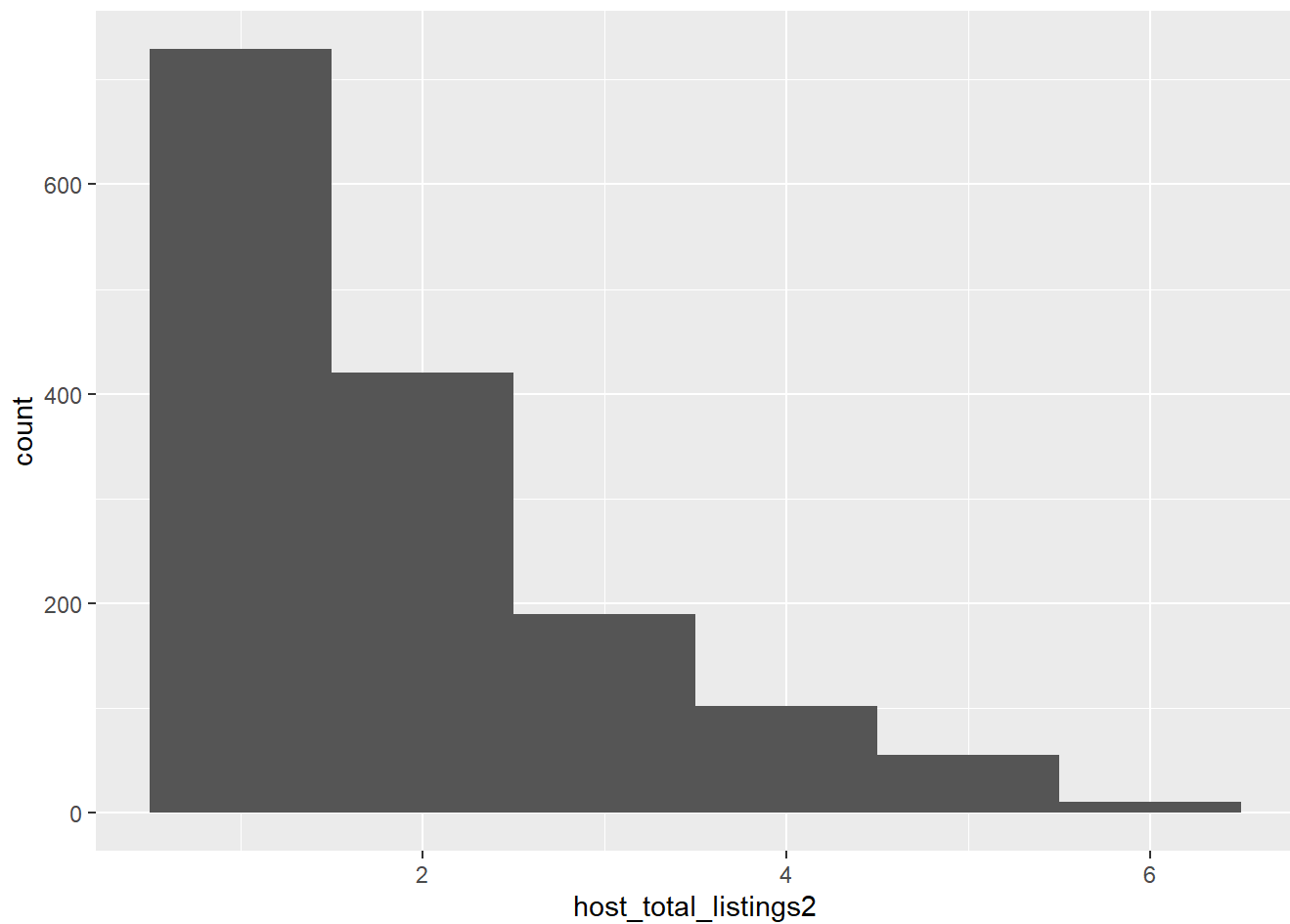
ggplot(df_listings_price, aes(host_total_listings)) + geom_histogram(binwidth = 5)
```



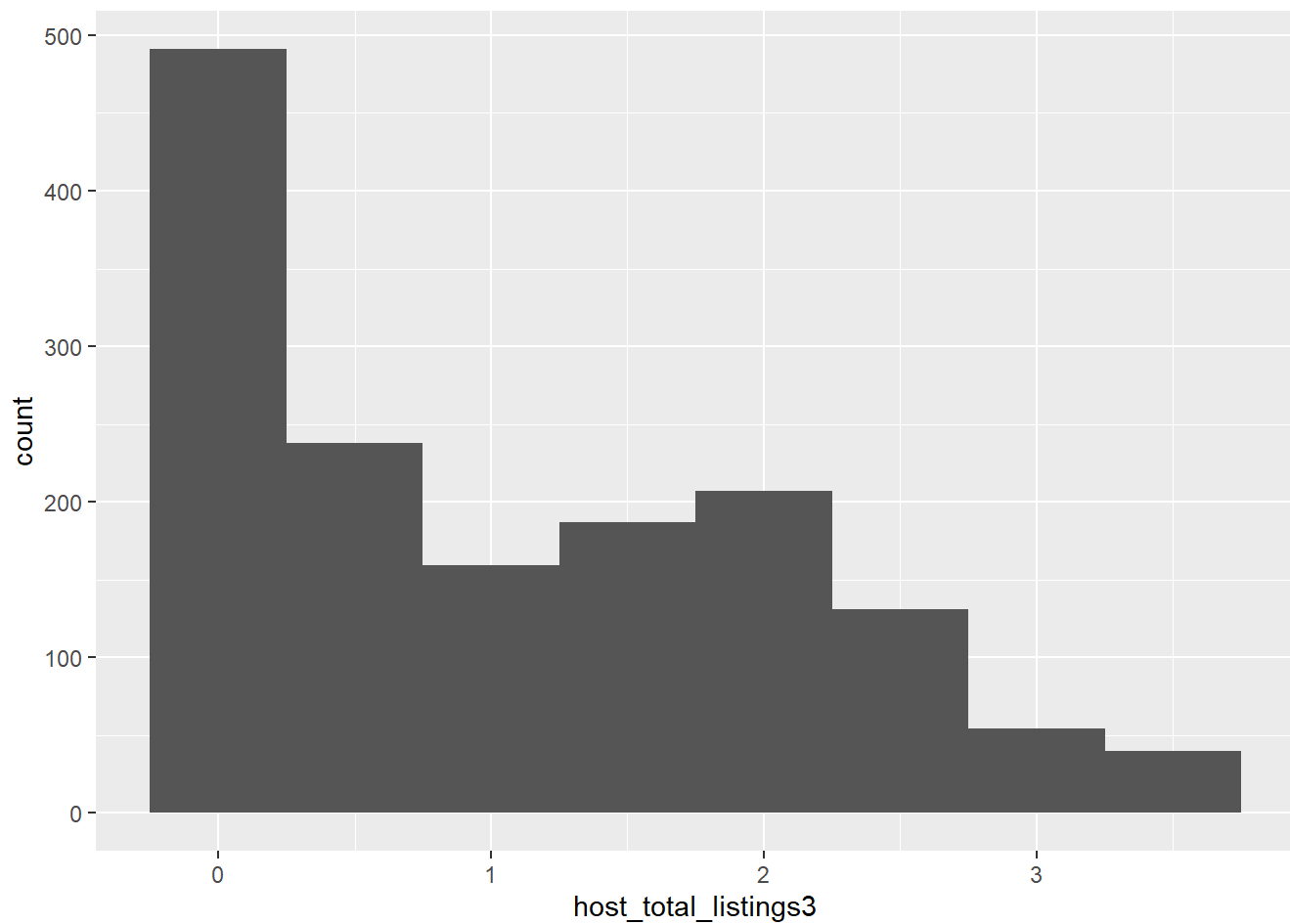
```
## data for host total listings is right skewed
## try sqrt transform

df3 = df_listings_price %>%
  mutate (host_total_listings2 = sqrt(host_total_listings),
          host_total_listings3 = log(host_total_listings))

ggplot(df3, aes(host_total_listings2)) + geom_histogram(binwidth = 1)
```



```
ggplot(df3, aes(host_total_listings3)) + geom_histogram(binwidth = 0.5)
```

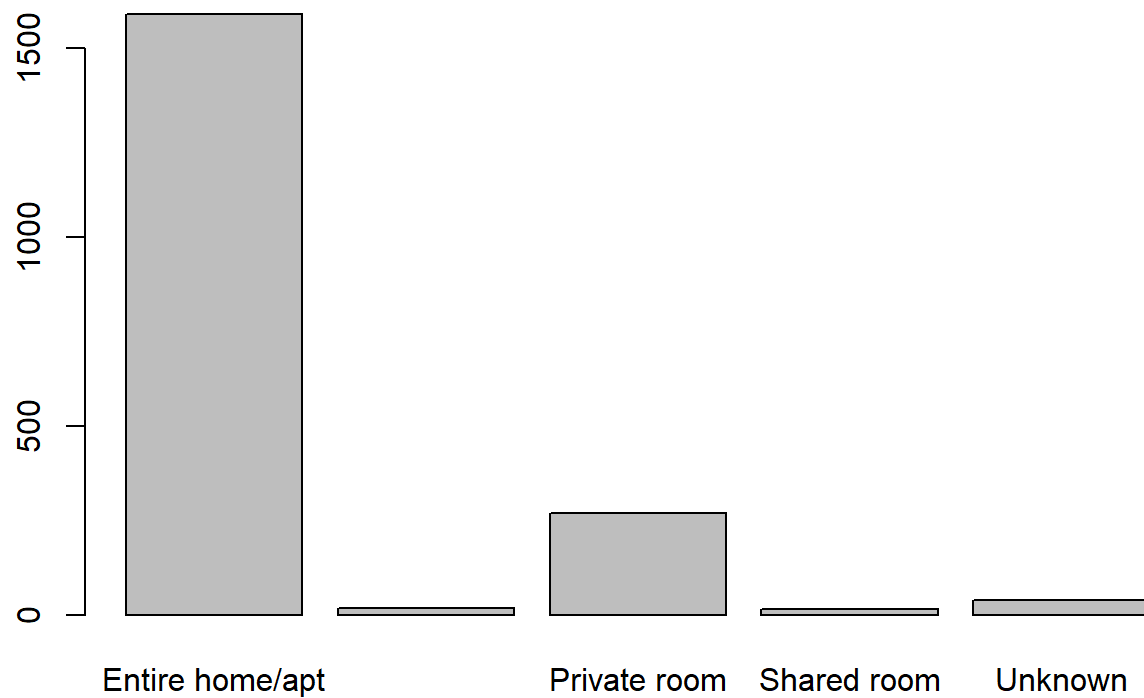


```
## Log transform was a little better
```

```
# 6) room type (already took care of the blanks)
```

```
## data type is character -> Lets convert this to a factor data type for better categorical analysis
```

```
barplot(table(df$room_type))
```



```
# some sampling error -> way to many entire home/apt compared to the others
```

```
# 7) accommodates -> is a integer data type
```

```
# first lets see the summary statistics
```

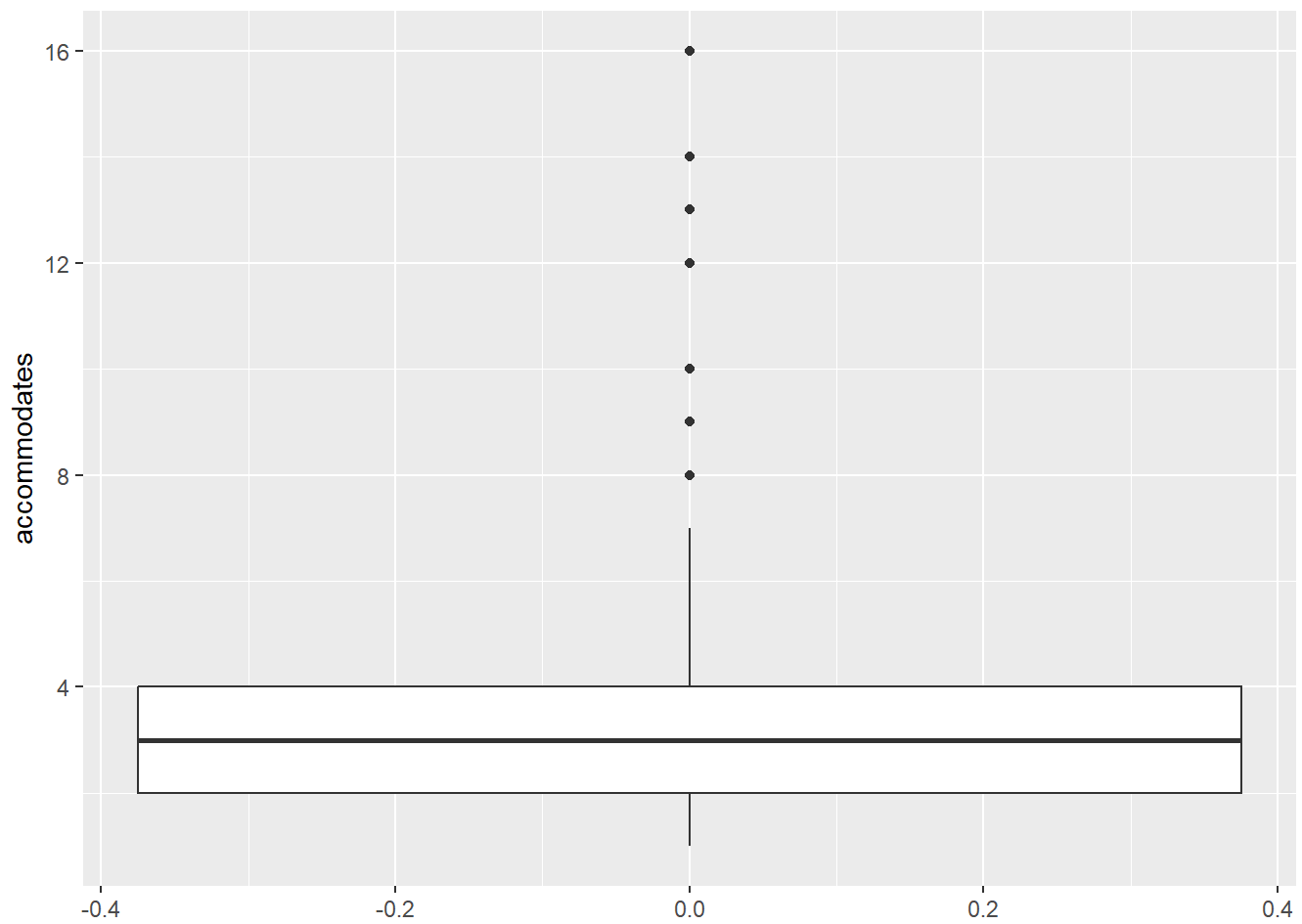
```
summary(df$accommodates)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      1.000   2.000   4.000   3.564   4.000  16.000
```

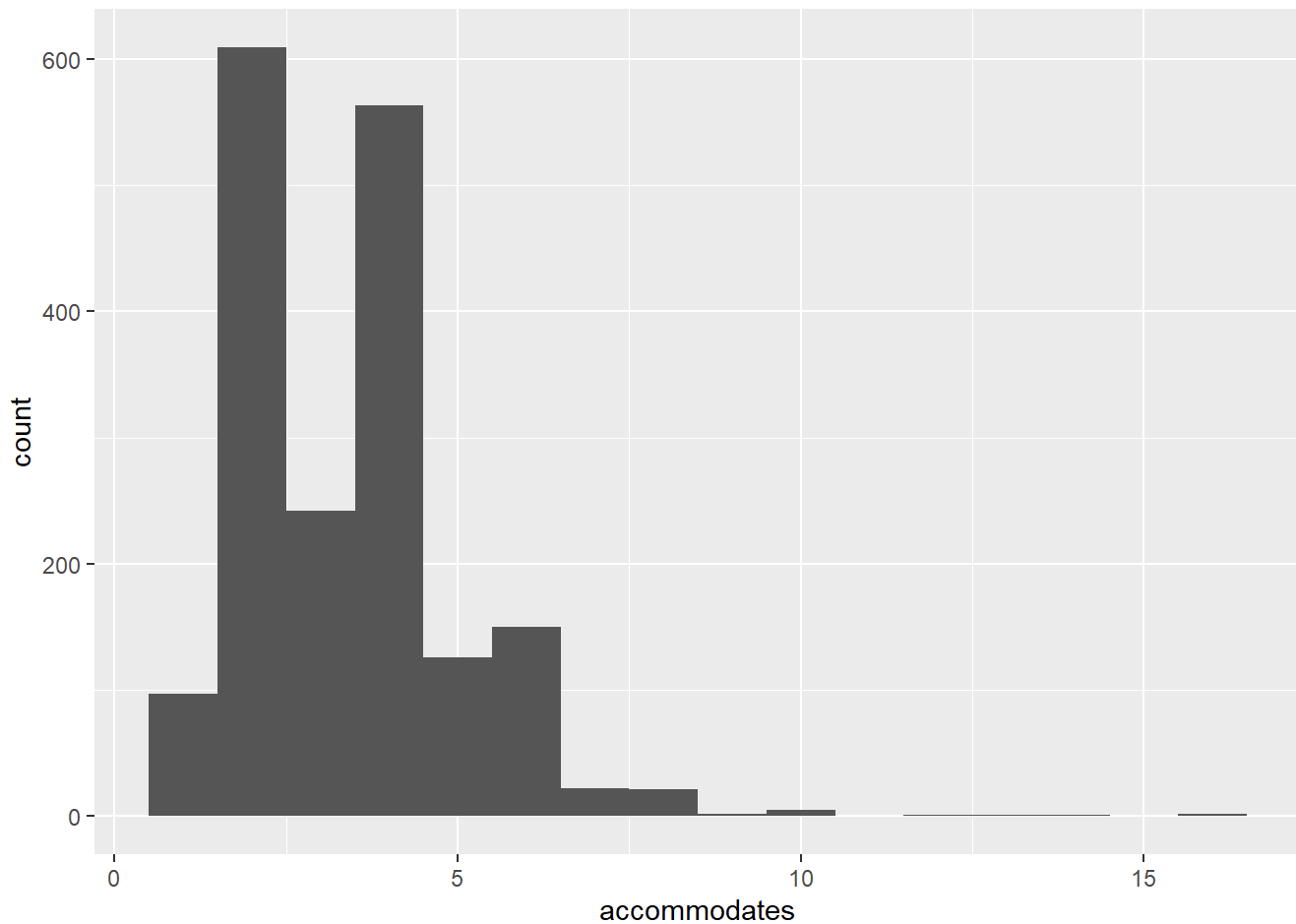
```
## check the distribution
```

```
df_price %>%
```

```
  ggplot(aes(accommodates)) + geom_boxplot() + coord_flip()
```

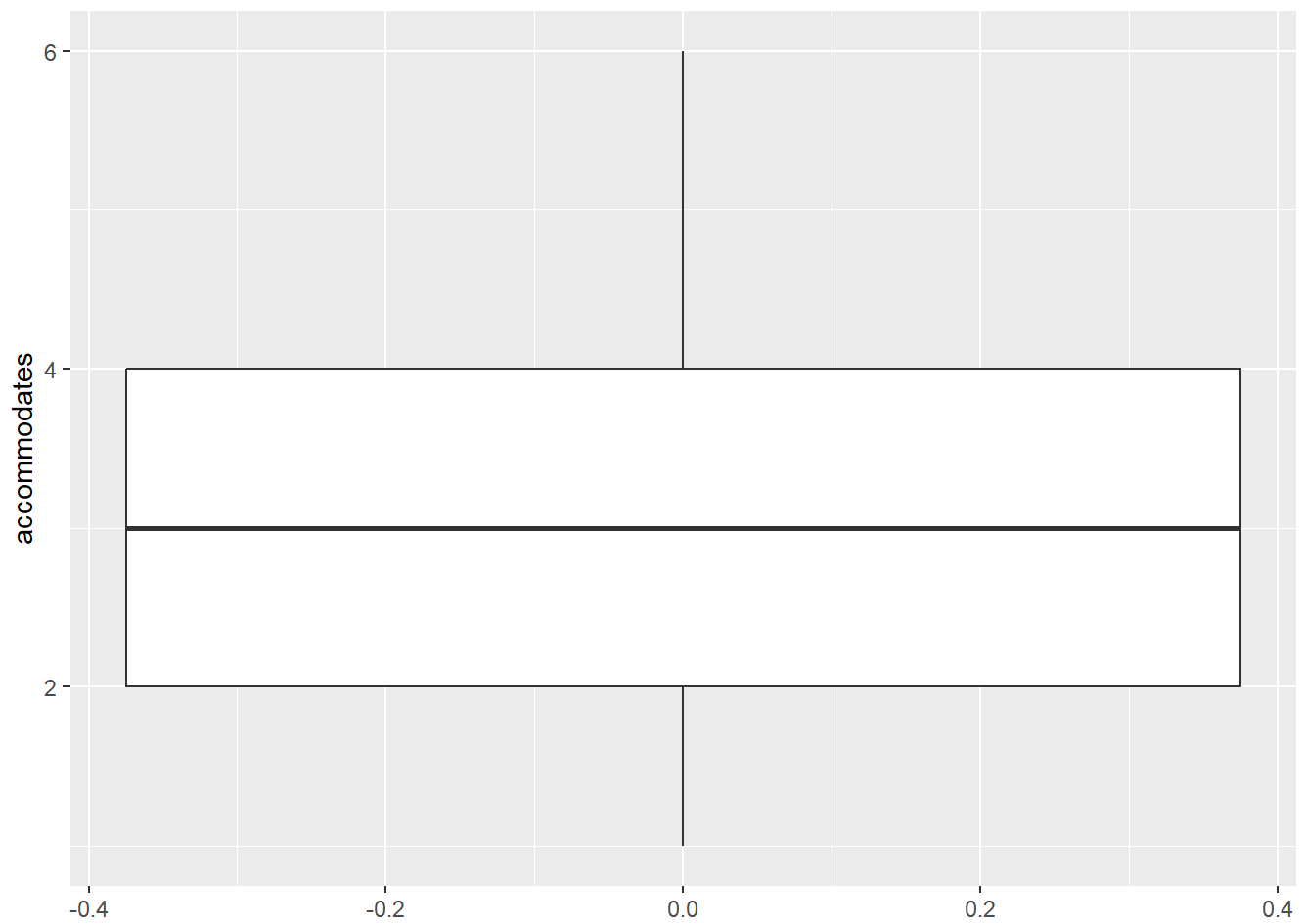


```
df_price %>%  
  ggplot(aes(accommodates)) + geom_histogram(binwidth = 1)
```

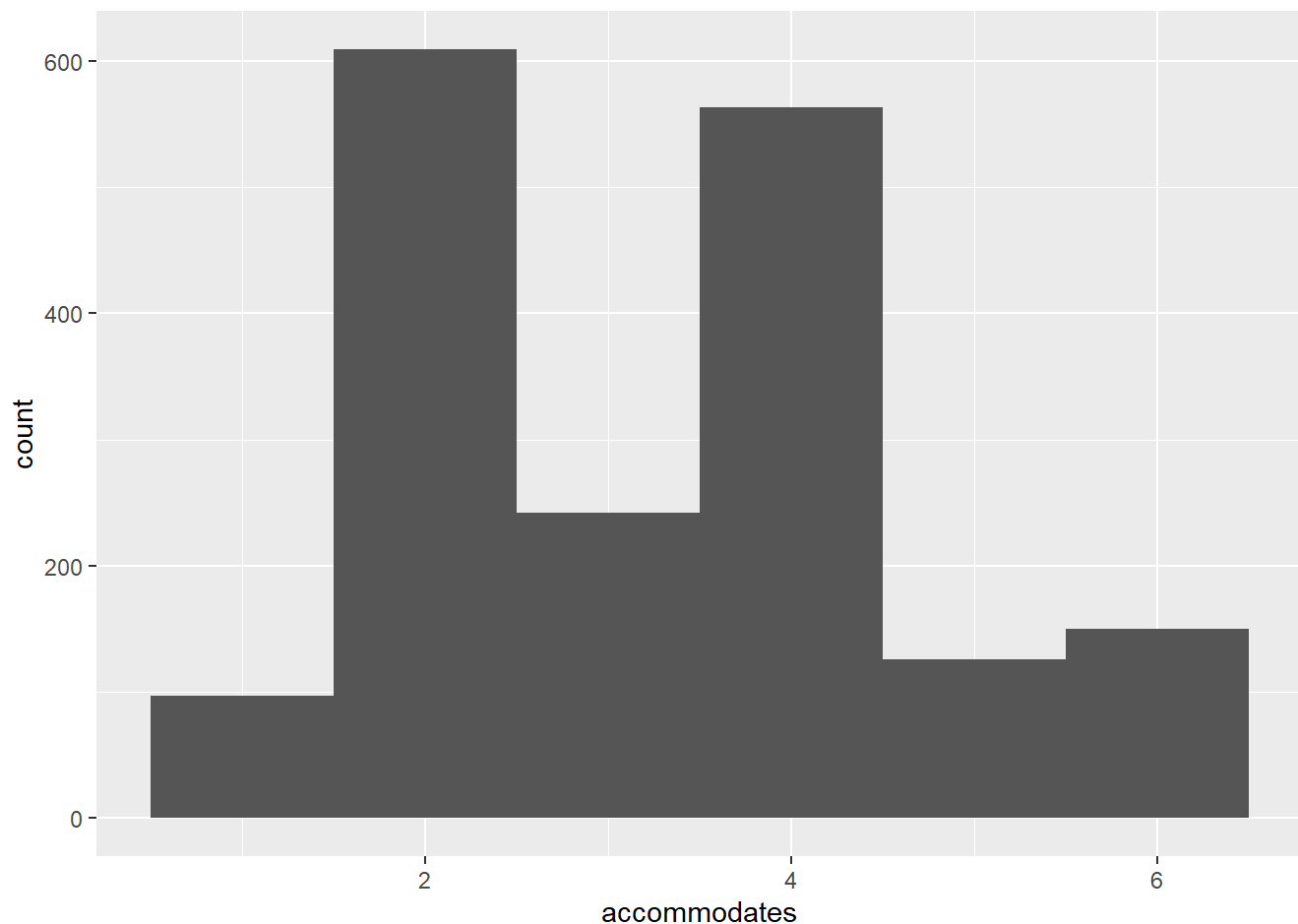



```
# bimodal distribution, slightly skewed to the right
IQR_ap = IQR(df_price$accommodates)
upper_a_p = quantile(df_price$accommodates, 0.75)
lower_a_p = quantile(df_price$accommodates, 0.25)
upper_outlier_ap = upper_a_p + 1.5*IQR_ap
lower_outlier_ap = lower_a_p - 1.5*IQR_ap

df_a_p = df_price %>%
  filter(accommodates > lower_outlier_ap & accommodates < upper_outlier_ap)
df_a_p %>%
  ggplot(aes(accommodates)) + geom_boxplot() + coord_flip()
```



```
df_a_p %>%  
  ggplot(aes(accommodates)) + geom_histogram(binwidth = 1)
```

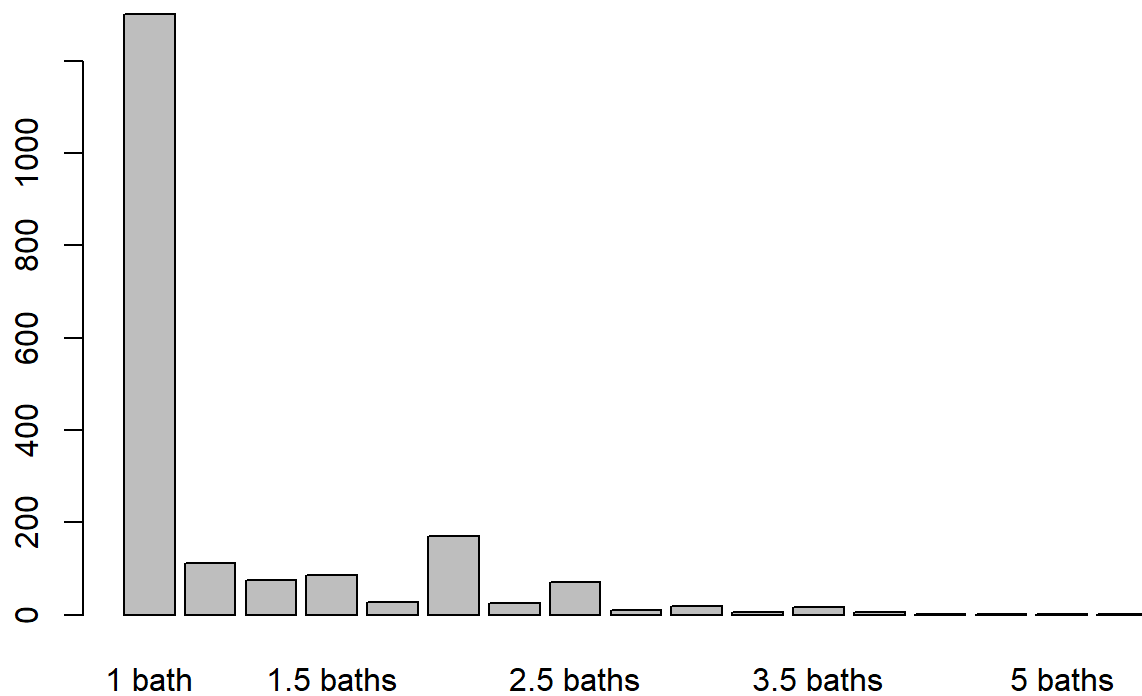


```
# 8) bathrooms is character data type -> convert to factor
```

```
bathroom_freq = table(df$bathrooms)
prop.table(bathroom_freq)
```

```
##
##          1 bath  1 private bath  1 shared bath    1.5 baths
##    0.6737441740  0.0580010357  0.0383221129  0.0445365096
## 1.5 shared baths      2 baths  2 shared baths    2.5 baths
##    0.0145002589  0.0885551528  0.0134645262  0.0367685137
## 2.5 shared baths      3 baths  3 shared baths    3.5 baths
##    0.0046607975  0.0098394614  0.0031071983  0.0088037286
##          4 baths      4.5 baths 4.5 shared baths    5 baths
##    0.0031071983  0.0010357328  0.0005178664  0.0005178664
## Shared half-bath
##    0.0005178664
```

```
barplot(bathroom_freq)
```



```
## Lots of 1 bath -> highly imbalanced sampling
```

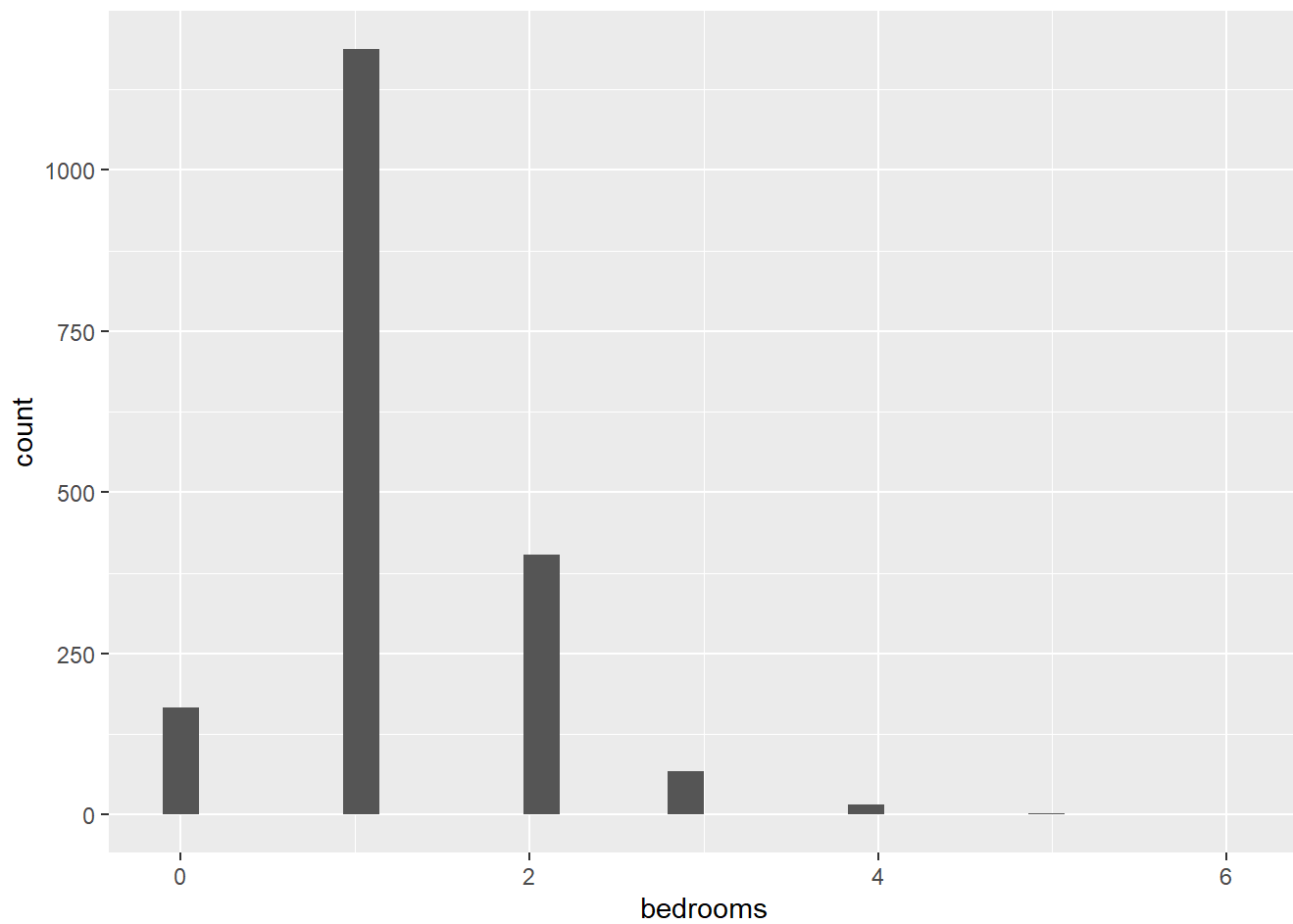
```
# 9) bedrooms -> int data type (makes sense, but could possibly use this as a categorical variable as well)
```

```
summary(df_price$bedrooms)
```

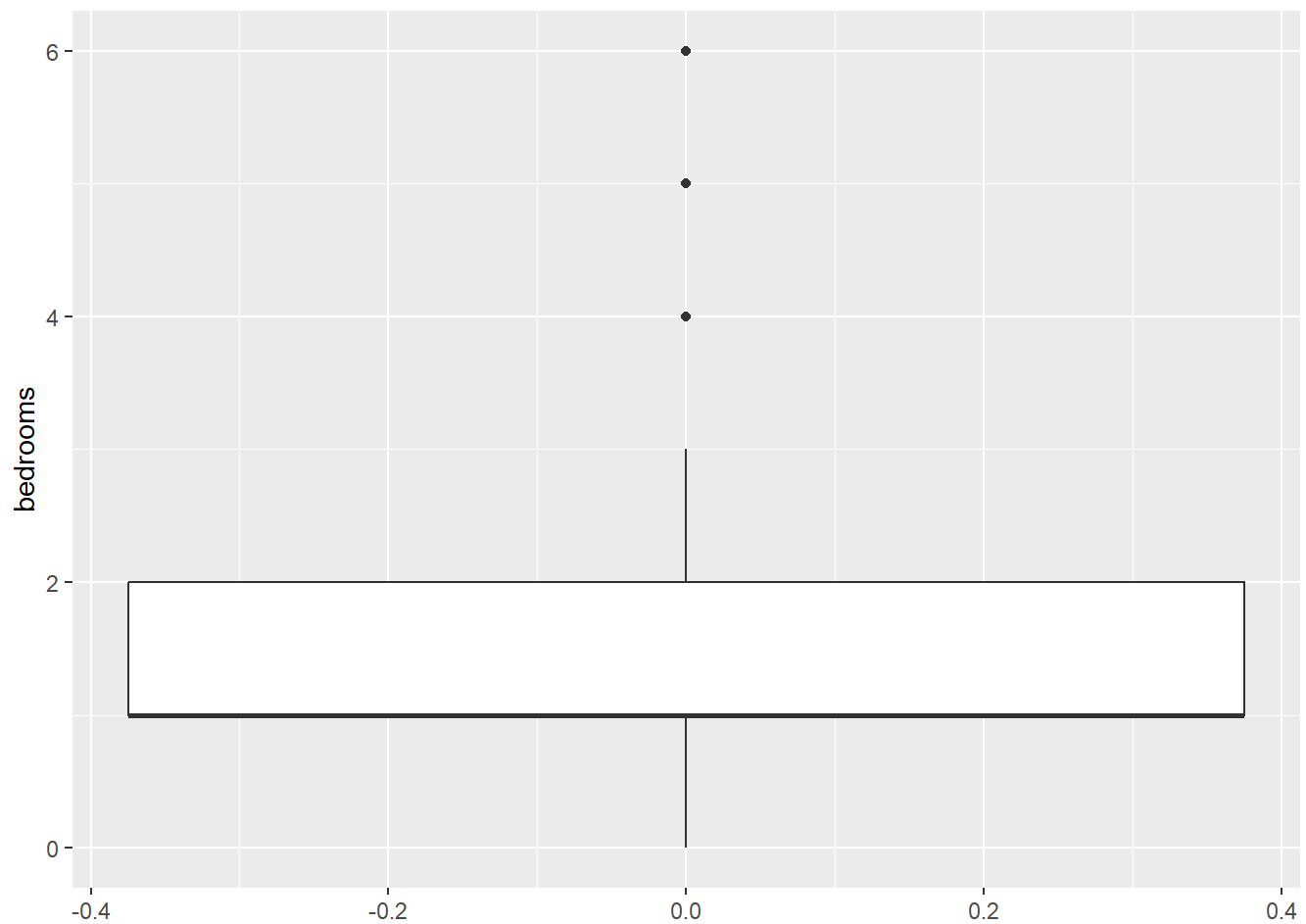
```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      0.000   1.000   1.000   1.234   2.000   6.000
```

```
ggplot(df_price, aes(bedrooms)) + geom_histogram()
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



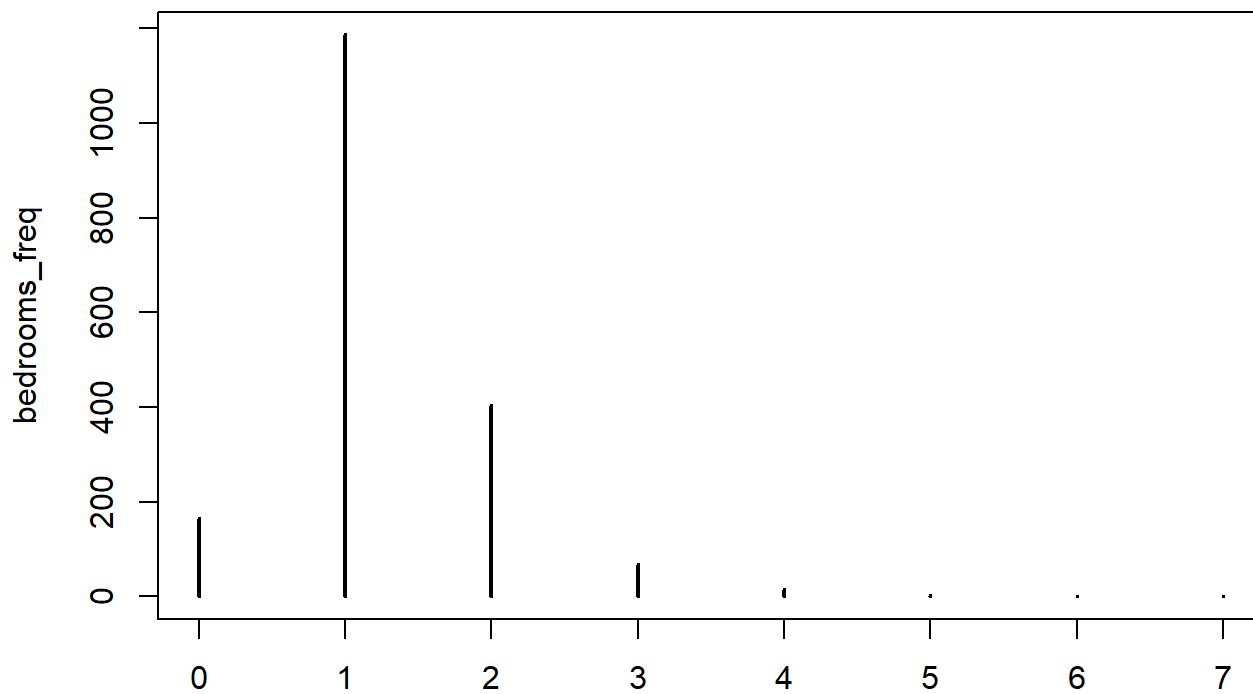
```
ggplot(df_price, aes(bedrooms)) + geom_boxplot() + coord_flip()
```



```
bedrooms_freq = table(df_price$bedrooms_f)
prop.table(bedrooms_freq)
```

```
##
##           0           1           2           3           4           5
## 0.0901194354 0.6444082519 0.2187839305 0.0369163952 0.0081433225 0.0010857763
##           6           7
## 0.0005428882 0.0000000000
```

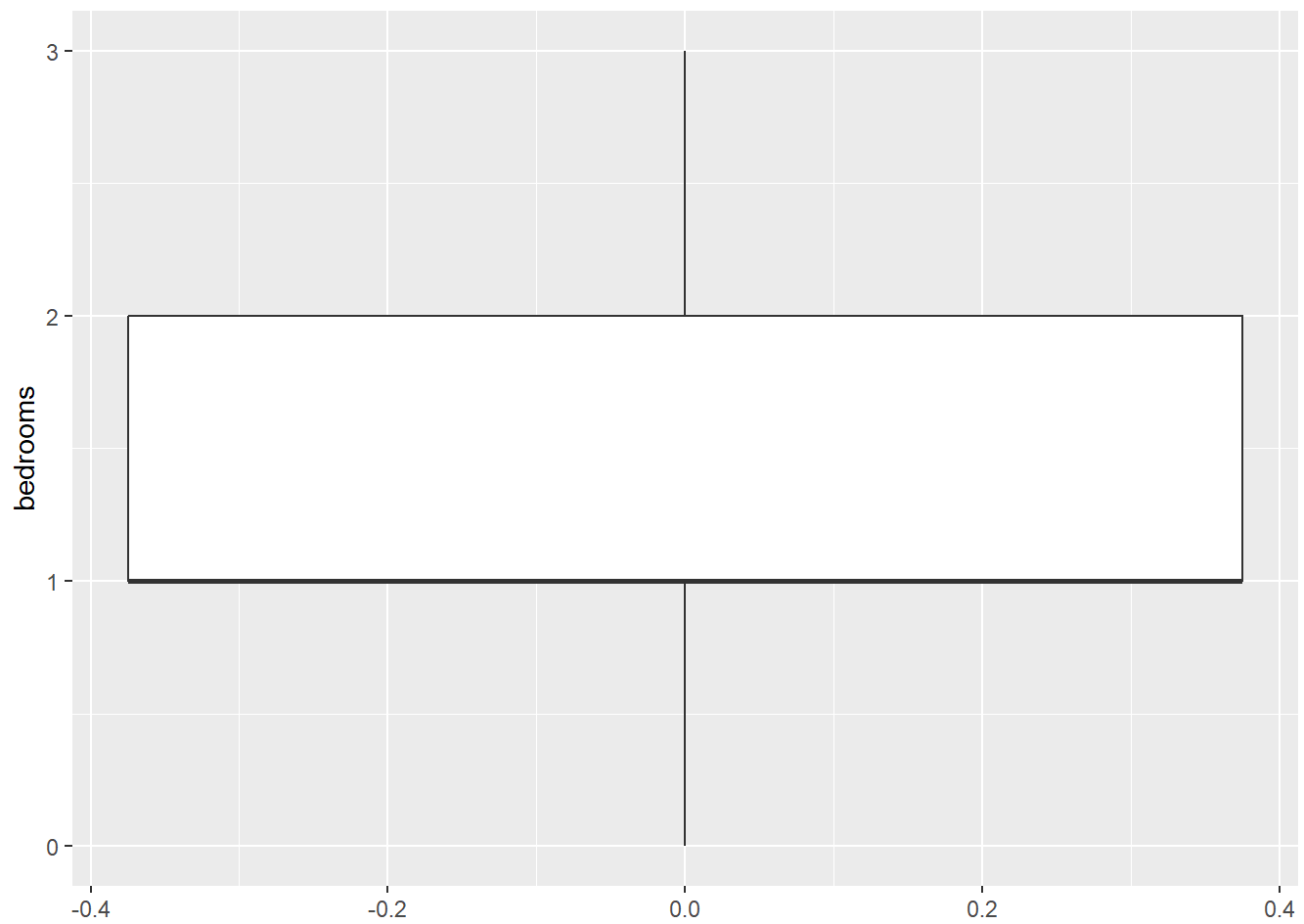
```
plot(bedrooms_freq)
```



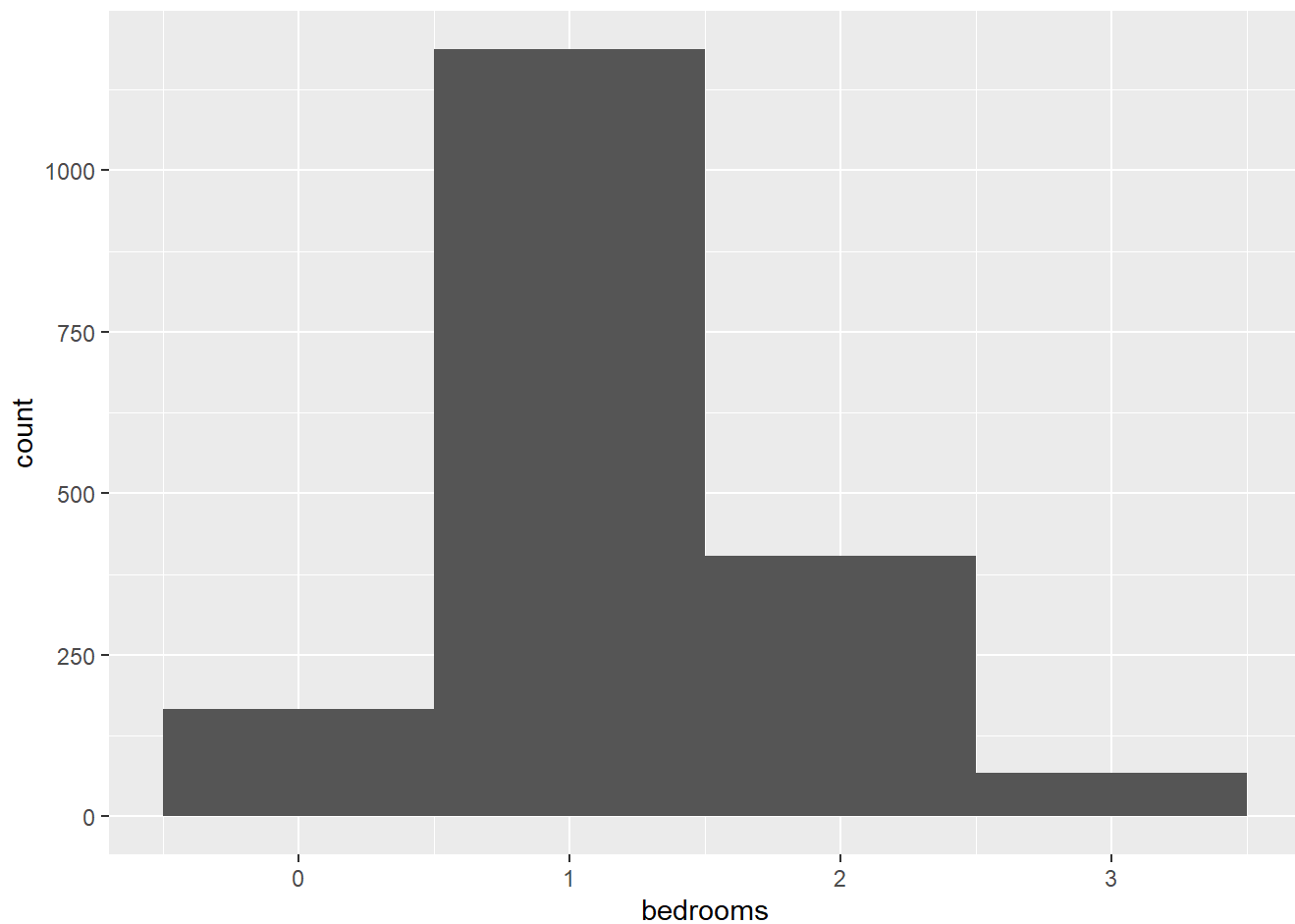
```
IQR_pb = IQR(df_price$bedrooms)
upper_pb = quantile(df_price$bedrooms, 0.75)
lower_pb = quantile(df_price$bedrooms, 0.25)
upper_outlier_pb = upper_pb + 1.5*IQR_pb
lower_outlier_pb = lower_pb - 1.5*IQR_pb

df_b_p = df_price %>%
  filter( bedrooms > lower_outlier_pb & bedrooms < upper_outlier_pb )

df_b_p %>%
  ggplot(aes(bedrooms)) + geom_boxplot() + coord_flip()
```



```
df_b_p %>%  
  ggplot(aes(bedrooms)) + geom_histogram(binwidth = 1)
```

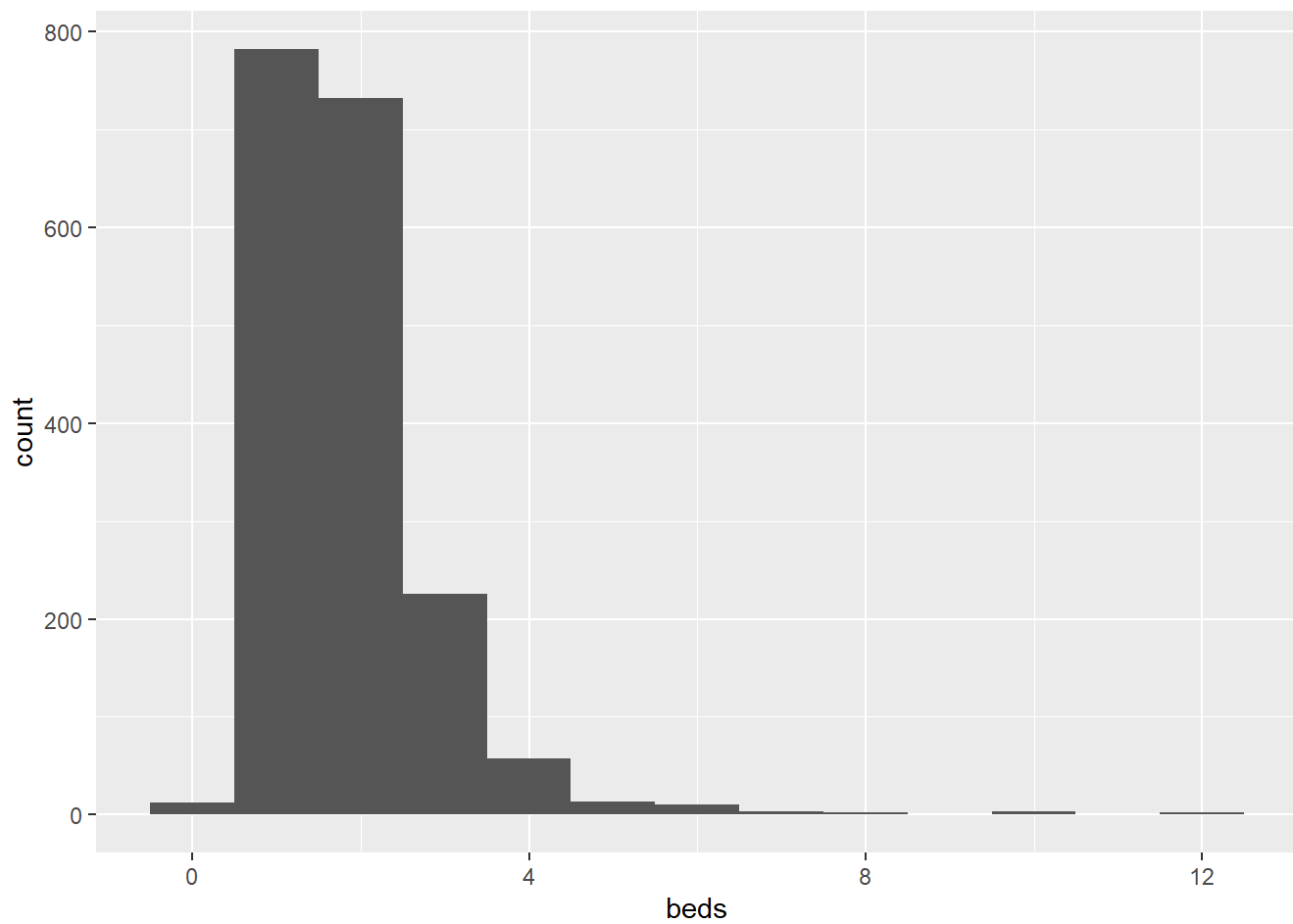



```
# 10) beds -> double data type
```

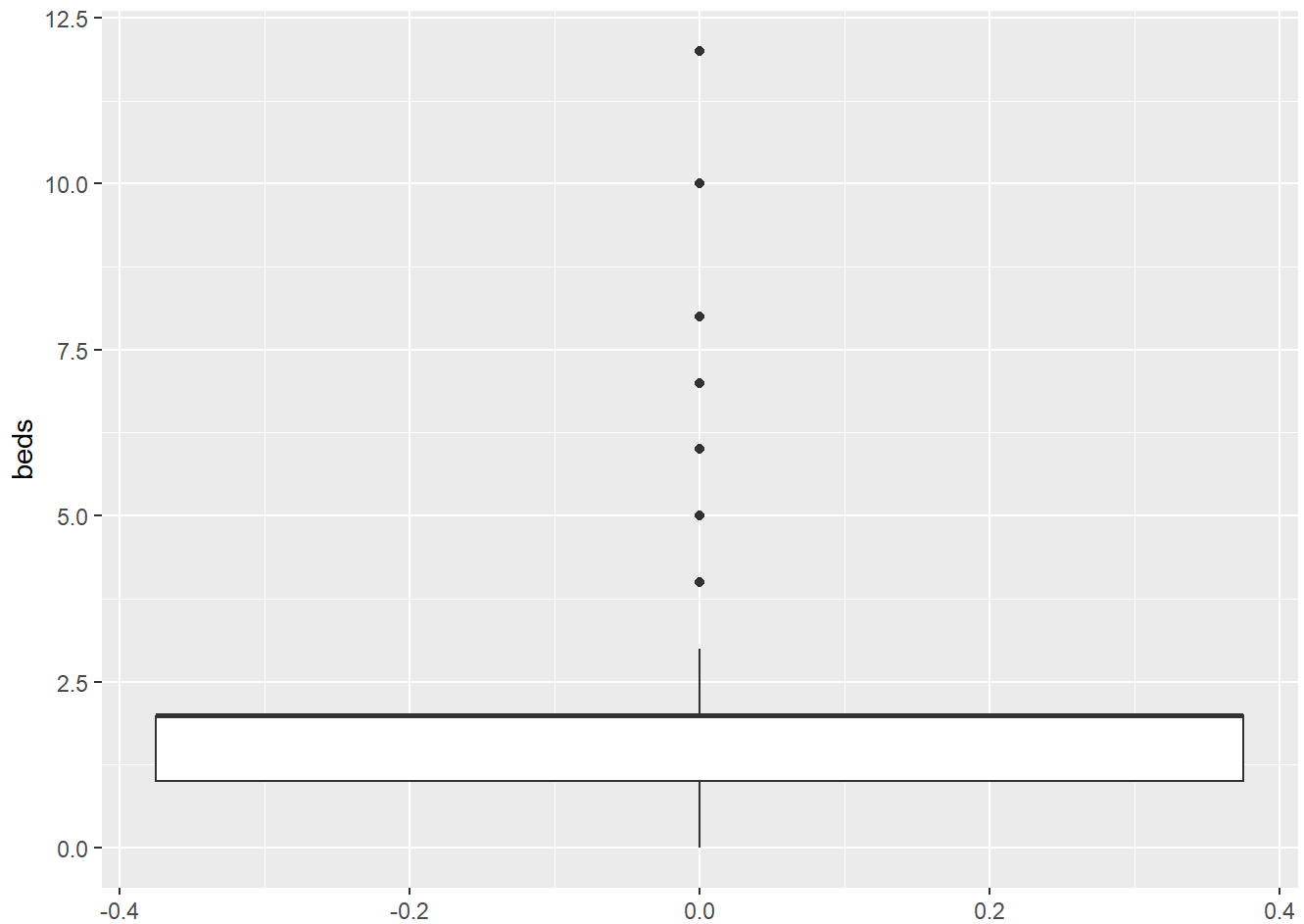
```
summary(df_price$beds)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.000   1.000   2.000   1.828   2.000  12.000
```

```
ggplot(df_price, aes(beds)) + geom_histogram(binwidth = 1)
```



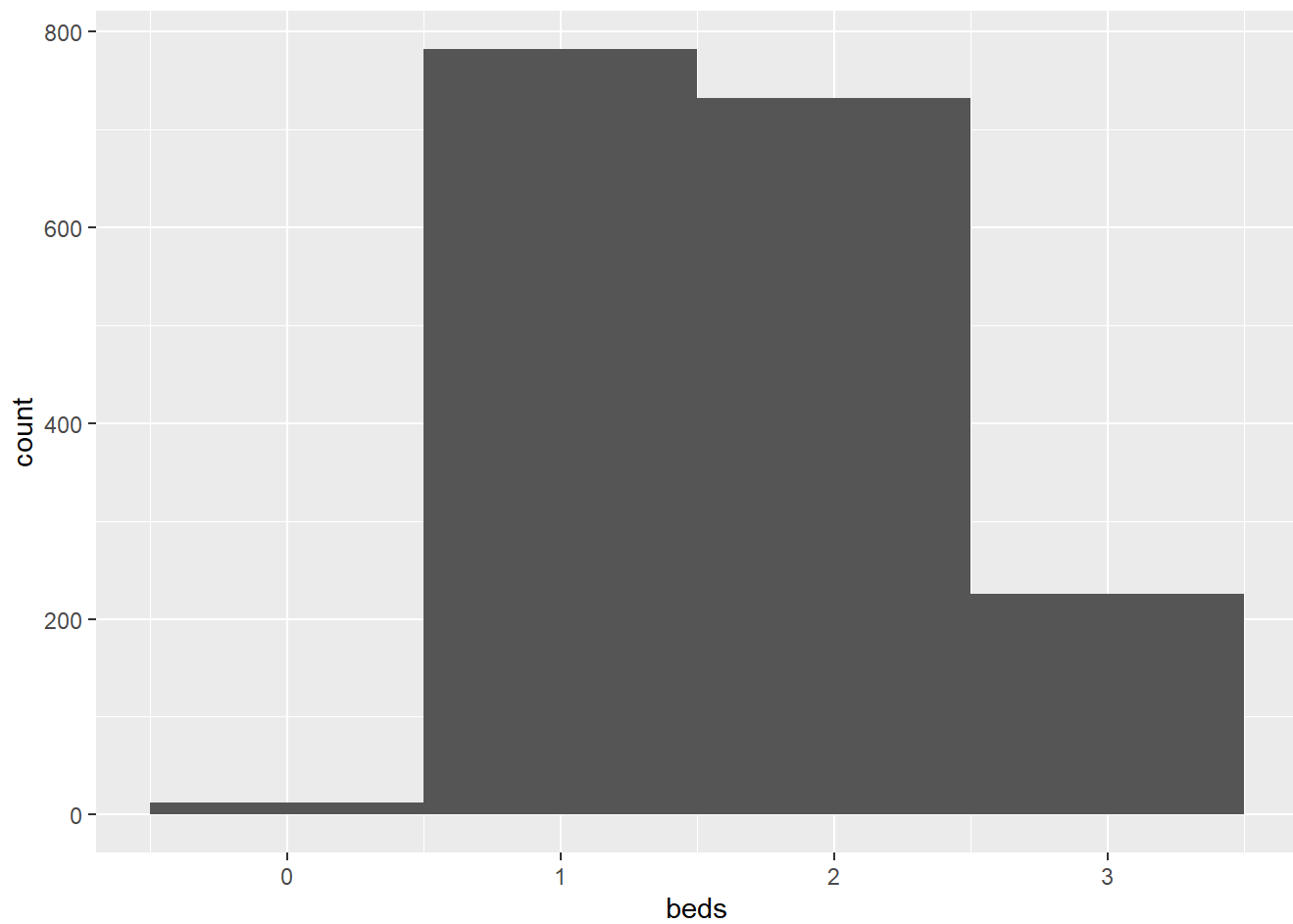
```
ggplot(df_price, aes(beds)) + geom_boxplot() + coord_flip()
```



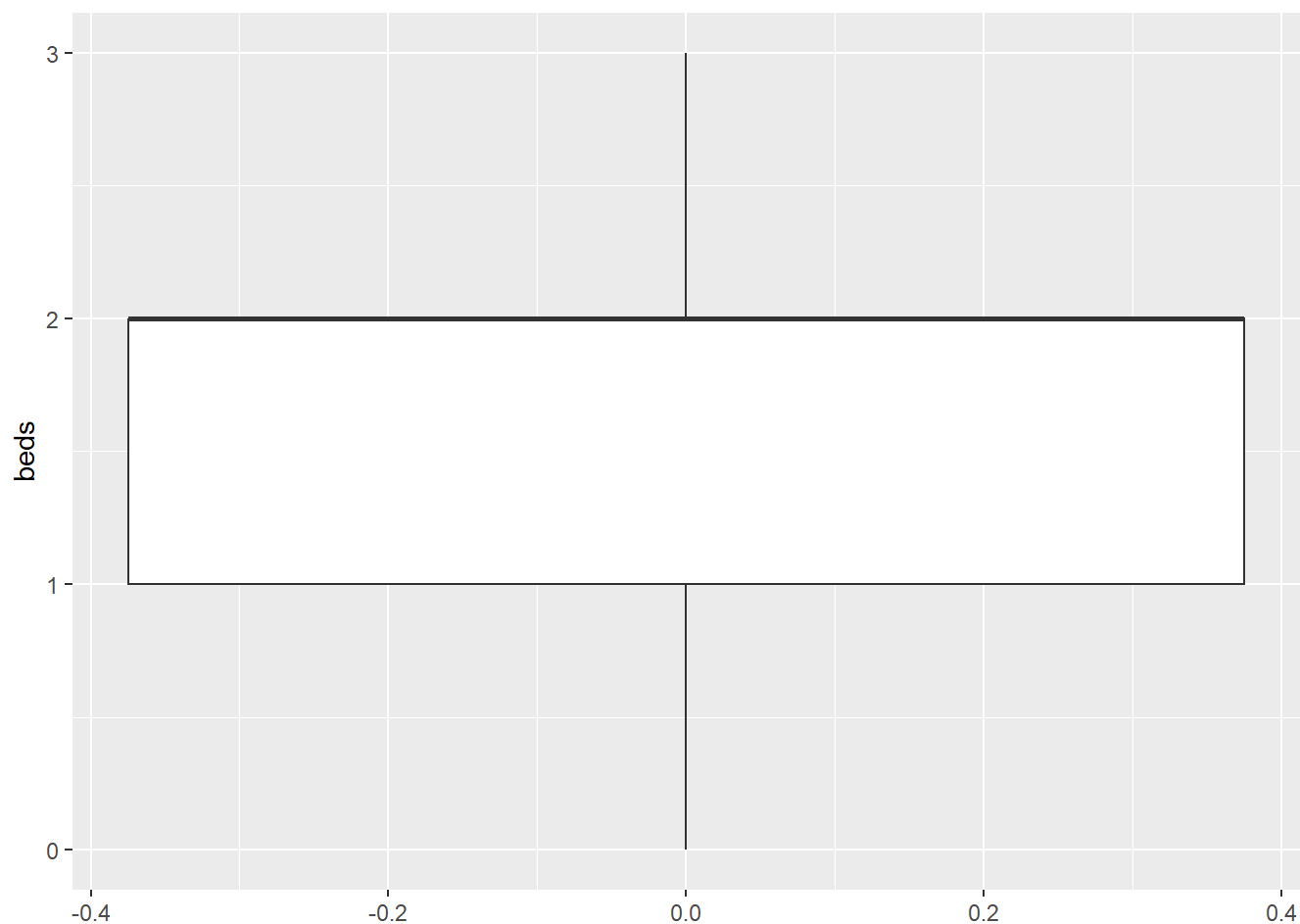
```
IQR_bed_p = IQR(df_price$beds)
upper_bed_p = quantile(df_price$beds , 0.75)
lower_bed_p = quantile(df_price$beds, 0.25)
upper_outlier_bed_p= upper_bed_p + 1.5 * IQR_bed_p
lower_outlier_bed_p = lower_bed_p - 1.5 * IQR_bed_p

df_bed_p = df_price %>%
  filter(beds > lower_outlier_bed_p & beds < upper_outlier_bed_p)

df_bed_p %>%
  ggplot(aes(beds)) + geom_histogram(binwidth = 1)
```



```
df_bed_p %>%  
  ggplot(aes(beds)) + geom_boxplot()+ coord_flip()
```

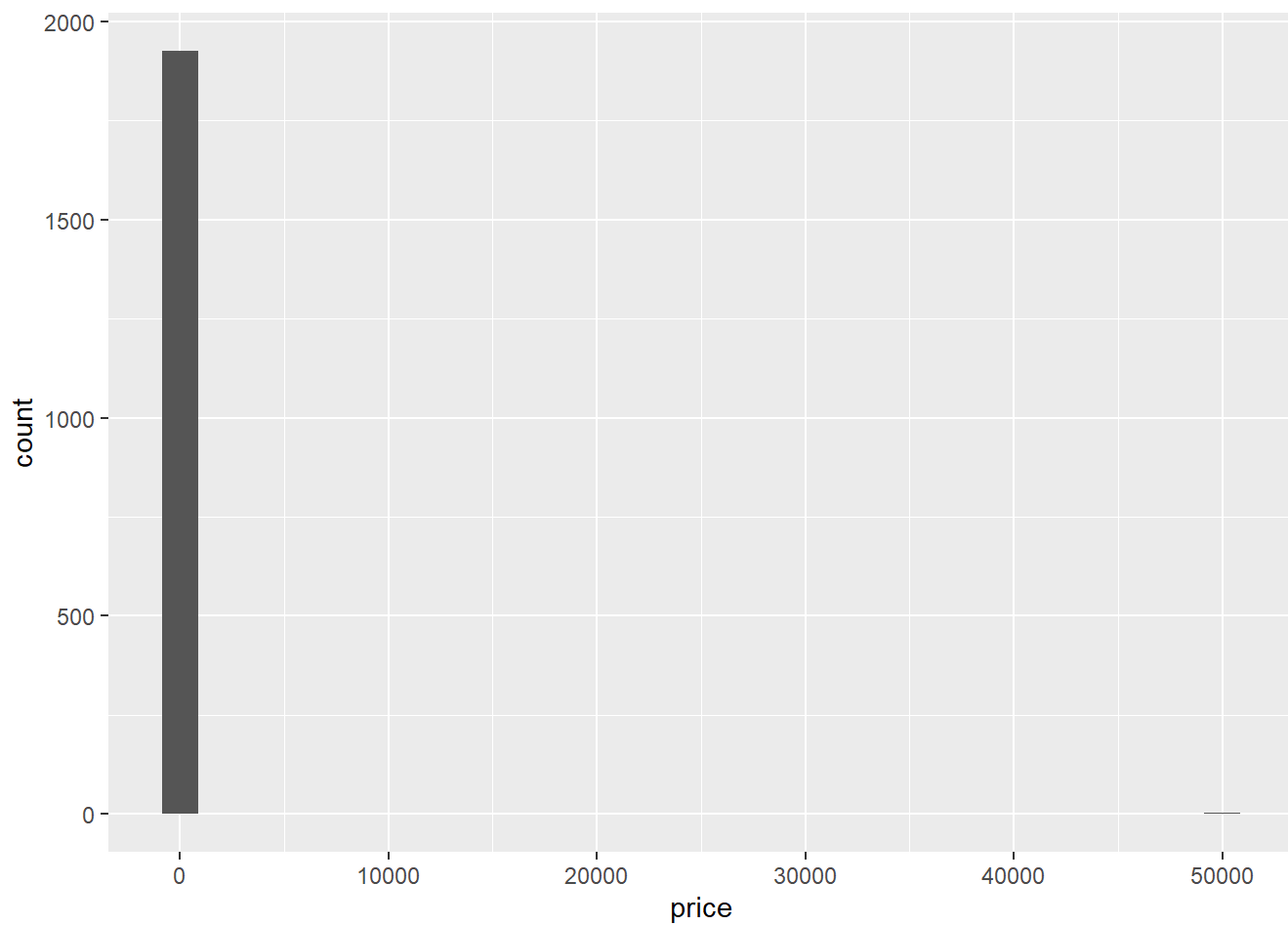


```
# 11) price is a double -> find summary statistics
summary(df$price)
```

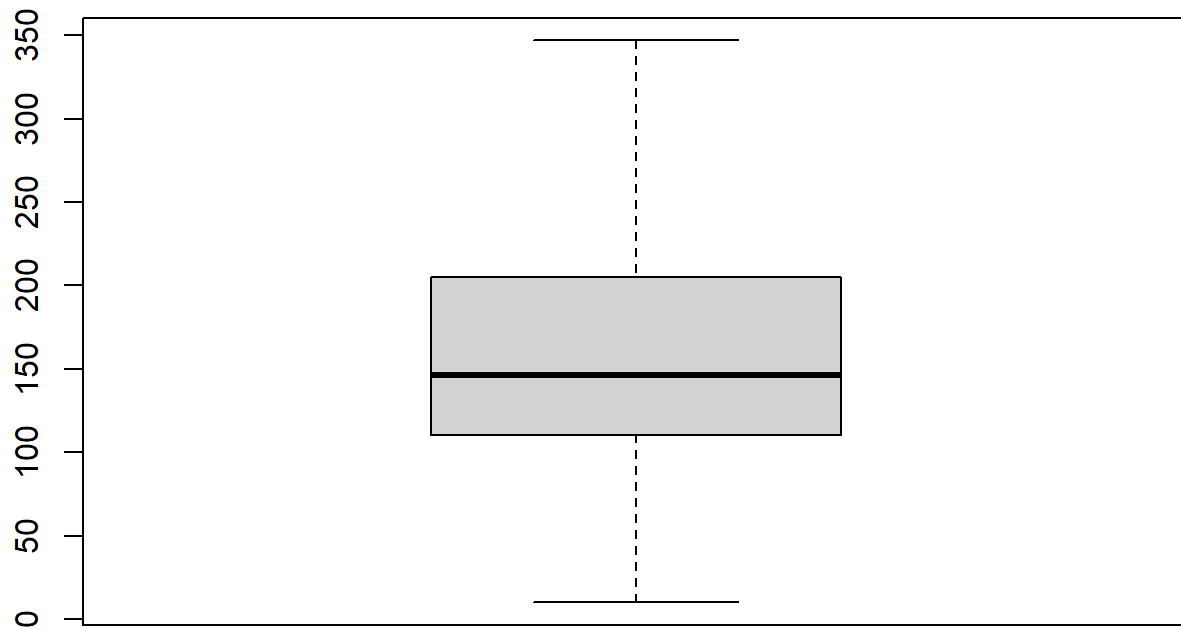
```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      10.0   110.0   146.0   272.4   205.0 50000.0
```

```
ggplot(df, aes(price)) + geom_histogram()
```

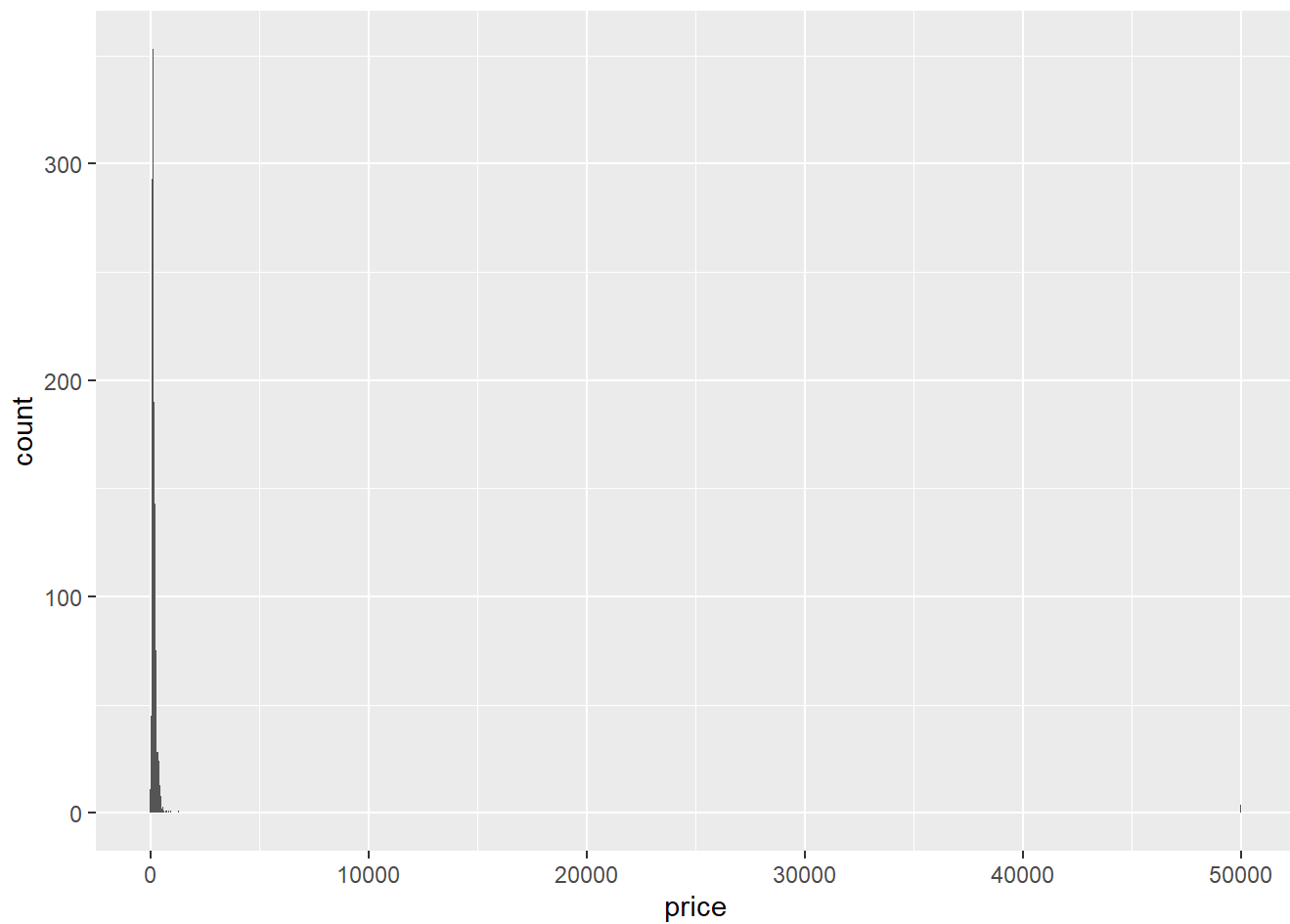
```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



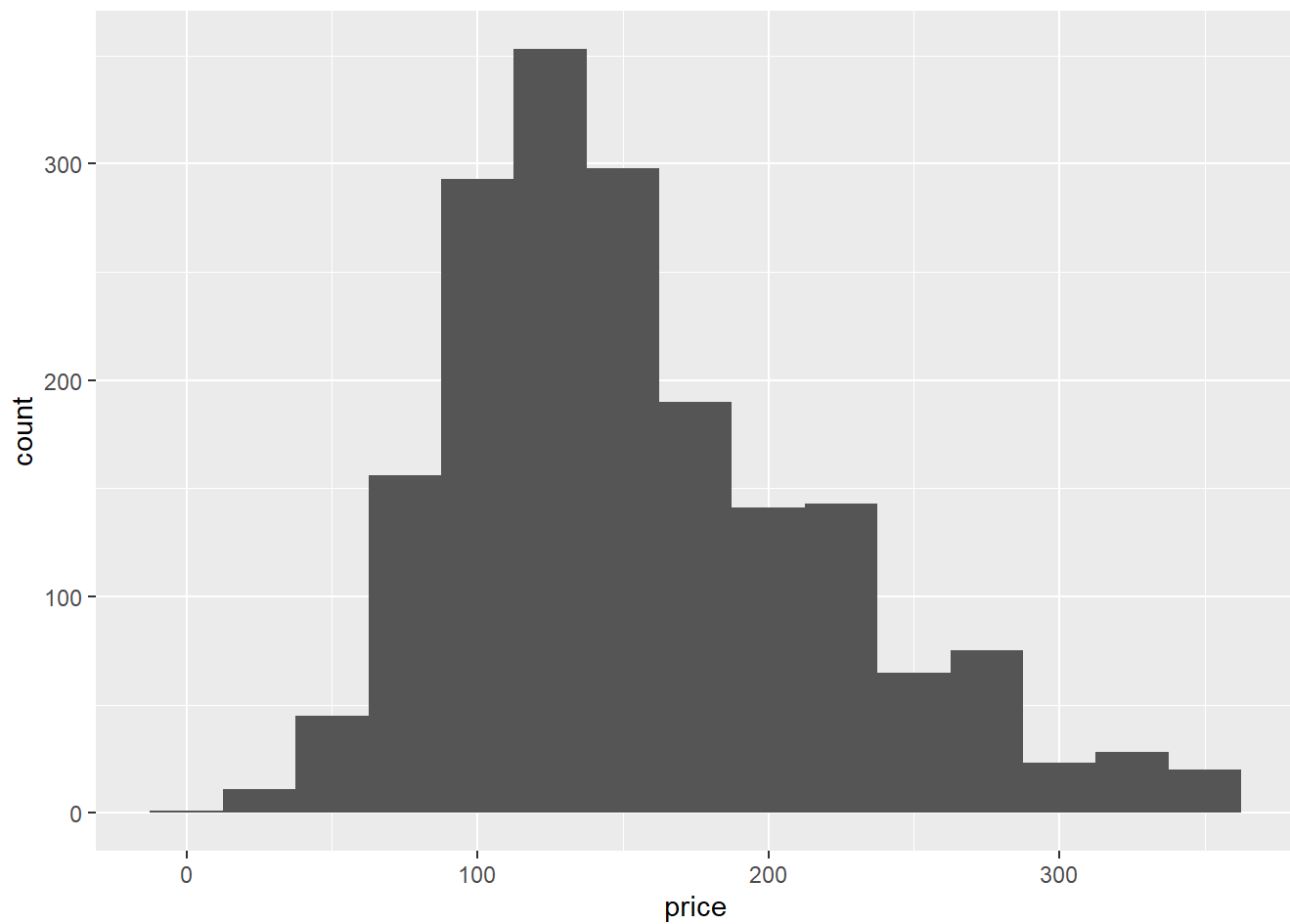
```
# there was a few data points that are clear outliers < 50000  
# simple test to first visualize the data in a box plot with these outliers removed  
boxplot(df$price , outline = F)
```



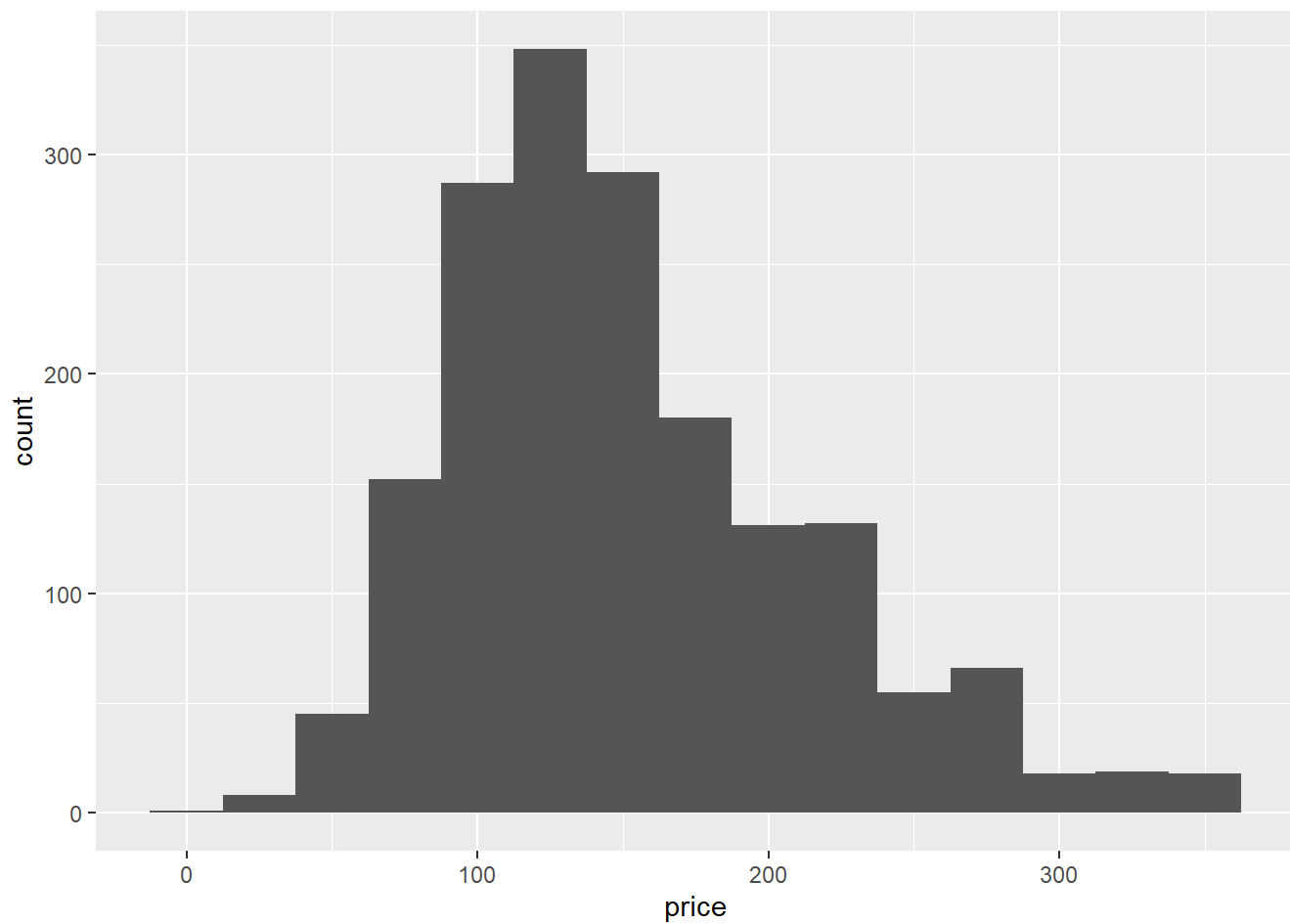
```
ggplot(df , aes(price)) + geom_histogram(binwidth = 25)
```



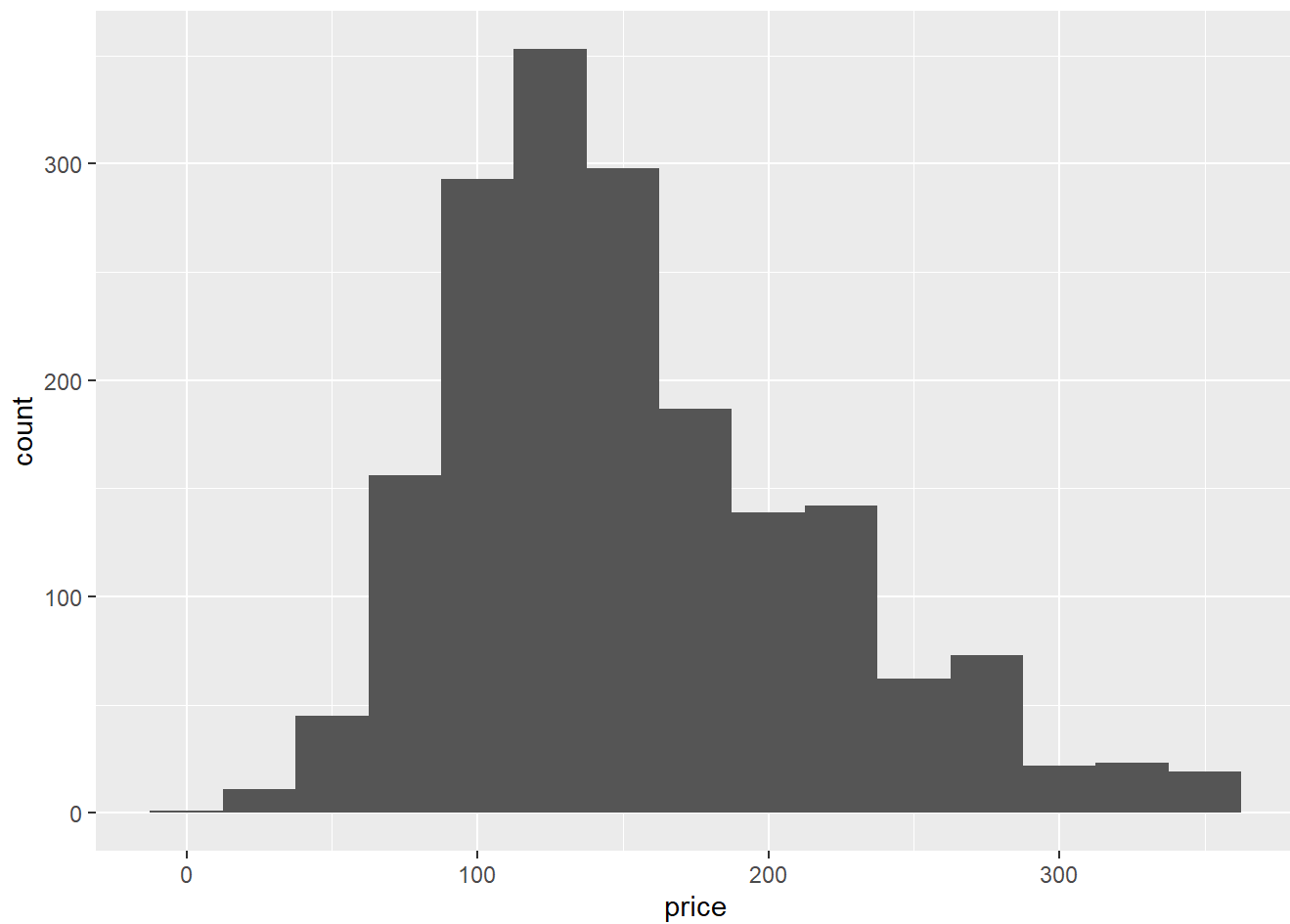
```
ggplot(df_price , aes(price)) + geom_histogram(binwidth = 25)
```

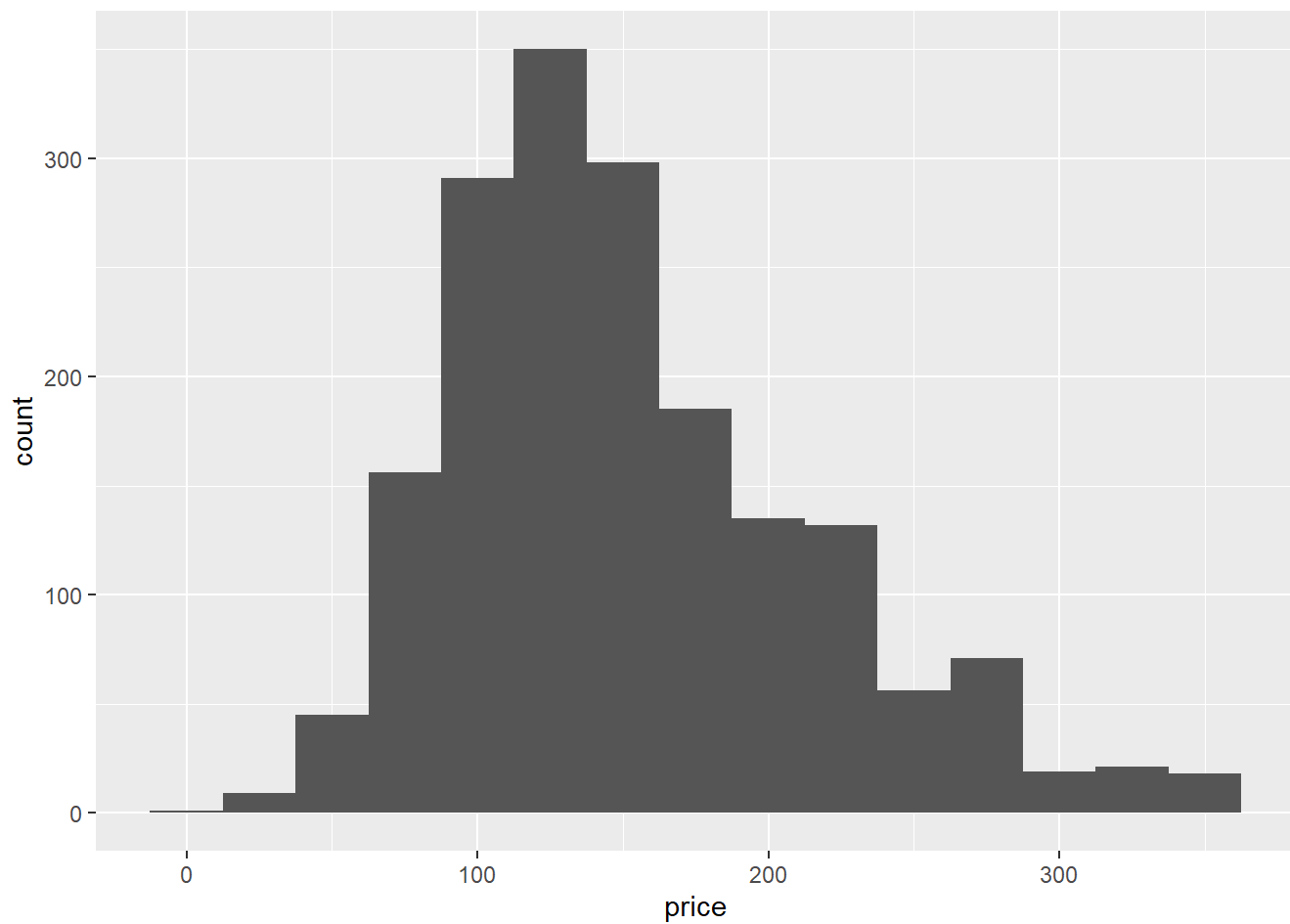
```
ggplot(df_bed_p, aes(price)) + geom_histogram(binwidth = 25)
```



```
ggplot(df_b_p, aes(price)) + geom_histogram(binwidth = 25)
```



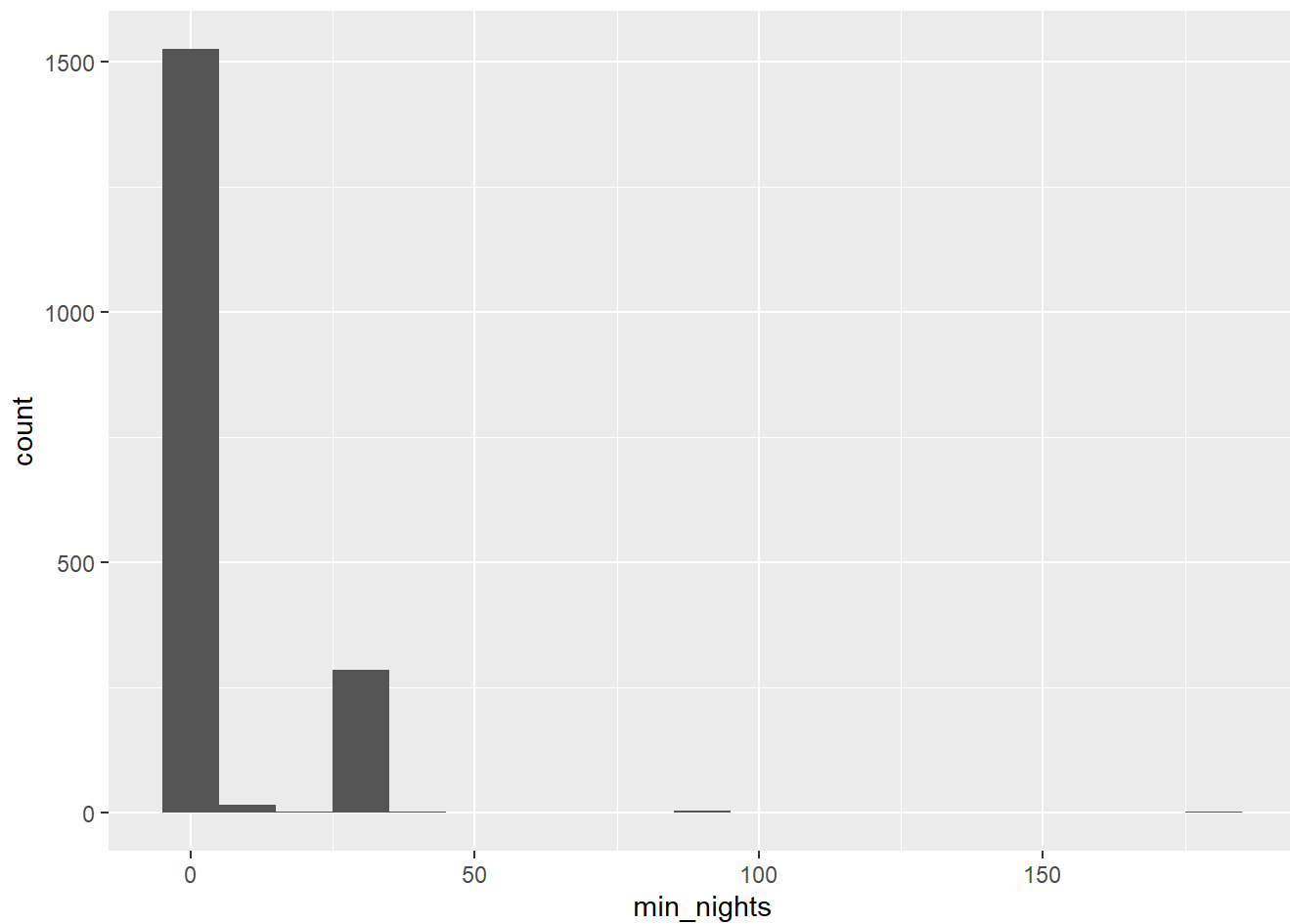
```
ggplot(df_a_p, aes(price)) + geom_histogram(binwidth = 25)
```



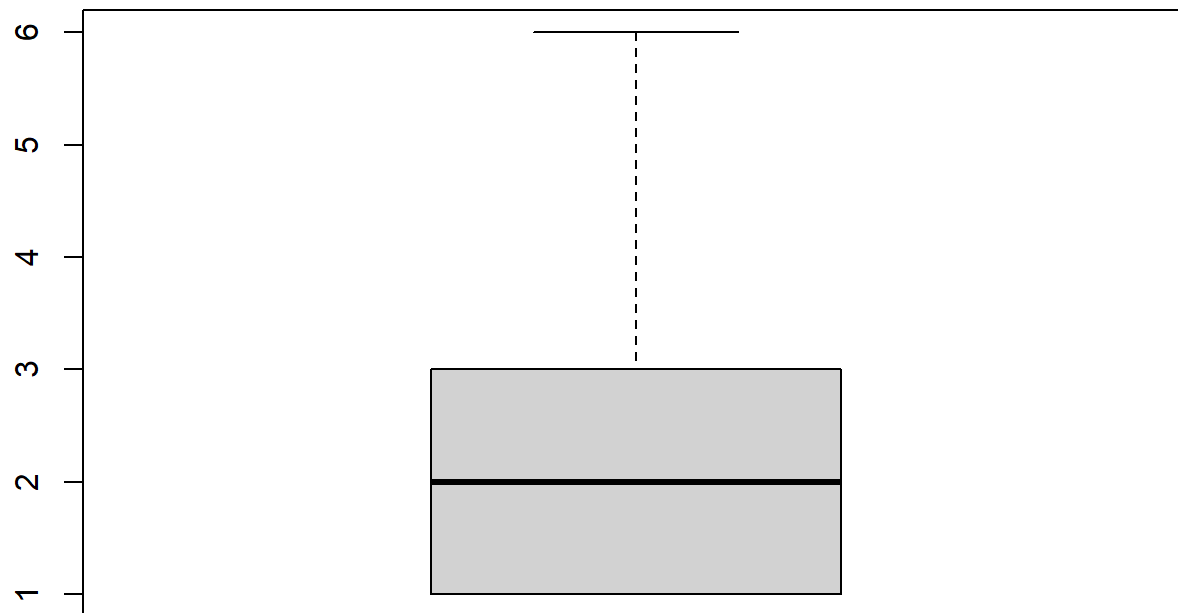
```
# 12) min_nights is an integer data type  
summary(df_price$min_nights)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   
##   1.000   1.000   2.000   7.104   3.000  182.000
```

```
ggplot(df_price, aes(min_nights)) + geom_histogram(binwidth = 10)
```



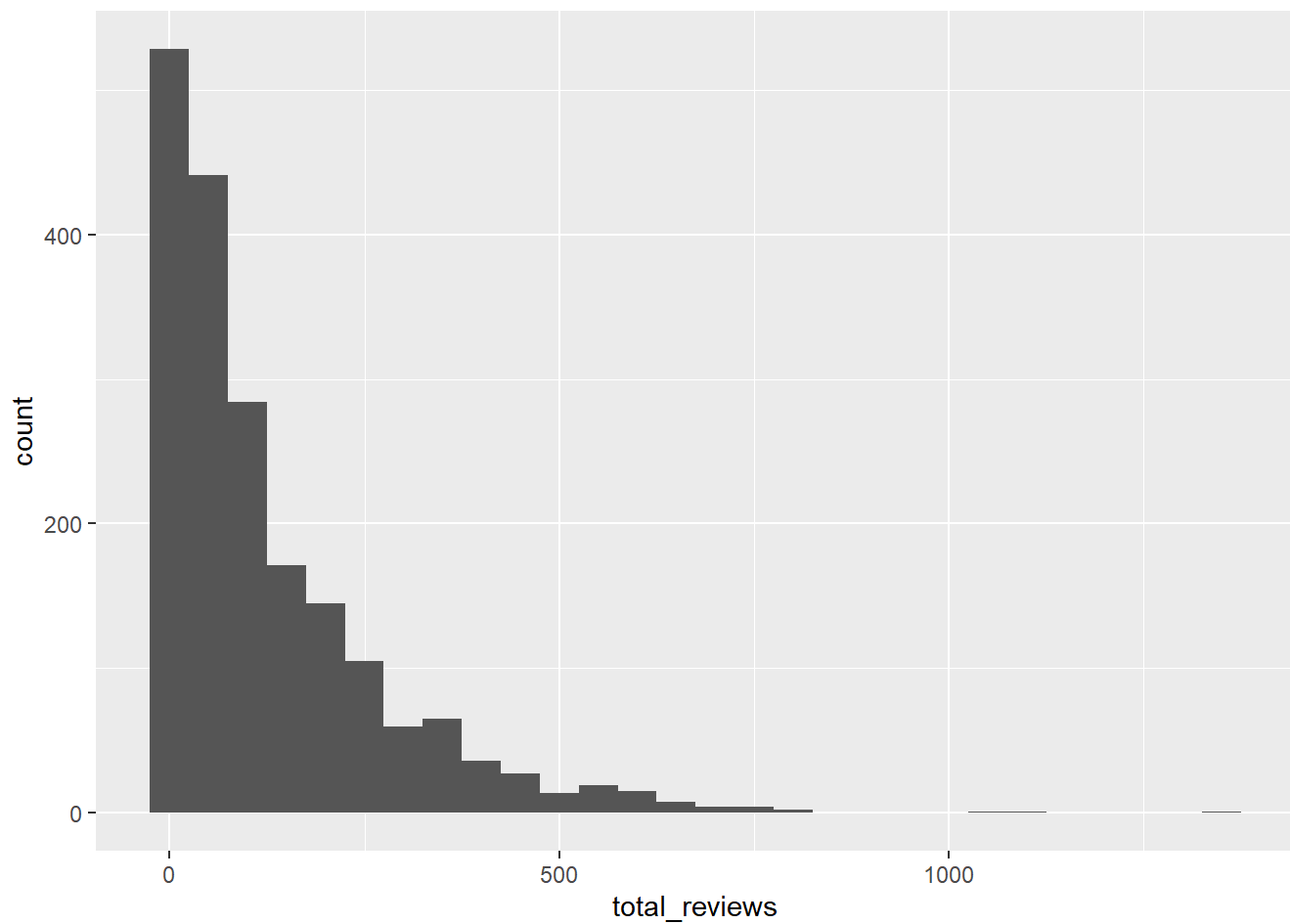
```
# same as price -> first lets visualize without the clear outliers  
boxplot(df$min_nights, outline = F)
```



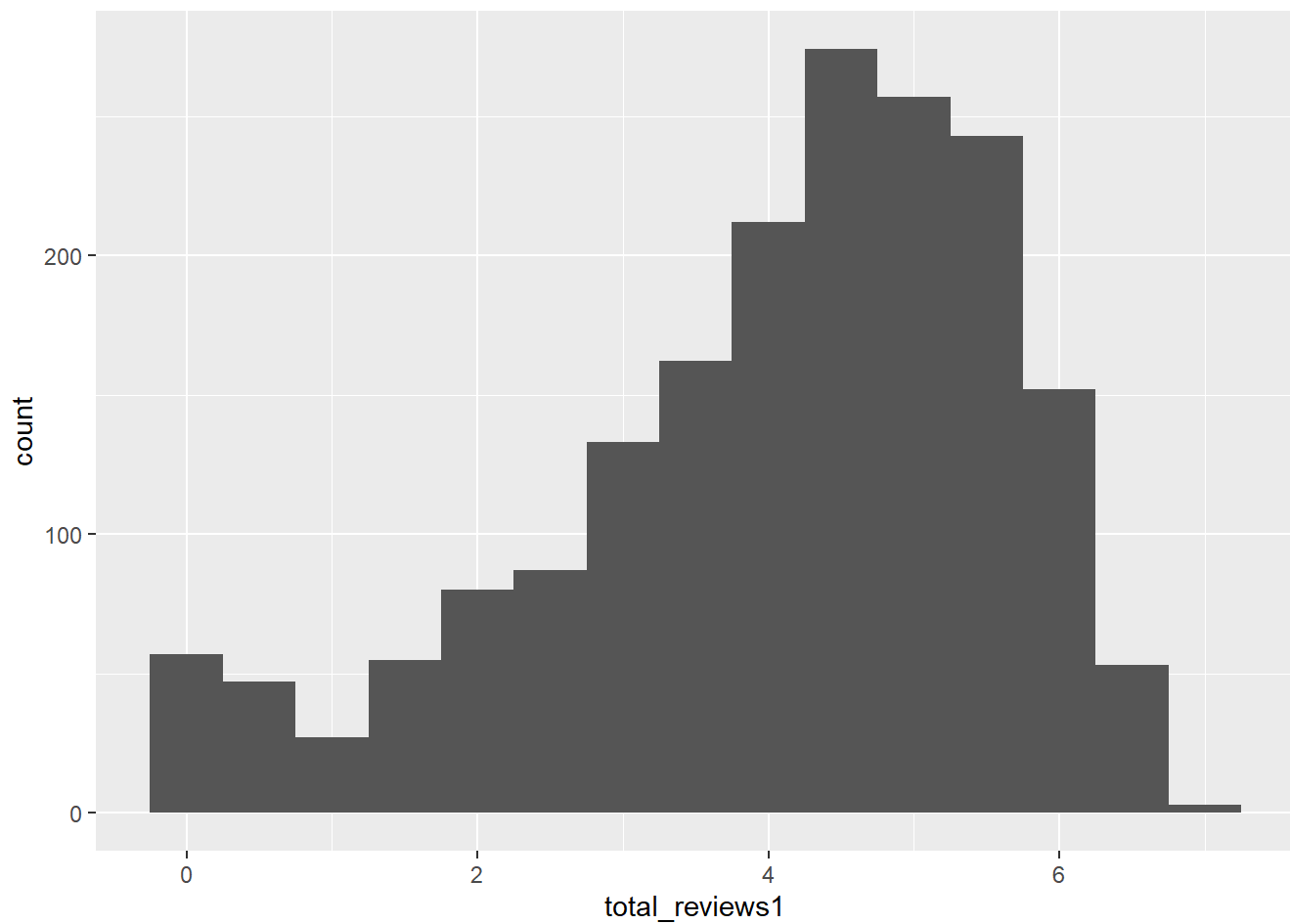
```
# 13) total reviews is an int data type  
summary(df$total_reviews)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   
##      1.0   22.5   75.0   126.6   184.0   1332.0
```

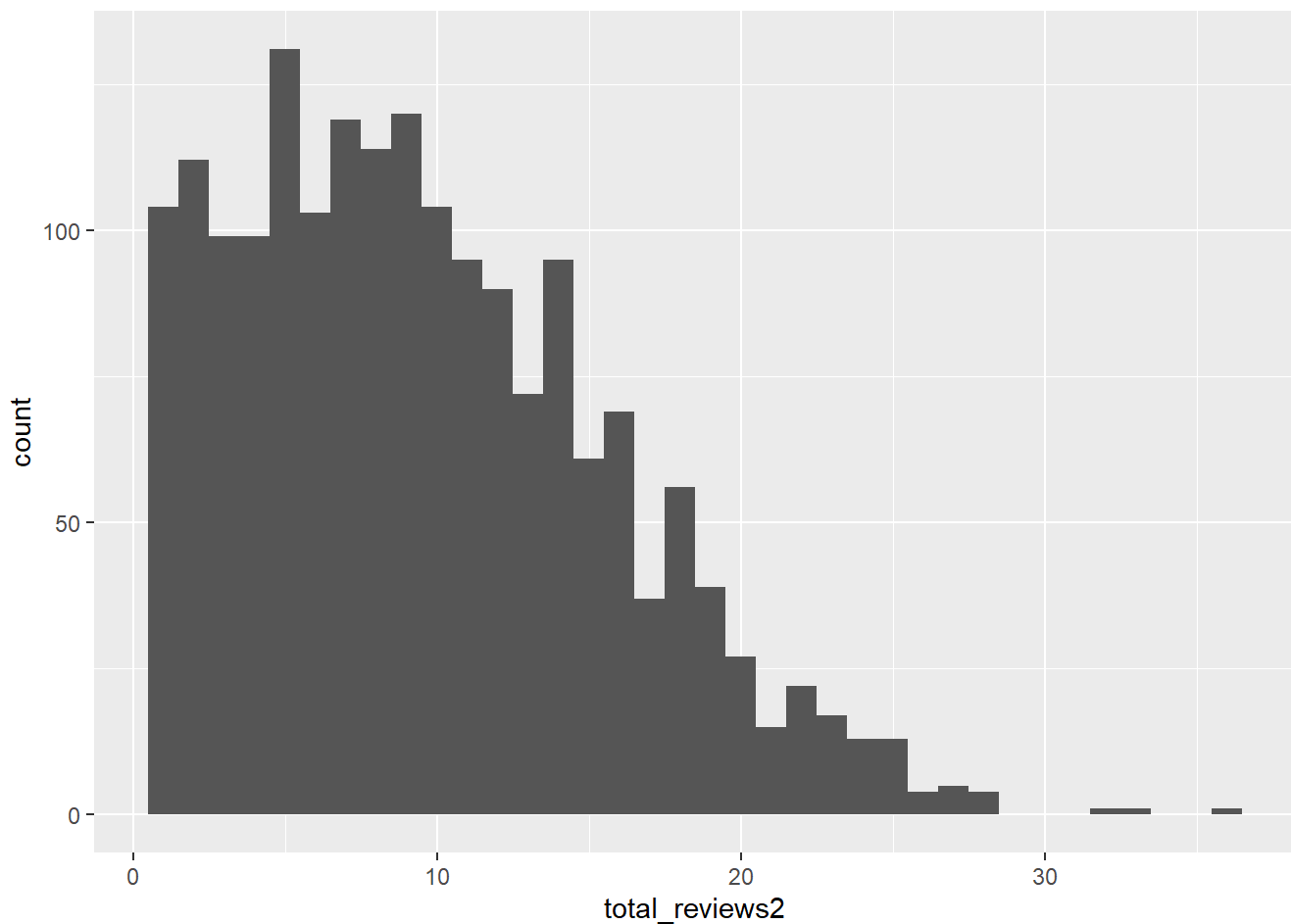
```
ggplot(df, aes(total_reviews)) + geom_histogram(binwidth=50)
```



```
df_tr_test = df_price %>%  
  mutate(total_reviews1 = log(total_reviews),  
         total_reviews2 = sqrt(total_reviews))  
  
ggplot(df_tr_test, aes(total_reviews1)) + geom_histogram(binwidth = 0.5)
```



```
ggplot(df_tr_test, aes(total_reviews2)) + geom_histogram(binwidth = 1)
```

```
## Log transform was better than sqrt transform
```

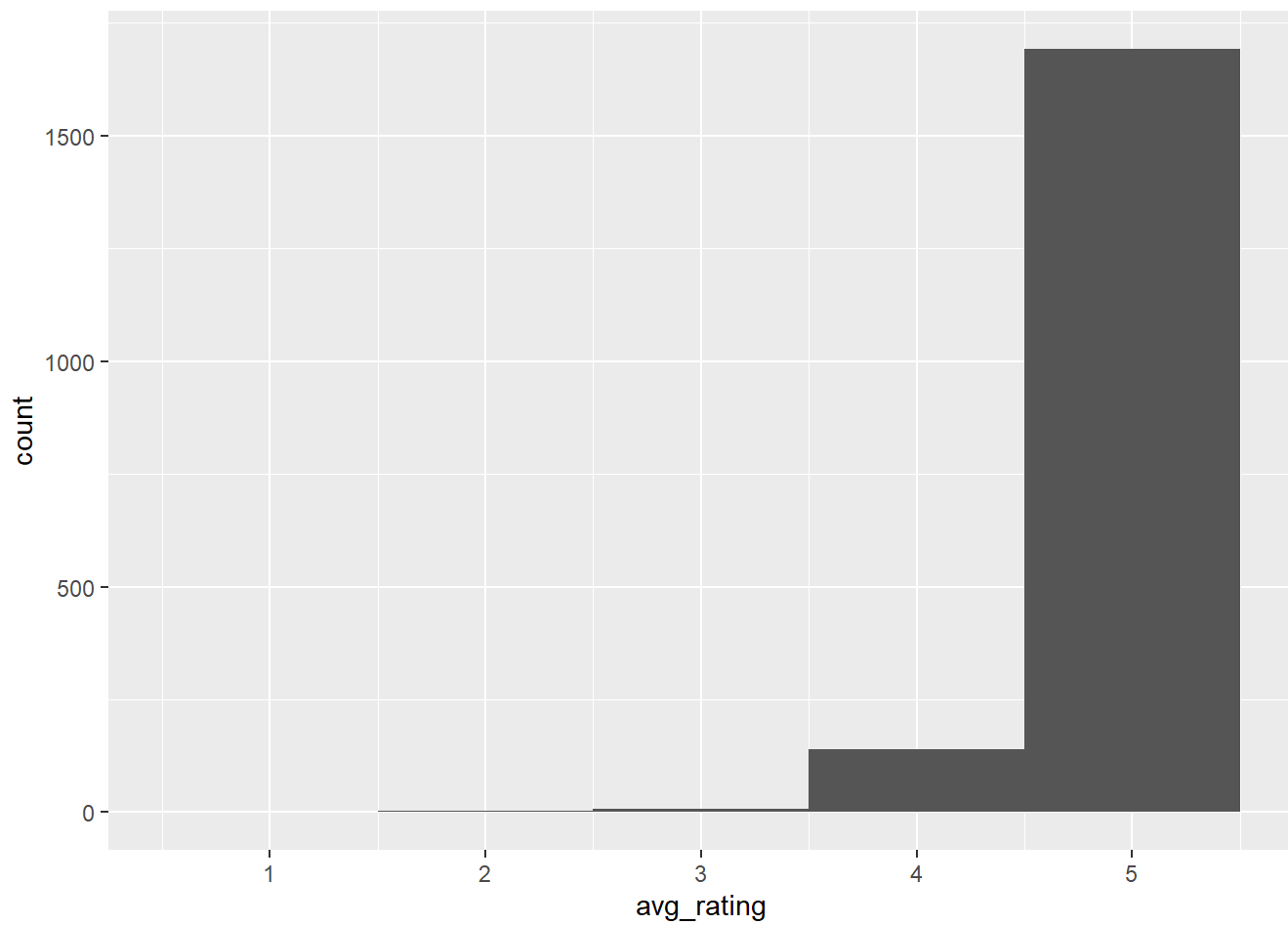
```
# 14) avg_rating is a double data type
summary(df$avg_rating)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      1.000   4.740   4.860   4.795   4.940   5.000
```

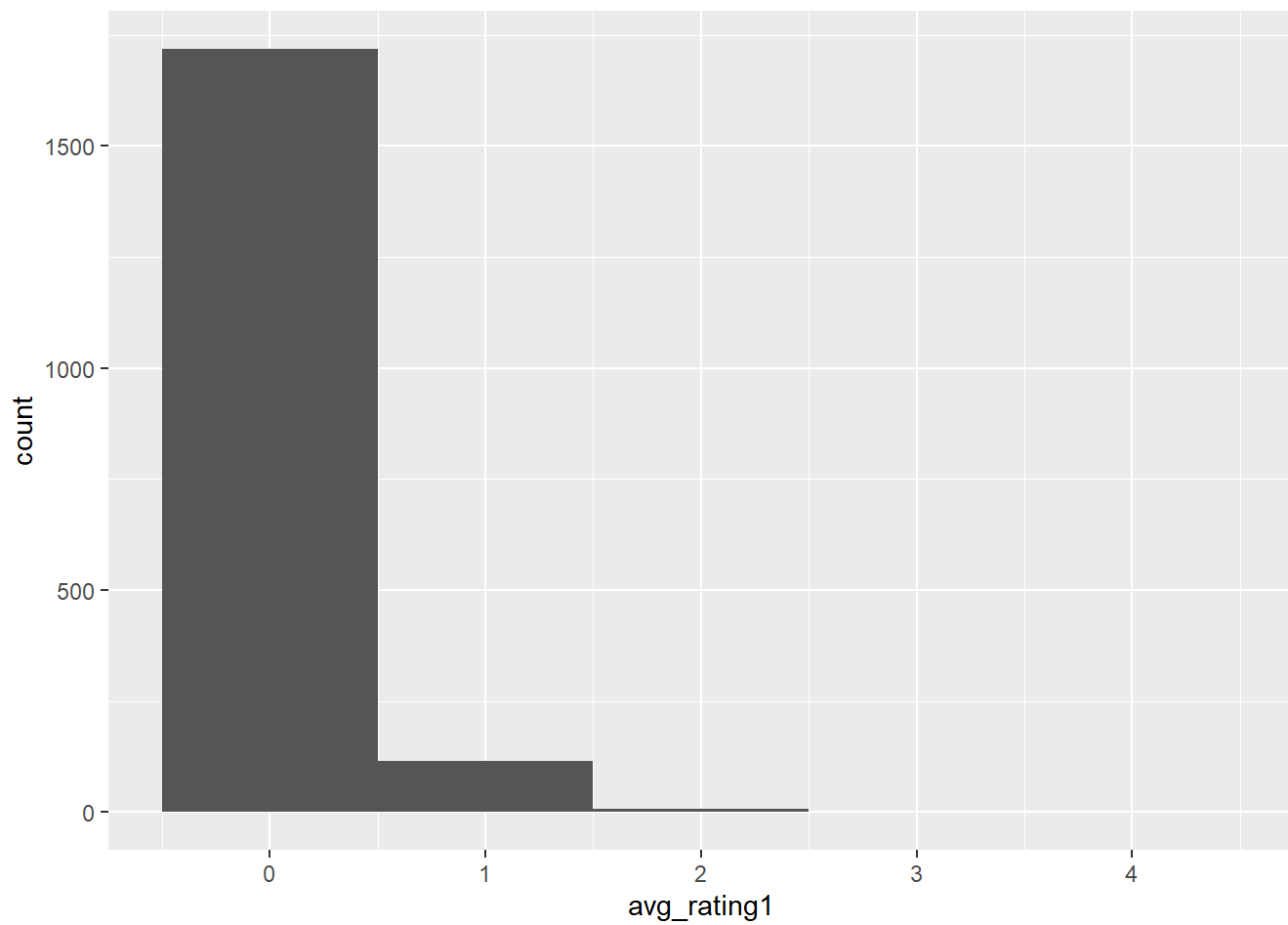
```
# Left skewed data, with boundary conditions of 1 to 5
```

```
df_avg_rating = df_price %>%
  mutate( avg_rating1 = 5- avg_rating,
           avg_rating2 = sqrt(5- avg_rating),
           avg_rating3 = log(5- avg_rating),
           avg_rating4 = log((5- avg_rating)/ avg_rating))

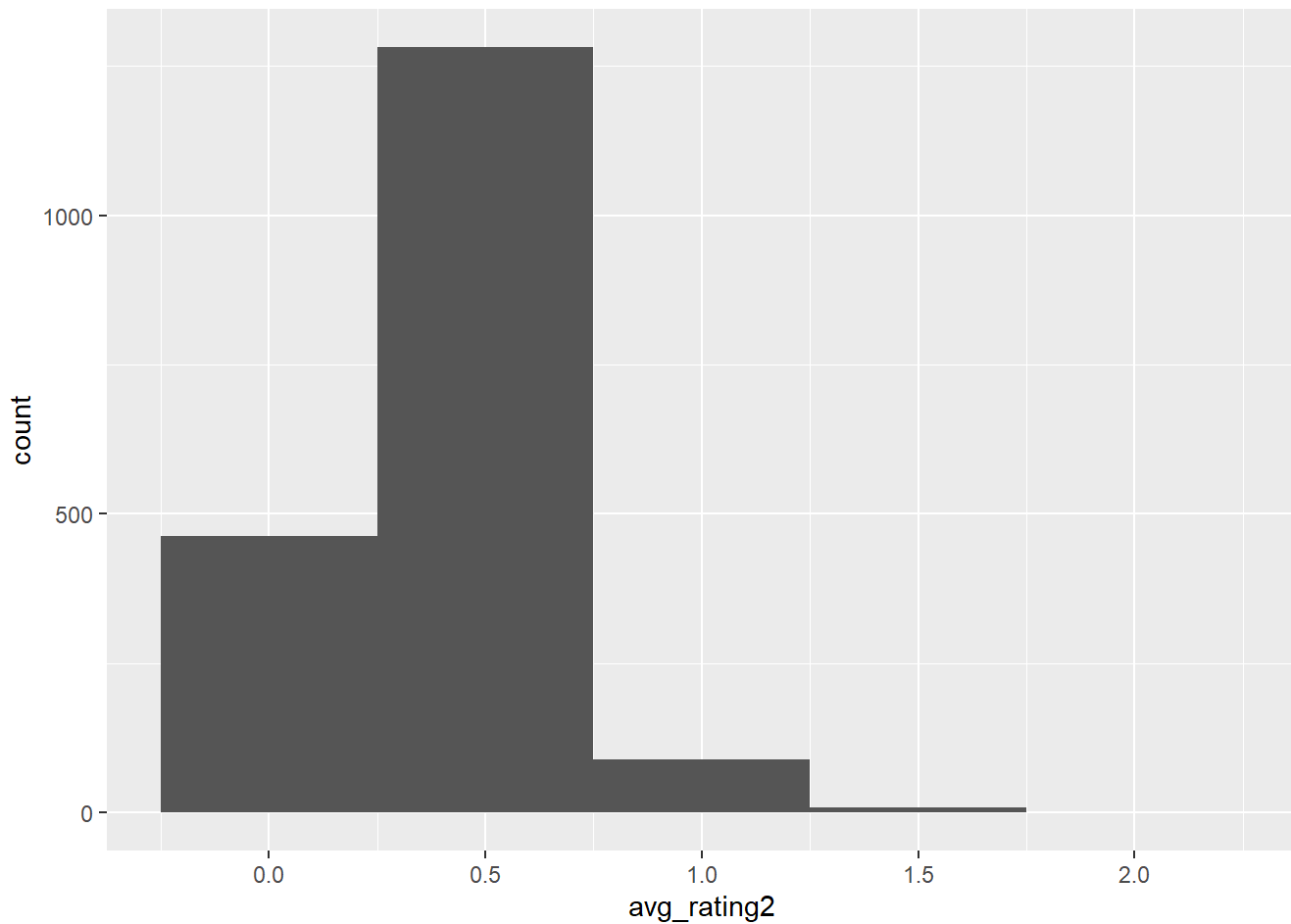
ggplot(df_price, aes(avg_rating)) + geom_histogram(binwidth = 1)
```



```
ggplot(df_avg_rating, aes(avg_rating1)) + geom_histogram(binwidth = 1)
```



```
ggplot(df_avg_rating, aes(avg_rating2)) + geom_histogram(binwidth = 0.5)
```



```
## this worked well however 224 non-finite outside the scale range
# ggplot(df_avg_rating, aes(avg_rating3)) + geom_histogram(binwidth = 0.5)
# ggplot(df_avg_rating, aes(avg_rating4)) + geom_histogram(binwidth=.2)
```

```
## Lets explore possible correlations between the numerical variables
```

```
correlation_matrix = cor(select_if(df, is.numeric))
diag(correlation_matrix) = NA
correlation_df = data.frame(correlation_matrix)
names(correlation_df[which.max(abs(correlation_df$price))])
```

```
## [1] "host_total_listings"
```

```
## before any filtering of data , host total listings is the most correlated with price

# now that the price outliers are filtered out
# Let us retest the correlation matrix
correlation_matrix_clean = cor(select_if(df_price, is.numeric))
diag(correlation_matrix_clean) = NA
correlation_df_clean = data.frame(correlation_matrix_clean)

a=correlation_df_clean %>%
  arrange(desc(price)) %>%
  select(price)

correlation_matrix_clean2 = cor(select_if(df_a_p, is.numeric))
diag(correlation_matrix_clean2) = NA
correlation_df_clean2 = data.frame(correlation_matrix_clean2)

price_acc=correlation_df_clean2 %>%
  arrange(desc(price)) %>%
  select(price)

correlation_matrix_clean3 = cor(select_if(df_b_p, is.numeric))
diag(correlation_matrix_clean3) = NA
correlation_df_clean3 = data.frame(correlation_matrix_clean3)

price_br=correlation_df_clean3 %>%
  arrange(desc(price)) %>%
  select(price)

correlation_matrix_clean4 = cor(select_if(df_bed_p, is.numeric))
diag(correlation_matrix_clean4) = NA
correlation_df_clean4 = data.frame(correlation_matrix_clean4)

price_beds=correlation_df_clean4 %>%
  arrange(desc(price)) %>%
  select(price)

test = tibble(a , price_acc$price , price_br$price , price_beds$price)
test
```

```
## # A tibble: 10 × 4
##   price `price_acc$price` `price_br$price` `price_beds$price`
##   <dbl>         <dbl>         <dbl>         <dbl>
## 1  0.465         0.449         0.454         0.440
## 2  0.449         0.404         0.428         0.392
## 3  0.316         0.271         0.291         0.344
## 4  0.226         0.241         0.233         0.244
## 5  0.163         0.158         0.161         0.164
## 6 -0.0195       -0.0275       -0.0161       -0.0421
## 7 -0.0369       -0.0500       -0.0345       -0.0629
## 8 -0.126        -0.121        -0.122        -0.133
## 9 -0.133        -0.138        -0.133        -0.142
## 10 NA          NA          NA          NA
```

```
## based on the correlation matrix, only filtering out the price outliers was the most effective
```

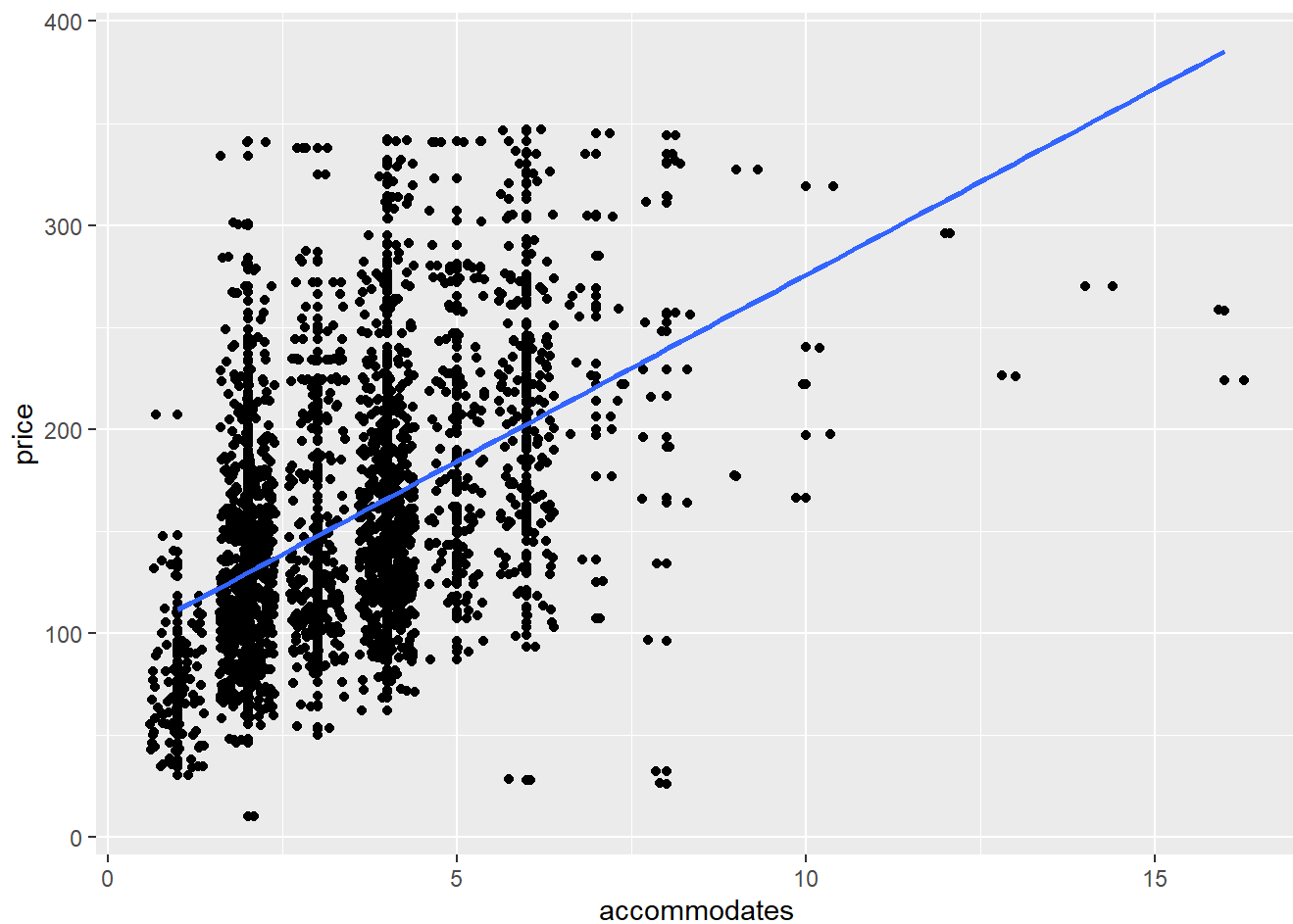
```
## best three to study are accommodates, bedrooms , beds (stays true for all)
```

```
## Lets start with visualizing price v accommodates, bedroom, bed
```

```
## price = dependent variable , the rest are independent variable
```

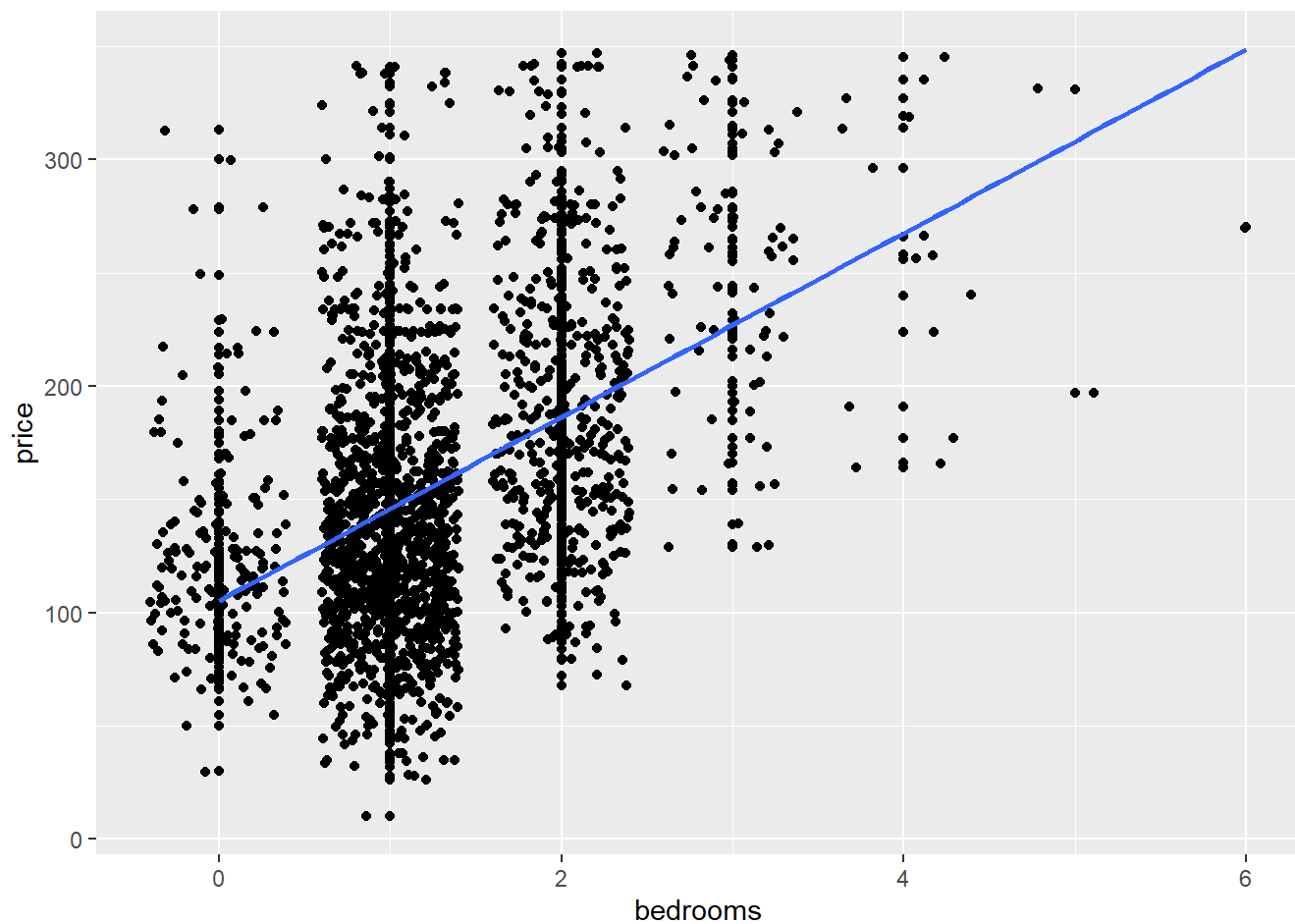
```
ggplot( df_price , aes(x = accommodates, y = price)) + geom_point() + geom_jitter() + geom_smooth(
  method = "lm" , se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



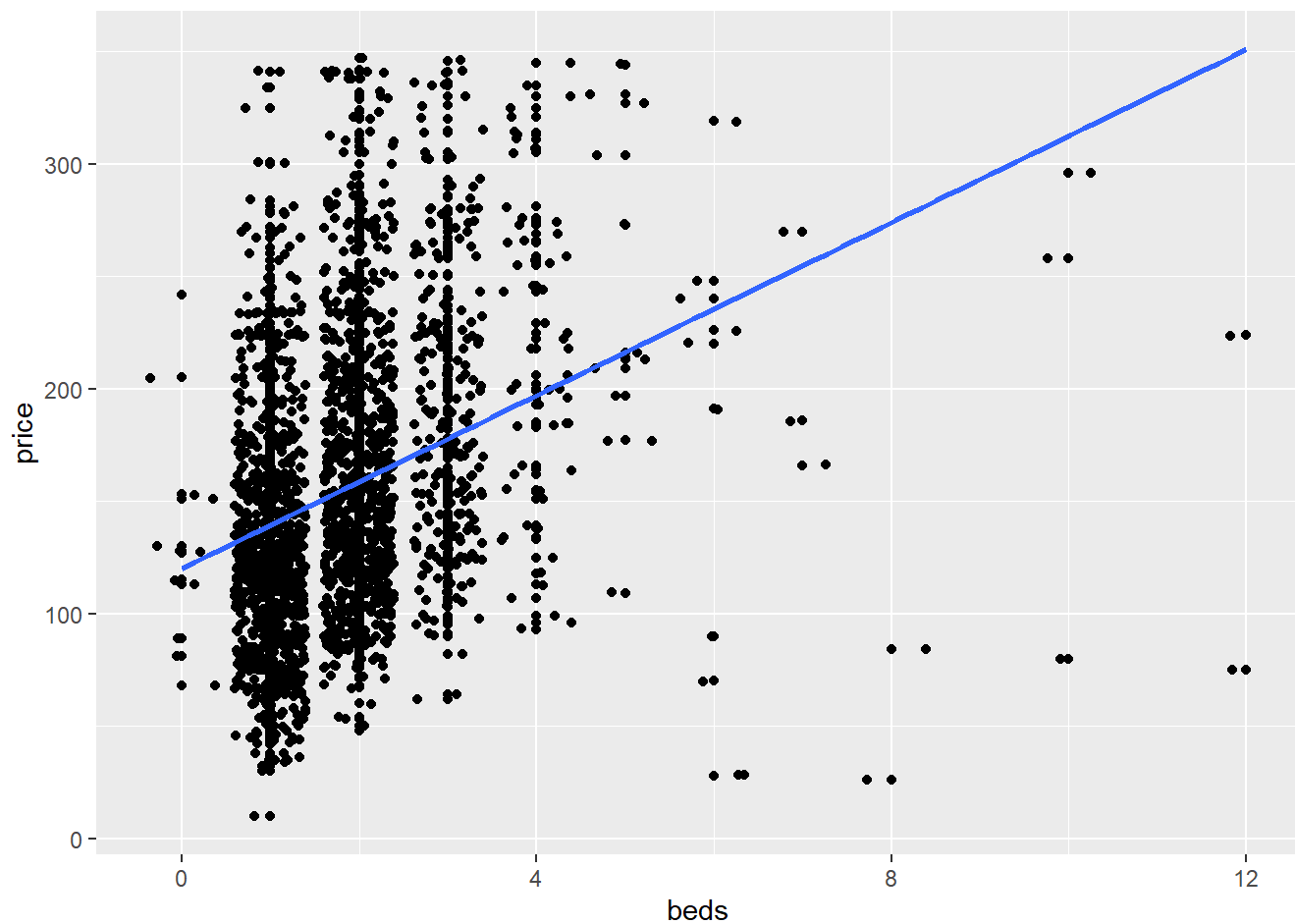
```
ggplot( df_price , aes(x = bedrooms, y = price)) + geom_point() + geom_jitter() + geom_smooth(me  
thod = "lm" , se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



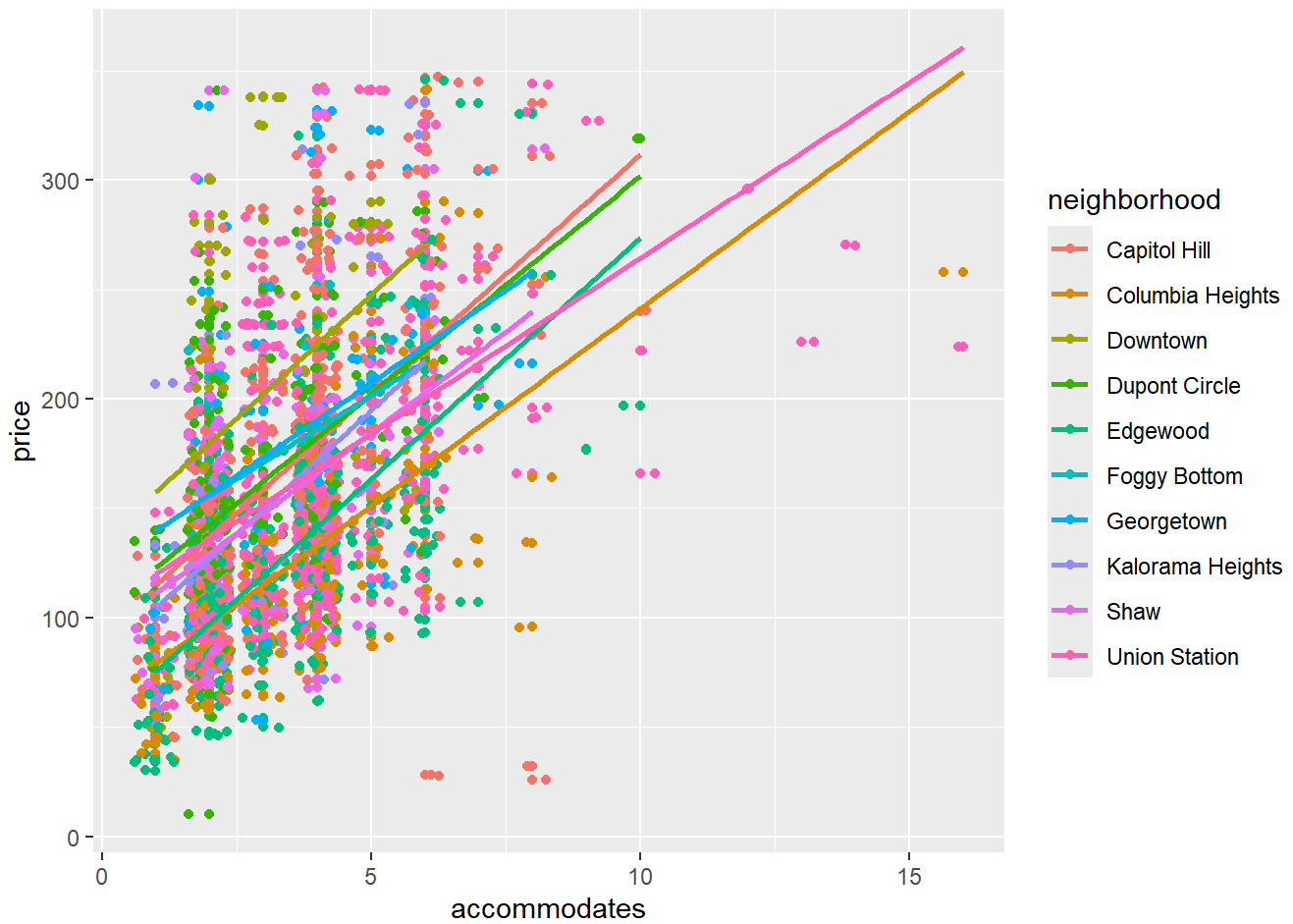
```
ggplot( df_price , aes(x = beds, y = price)) + geom_point() + geom_jitter() + geom_smooth(method  
= "lm" , se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

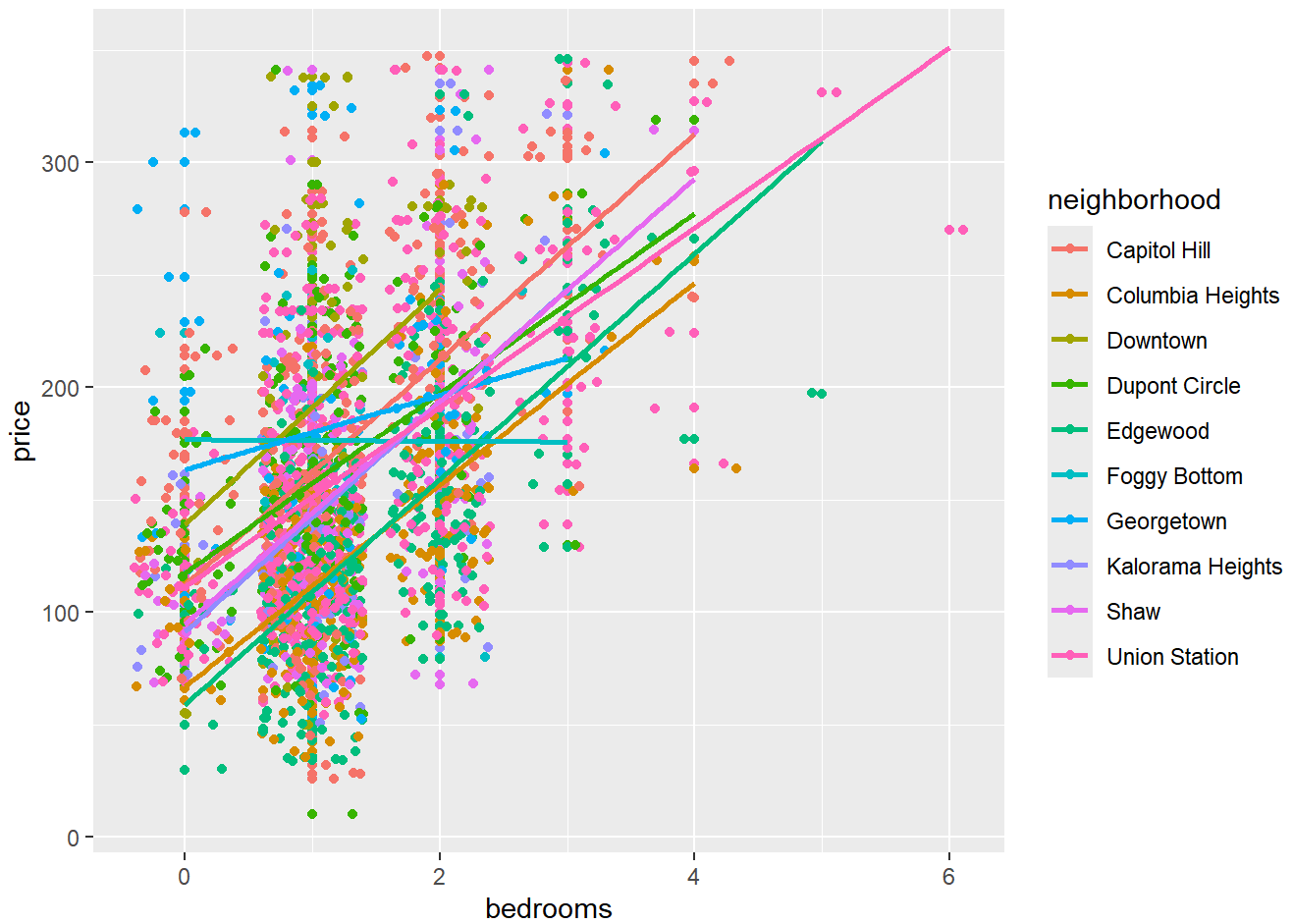
```
ggplot( df_price , aes(x = accommodates, y = price , color = neighborhood)) + geom_point() + geom_jitter() + geom_smooth(method = "lm" , se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



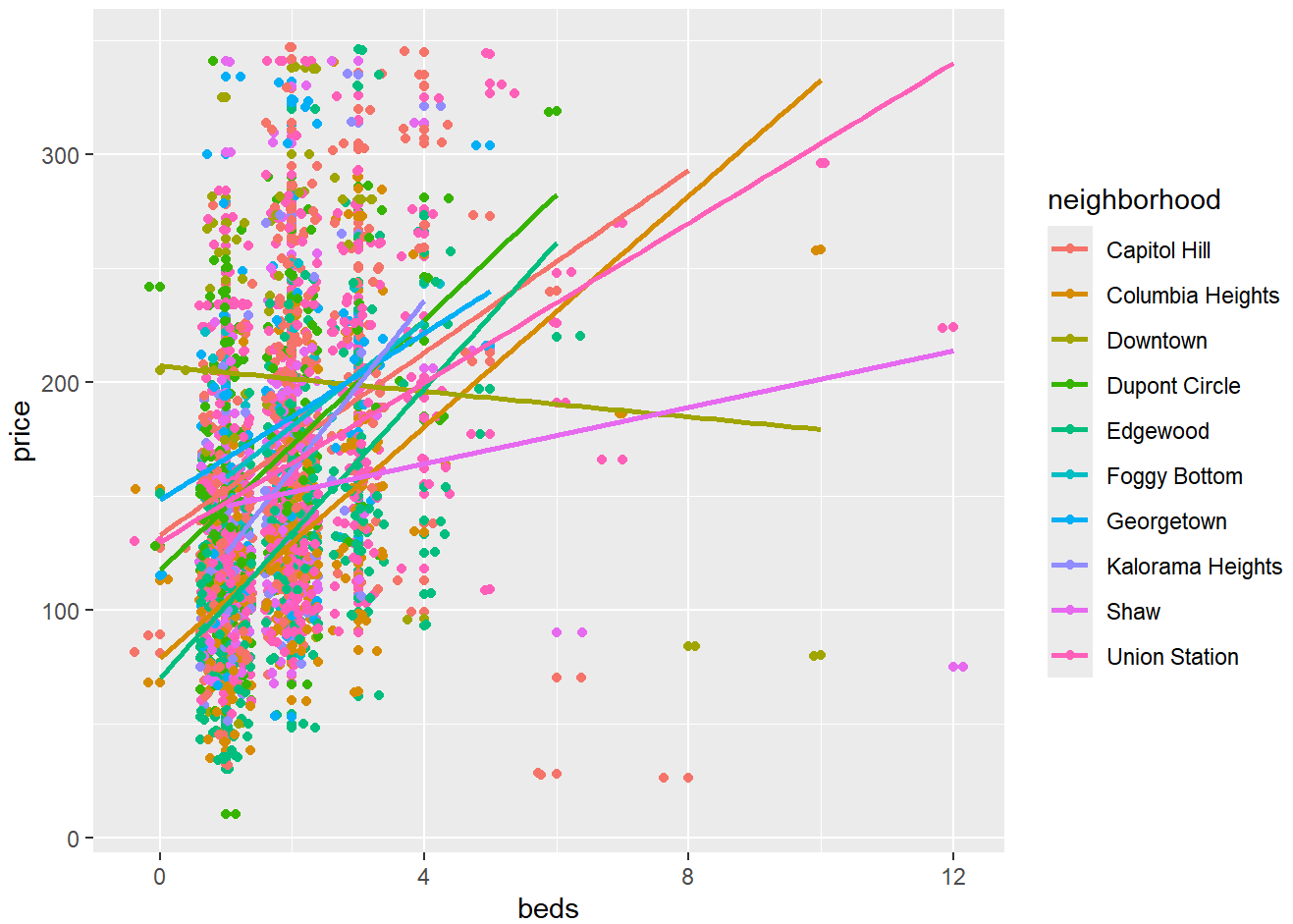
```
ggplot( df_price , aes(x = bedrooms, y = price , color = neighborhood)) + geom_point() + geom_jitter() + geom_smooth(method = "lm" , se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



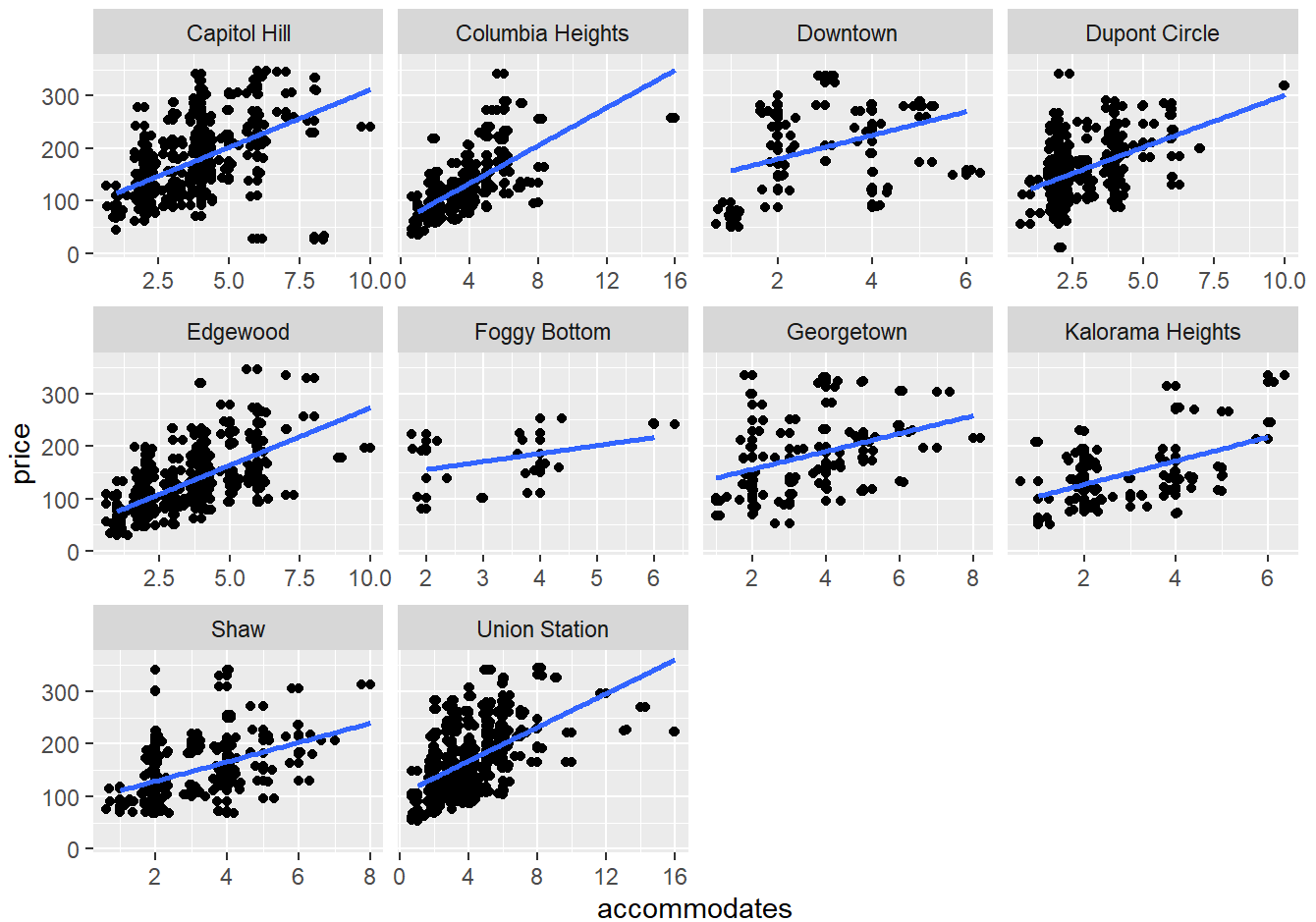
```
ggplot( df_price , aes(x = beds, y = price , color = neighborhood)) + geom_point() + geom_jitter(
) + geom_smooth(method = "lm" , se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



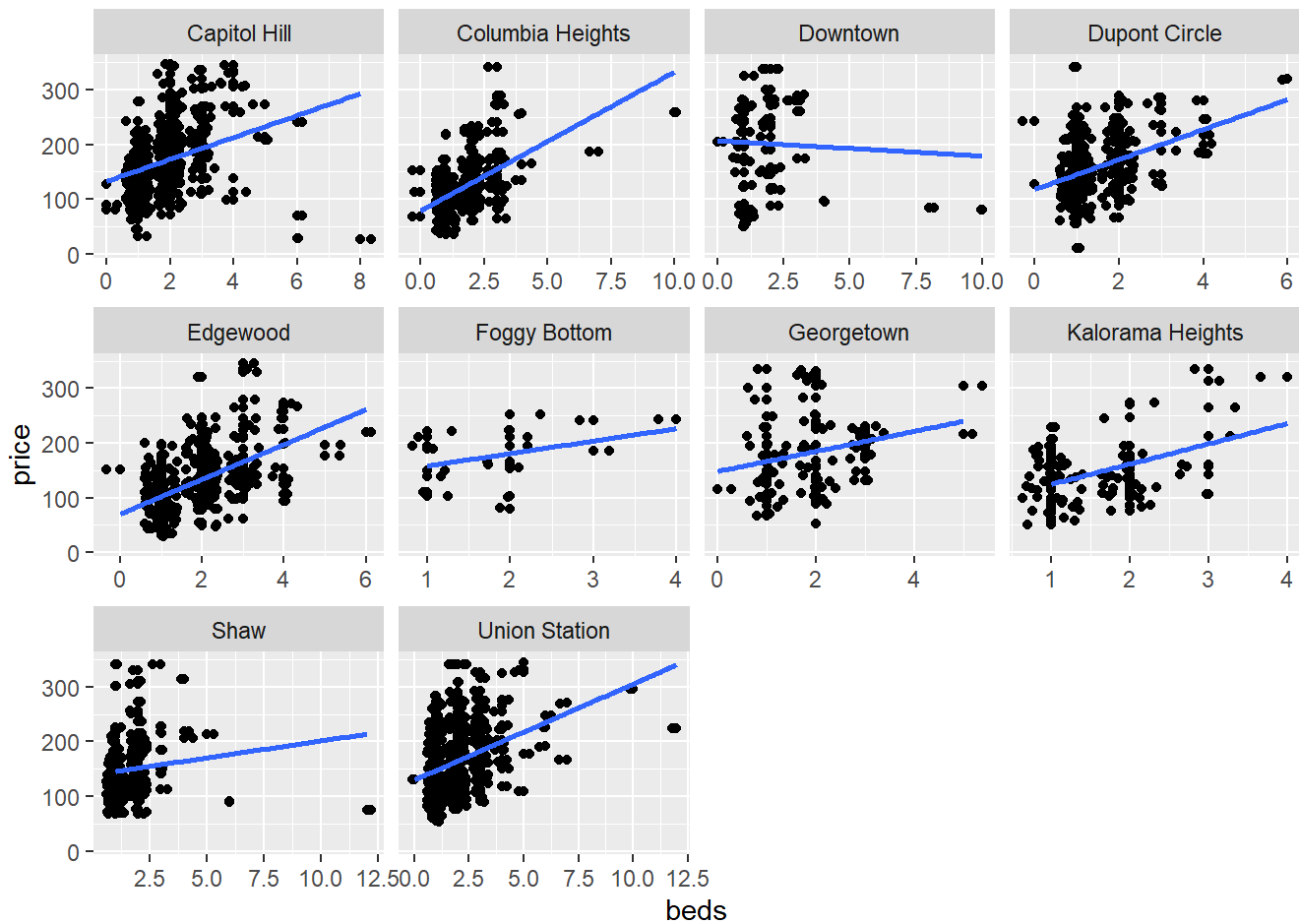
```
ggplot(df_price , aes(x = accommodates, y = price)) +
  geom_point() + geom_jitter() + geom_smooth(method = "lm" , se = FALSE) +
  facet_wrap(~neighborhood, scales = "free_x")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



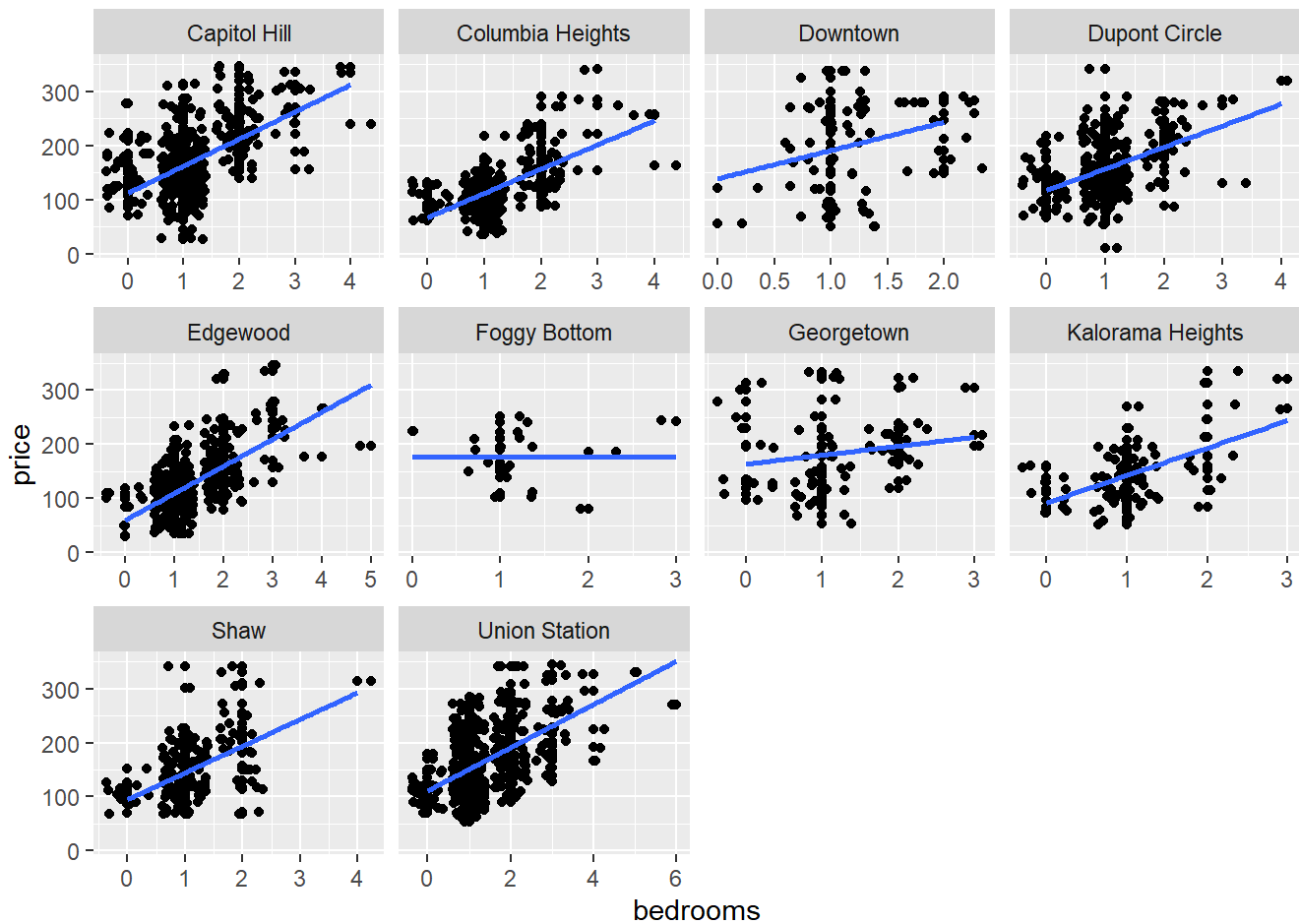
```
ggplot(df_price , aes(x = beds, y = price)) +
  geom_point() + geom_jitter() + geom_smooth(method = "lm" , se = FALSE) +
  facet_wrap(~neighborhood, scales = "free_x")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



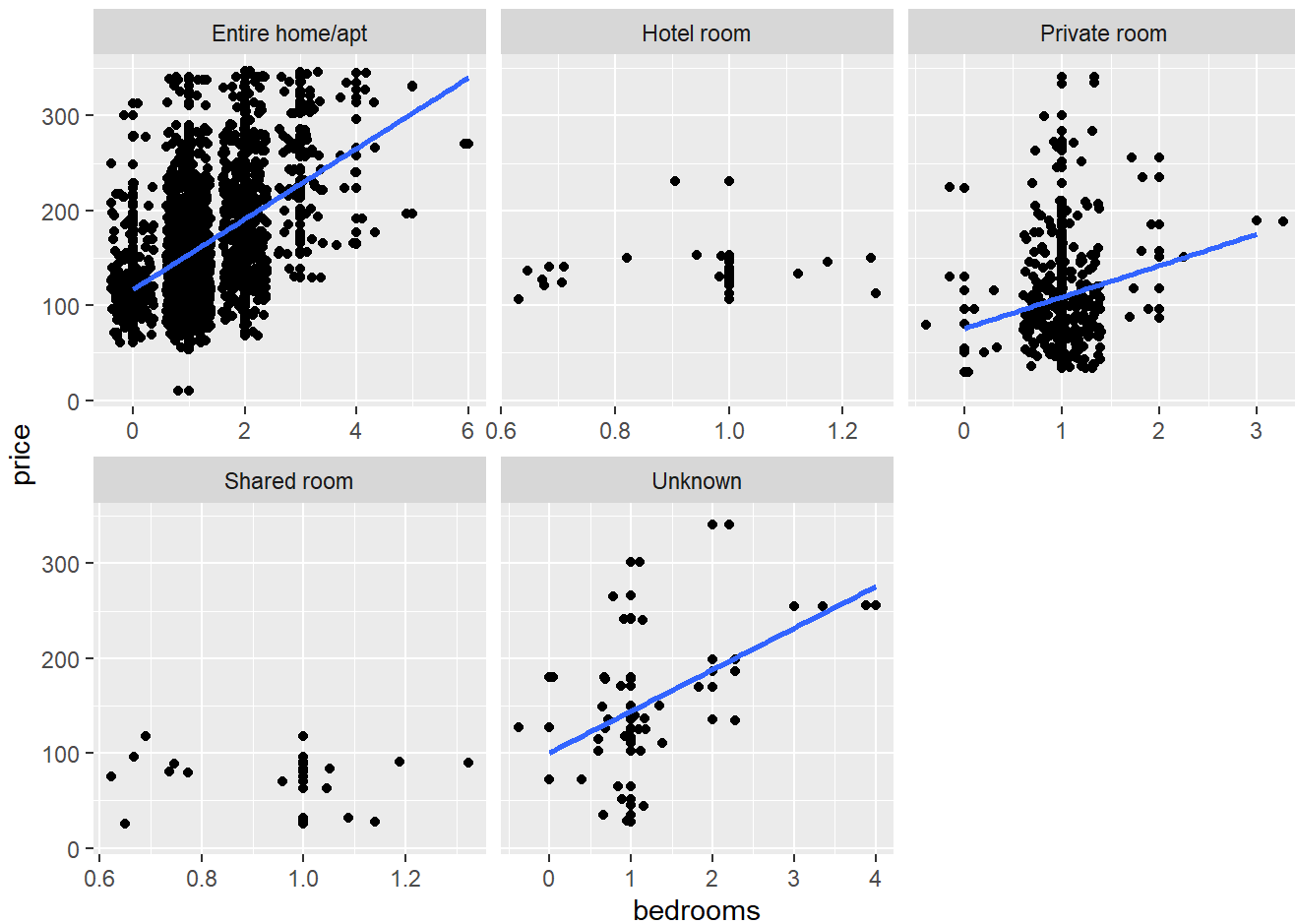
```
ggplot(df_price , aes(x = bedrooms, y = price)) +
  geom_point() + geom_jitter() + geom_smooth(method = "lm" , se = FALSE) +
  facet_wrap(~neighborhood, scales = "free_x")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



```
ggplot(df_price , aes(x = bedrooms, y = price)) +
  geom_point() + geom_jitter() + geom_smooth(method = "lm" , se = FALSE) +
  facet_wrap(~room_type, scales = "free_x")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



```
## Summary statistics, Group By Room_type , Bathrooms, Neighborhood
```

```
df_price %>%
  group_by(neighborhood) %>%
  summarize(avg_price = mean(price) , sd_price = sd(price) , median_price = quantile(price, 0.5)
, IQR_price = IQR(price))
```

```
## # A tibble: 10 × 5
##   neighborhood    avg_price sd_price median_price IQR_price
##   <fct>          <dbl>    <dbl>         <dbl>    <dbl>
## 1 Capitol Hill      171.      66.0         158.      89
## 2 Columbia Heights  125.      50.6         119       58
## 3 Downtown          202.      87.1         213     158
## 4 Dupont Circle     158.      54.6         150     69.8
## 5 Edgewood          128.      56.3         119       62
## 6 Foggy Bottom      176.      51.1         188.     67.2
## 7 Georgetown        183.      72.0         178     98.5
## 8 Kalorama Heights  147.      59.1         138.      66
## 9 Shaw              150.      59.9         136.     77.5
## 10 Union Station    165.      63.0         152      95
```



```
df_price %>%
  group_by(room_type) %>%
  summarize(avg_price = mean(price) , sd_price = sd(price) , median_price = quantile(price, 0.5)
, IQR_price = IQR(price))
```

```
## # A tibble: 5 × 5
##   room_type      avg_price sd_price median_price IQR_price
##   <fct>          <dbl>    <dbl>         <dbl>    <dbl>
## 1 Entire home/apt    164.      61.8          151      85
## 2 Hotel room         141.      27.9          138     23.8
## 3 Private room      109.      56.1          97.5    60.2
## 4 Shared room        73.1      27.3          80.5     25
## 5 Unknown           153.      75.2          137     70
```

```
df_price %>%
  filter(room_type == "Entire home/apt") %>%
  group_by(neighborhood) %>%
  summarize(avg_price = mean(price) , sd_price = sd(price) , median_price = quantile(price, 0.5)
, IQR_price = IQR(price))
```

```
## # A tibble: 10 × 5
##   neighborhood      avg_price sd_price median_price IQR_price
##   <fct>          <dbl>    <dbl>         <dbl>    <dbl>
## 1 Capitol Hill      181.      61.9          168      81
## 2 Columbia Heights  134.      48.4          125      50
## 3 Downtown          226.      74.9          241     126
## 4 Dupont Circle     164.      57.6          160     82.5
## 5 Edgewood          144.      52.5          134.     59.2
## 6 Foggy Bottom      164.      48.4          166     64.5
## 7 Georgetown        189.      69.1          189     95.8
## 8 Kalorama Heights  152.      59.7          139     50.5
## 9 Shaw              160.      57.4          150     74.8
## 10 Union Station    169.      61.6          156     102
```

```
df_price %>%
  filter(room_type == "Entire home/apt" & neighborhood == "Union Station") %>%
  group_by(bathrooms) %>%
  summarize(avg_price = mean(price) , sd_price = sd(price) , median_price = quantile(price, 0.5)
, IQR_price = IQR(price))
```

```
## # A tibble: 6 × 5
##   bathrooms avg_price sd_price median_price IQR_price
##   <fct>      <dbl>    <dbl>      <dbl>    <dbl>
## 1 1 bath      155.      52.1       142      71.5
## 2 1.5 baths   206.      61.3       185       86
## 3 2 baths    230.      67.3       226.     95.8
## 4 2.5 baths   229.      44.4       223      67.5
## 5 3 baths    274.      75.9       325     105
## 6 4 baths    247       32.5       247       23
```

```
## Question 2 ##
```

```
## Lets also get some numbers:
table(df_price$room_type)
```

```
##
## Entire home/apt      Hotel room      Private room      Shared room      Unknown
##           1509              16           268           14              35
```

```
table(df_price$neighborhood)
```

```
##
## Capitol Hill Columbia Heights      Downtown      Dupont Circle
##           310           213           69           216
## Edgewood      Foggy Bottom      Georgetown Kalorama Heights
##           275           20           74           80
## Shaw      Union Station
##           148           437
```

```
table(df_price$neighborhood, df_price$room_type)
```

```
##
##           Entire home/apt Hotel room Private room Shared room Unknown
## Capitol Hill           271         1         26         6         6
## Columbia Heights       172         0         34         0         7
## Downtown               45         1         20         3         0
## Dupont Circle          171        14         27         0         4
## Edgewood               212         0        60         0         3
## Foggy Bottom           15         0         4         0         1
## Georgetown             64         0         8         0         2
## Kalorama Heights        59         0        17         1         3
## Shaw                   102         0        39         4         3
## Union Station          398         0        33         0         6
```

```
df_price %>%
  group_by(neighborhood) %>%
  summarize(avg_price = mean(price, na.rm = TRUE), Q_price25=quantile(price, 0.25), Q_price50=quantile(price, 0.5), Q_price75=quantile(price, 0.75)) %>%
  arrange(desc(avg_price))
```

```
## # A tibble: 10 × 5
##   neighborhood    avg_price Q_price25 Q_price50 Q_price75
##   <fct>          <dbl>    <dbl>    <dbl>    <dbl>
## 1 Downtown        202.      122      213      280
## 2 Georgetown      183.      128      178      226.
## 3 Foggy Bottom    176.      146.     188.     214.
## 4 Capitol Hill    171.      123      158.     212
## 5 Union Station   165.      118      152      213
## 6 Dupont Circle   158.      122.     150      192.
## 7 Shaw            150.      109.     136.     186.
## 8 Kalorama Heights 147.      106.     138.     172.
## 9 Edgewood        128.       92      119      154
## 10 Columbia Heights 125.       91      119      149
```

```
df_price %>%
  group_by(neighborhood, room_type) %>%
  summarize(avg_price = mean(price, na.rm = TRUE), Q_price25=quantile(price, 0.25), Q_price50=quantile(price, 0.5), Q_price75=quantile(price, 0.75)) %>%
  arrange(desc(avg_price))
```

```
## `summarise()` has grouped output by 'neighborhood'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 36 × 6
## # Groups:   neighborhood [10]
##   neighborhood room_type    avg_price Q_price25 Q_price50 Q_price75
##   <fct>        <fct>        <dbl>    <dbl>    <dbl>    <dbl>
## 1 Foggy Bottom Unknown        241      241      241      241
## 2 Downtown     Hotel room      231      231      231      231
## 3 Downtown     Entire home/apt 226.     154      241      280
## 4 Union Station Unknown        216.     172.     190.     241
## 5 Foggy Bottom Private room    206.     192.     217      231
## 6 Shaw         Unknown        199.     148.     171      236
## 7 Georgetown   Entire home/apt 189.     132.     189      228.
## 8 Capitol Hill Entire home/apt 181.     136.     168      218.
## 9 Union Station Entire home/apt 169.     120      156      222
## 10 Dupont Circle Entire home/apt 164.     122.     160      205
## # i 26 more rows
```

Question 3

kind of interesting since we can simply use the `t.test` function.

```
conf_test = function(level , name , dataframe) {
  # the function will take three arguments
  # level: numerical confidence level
  # name of the numerical variable (string)
  # name of the dataframe

  # need to only grab all the columns and work with that data
  # since double [[]] for dataframes will pull only the data from that column
  var = dataframe[[name]]
  avg = mean(var , na.rm = TRUE)
  std = sd(var, na.rm = TRUE)
  alpha = 1 -level
  d_f = length(var) - 1
  # one tailed
  t_val = abs(qt( alpha, d_f ))

  conf_int = c( avg - (t_val * (std / sqrt(length(var))))
               ,
               avg + (t_val * (std / sqrt(length(var)))) )
  # two tailed
  t_val2 = abs(qt( alpha/2 , d_f ))
  conf_int2 = c( avg - (t_val2 * (std / sqrt(length(var))))
               ,
               avg + (t_val2 * (std / sqrt(length(var)))) )

  return(list(one_tailed = conf_int , two_tailed = conf_int2))
}
```

#Q3 (Jackie's Attempt)

```
confInt = function(level , var, data) {

  x = data[[var]]
  n = length(x)

  alpha = 1-level
  avg = mean(x)
  stdev = sd(x)
  sterr = stdev / sqrt(n)

  cvT = qt(1-alpha/2, df = n-1)

  c(lower = avg-cvT * sterr,
     mean = avg,
     upper = avg + cvT * sterr,
     n = n)
```

```
}  
  
confInt95 = confInt(.95, "price", df_price)  
  
round(confInt95, 2)
```

```
##   lower    mean  upper      n  
##  152.36  155.29  158.23 1842.00
```

#Q4 : A T-Test was performed.

```
t.test(df_price$price, mu = 200, alternative = "greater", conf.level = 0.95)
```

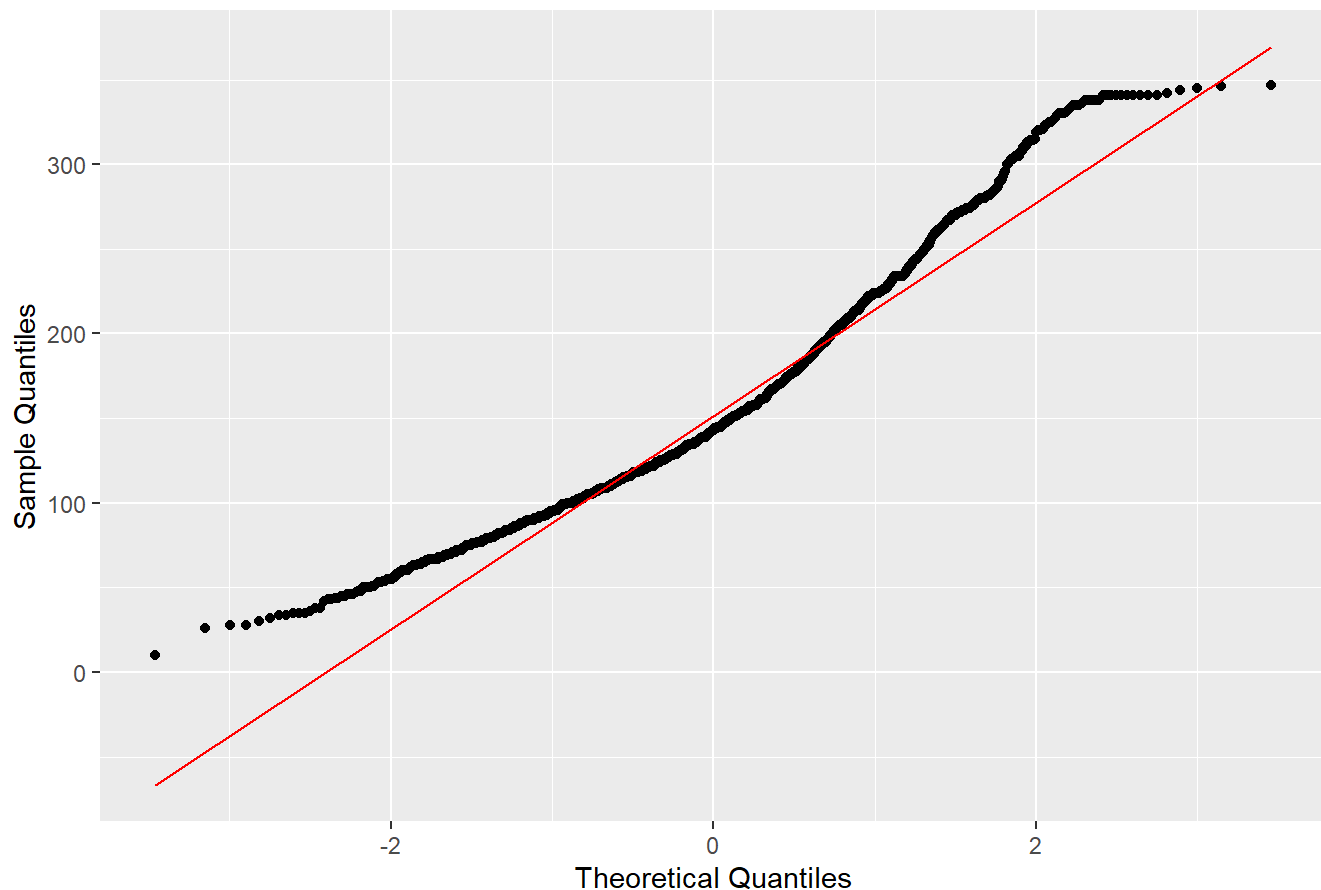
```
##  
## One Sample t-test  
##  
## data: df_price$price  
## t = -29.848, df = 1841, p-value = 1  
## alternative hypothesis: true mean is greater than 200  
## 95 percent confidence interval:  
##  152.8294      Inf  
## sample estimates:  
## mean of x  
##  155.2942
```

#Q5. Visualize price to test for normality.

#Use Q-Q Plot to test for normality.

```
ggplot(df_price, aes(sample = price)) +  
  stat_qq() +  
  stat_qq_line(color = "red") +  
  labs(x = "Theoretical Quantiles",  
       y = "Sample Quantiles",  
       title = "Q-Q Plot of Airbnb Prices in Washington, DC")
```

Q-Q Plot of Airbnb Prices in Washington, DC



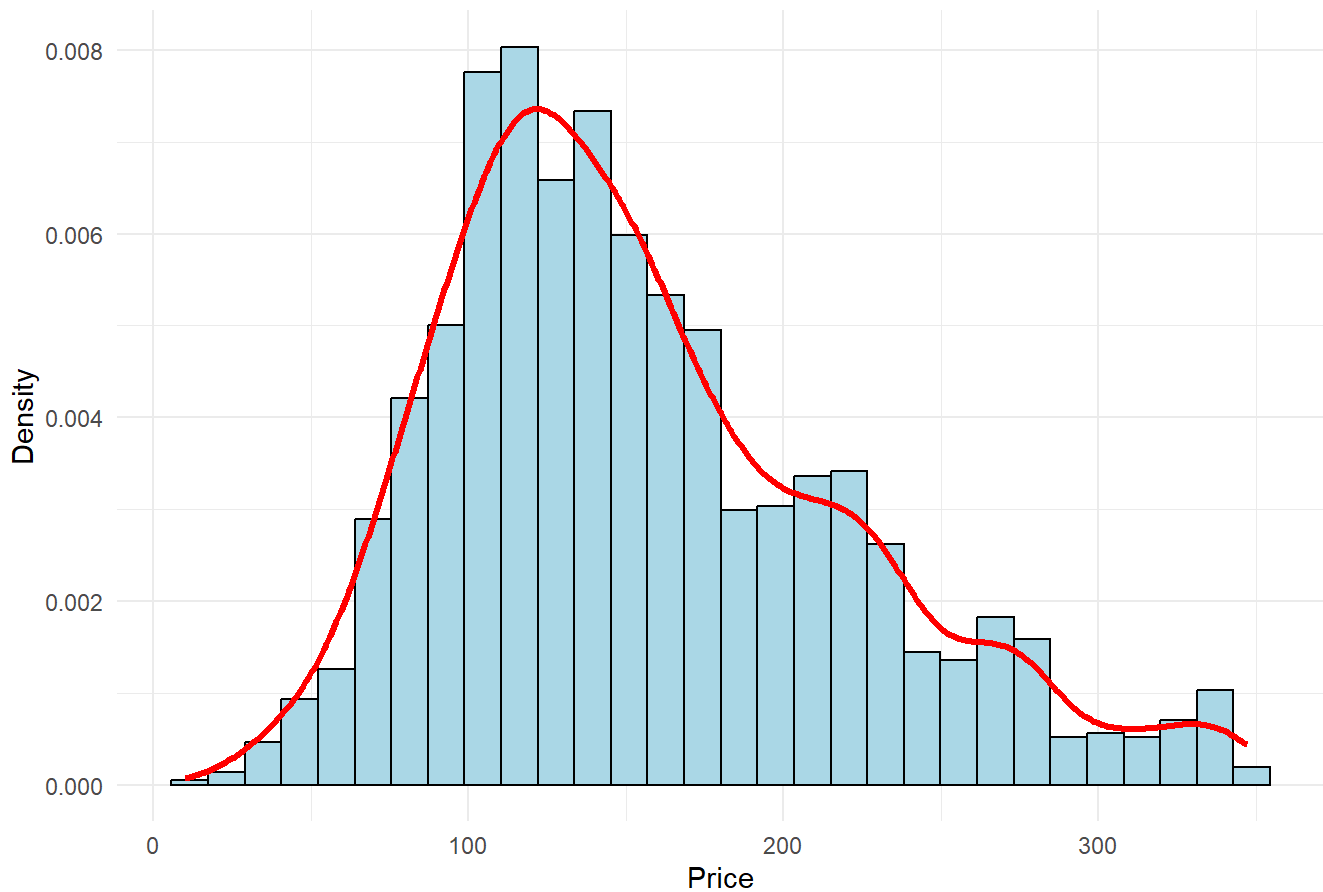
#Use Histogram to test for normality

```
ggplot(df_price, aes(x = price)) +
  geom_histogram(aes(y = ..density..), bins = 30, fill = "lightblue",
                 color = "black")+
  geom_density(color = "red", size = 1.2) +
  labs(x = "Price",
       y = "Density",
       title = "Airbnb Prices in Washington, DC") +
  theme_minimal()
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

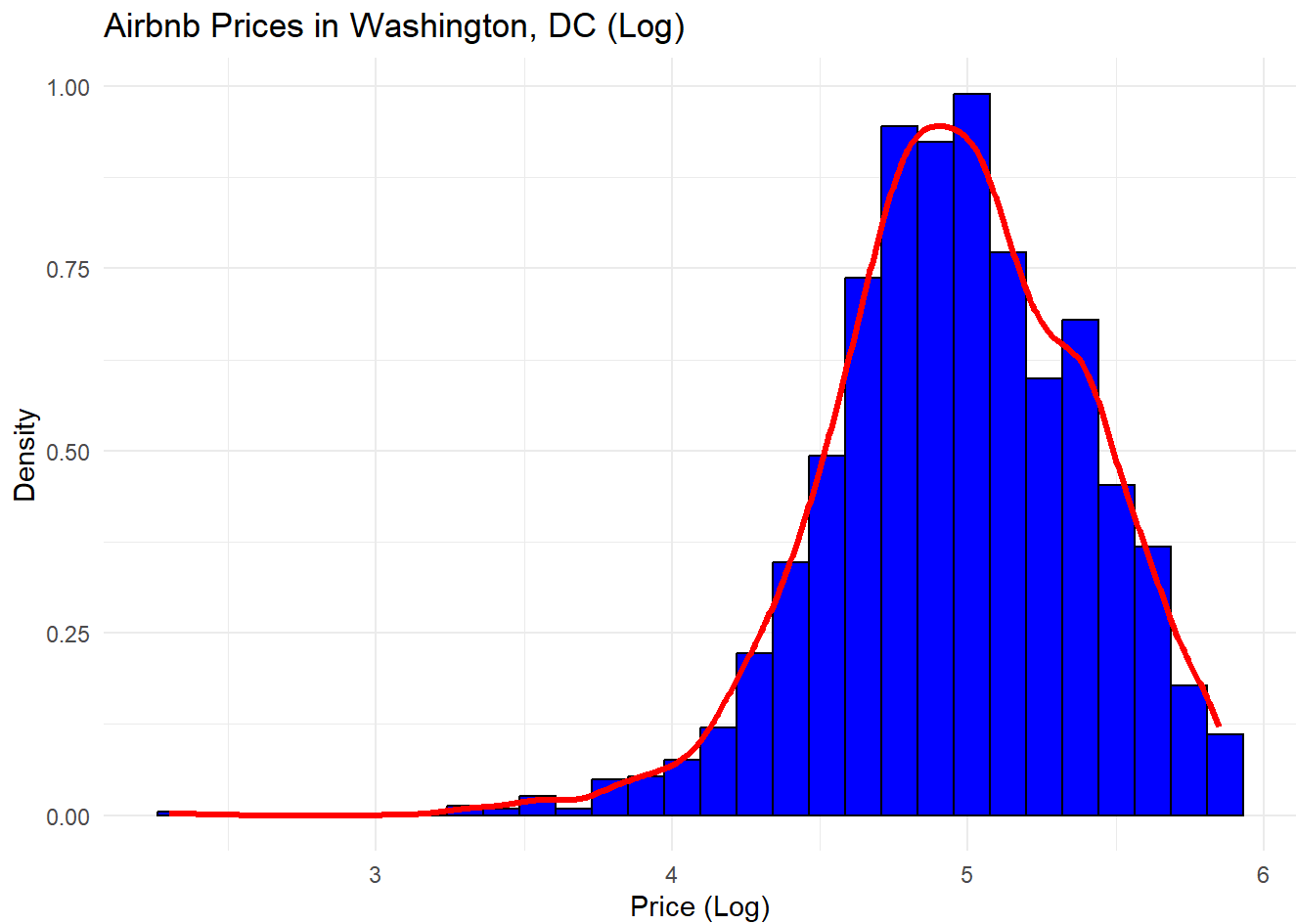
```
## Warning: The dot-dot notation (`..density..`) was deprecated in ggplot2 3.4.0.
## i Please use `after_stat(density)` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

Airbnb Prices in Washington, DC



```
#Use log(price) to try to normalize and make less skewed  
#Log(price) did not work
```

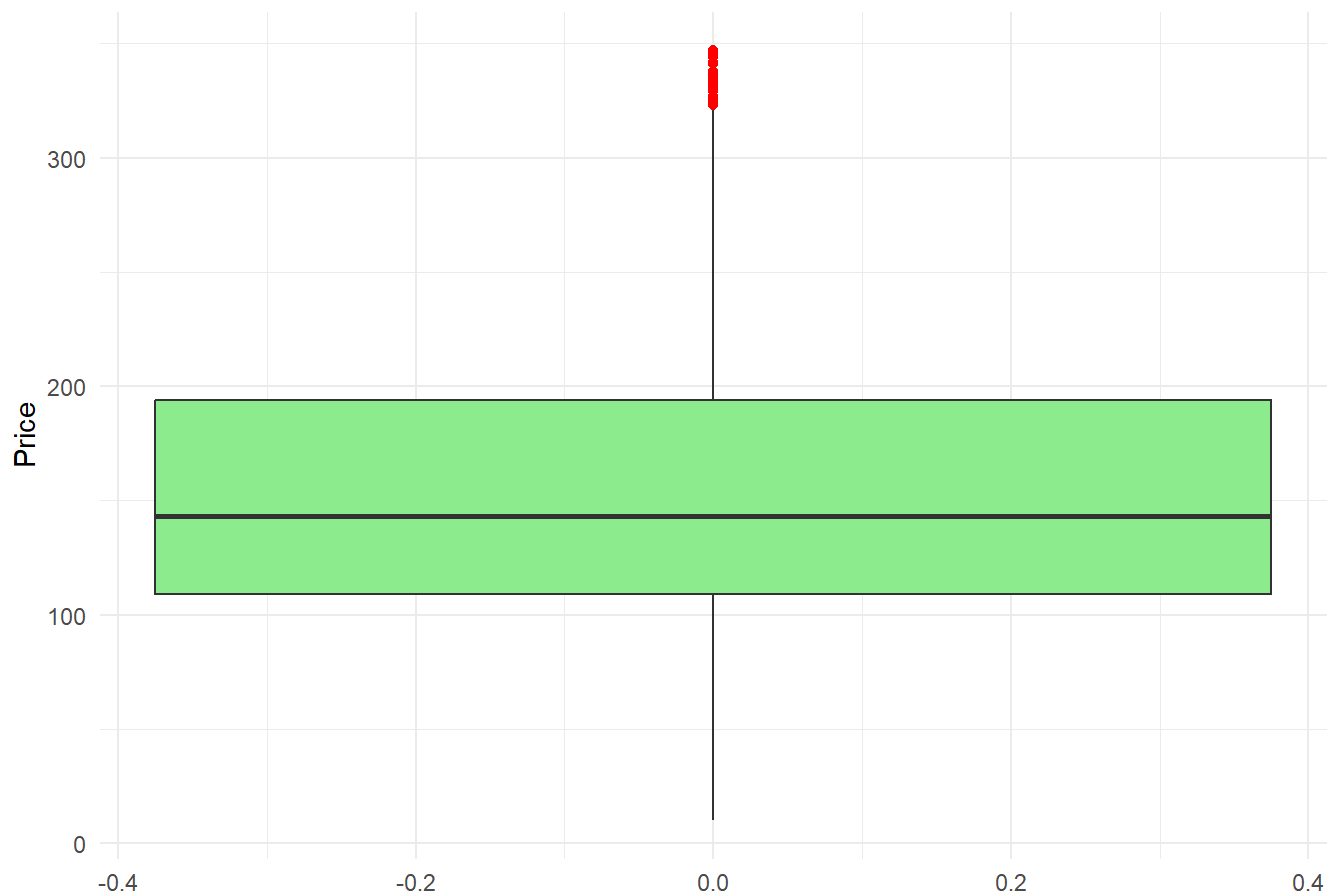
```
ggplot(df_price, aes(x = log(price))) +  
  geom_histogram(aes(y = ..density..), bins = 30, fill = "blue",  
                 color = "black")+  
  geom_density(color = "red", size = 1.2) +  
  labs(x = "Price (Log)",  
       y = "Density",  
       title = "Airbnb Prices in Washington, DC (Log)") +  
  theme_minimal()
```



#Boxplot (can removed since it does not answer #5)

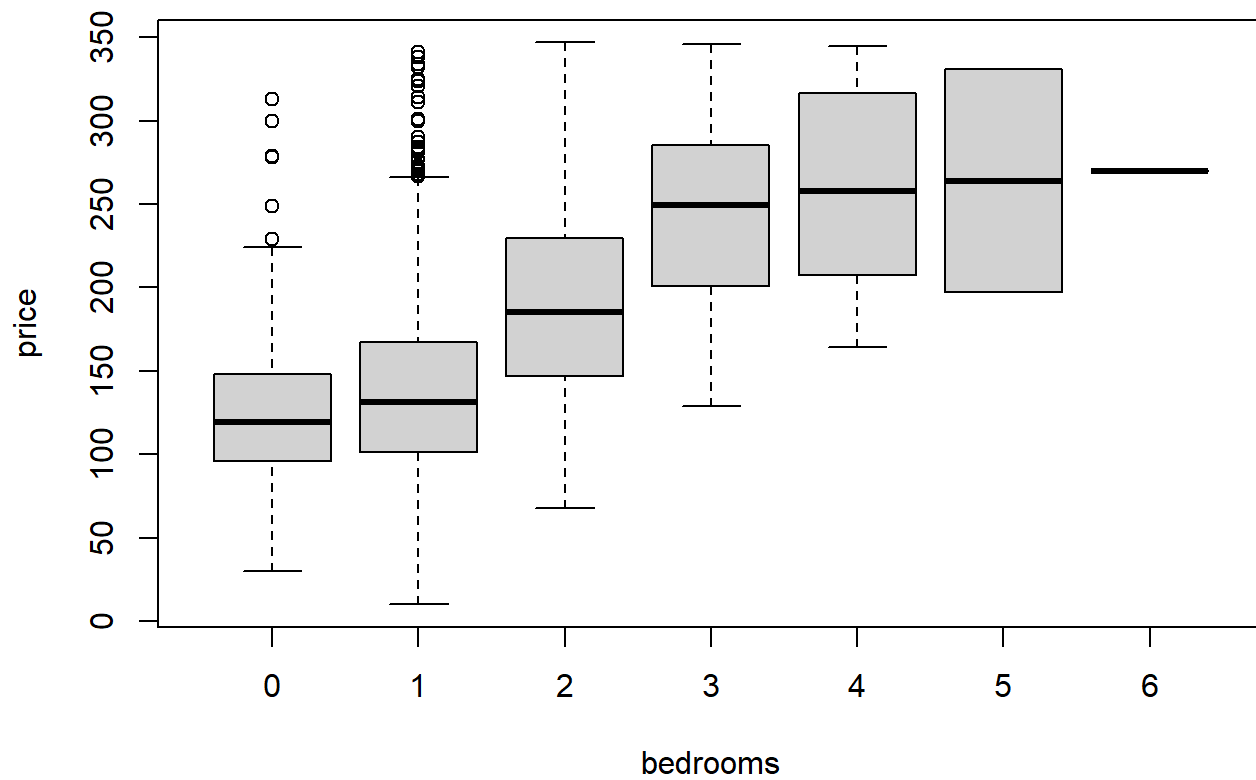
```
ggplot(df_price, aes(y = price)) +  
  geom_boxplot(fill = "lightgreen", outlier.color = "red") +  
  labs(y = "Price",  
       title = "Boxplot of Airbnb Prices in DC") +  
  theme_minimal()
```


Boxplot of Airbnb Prices in DC



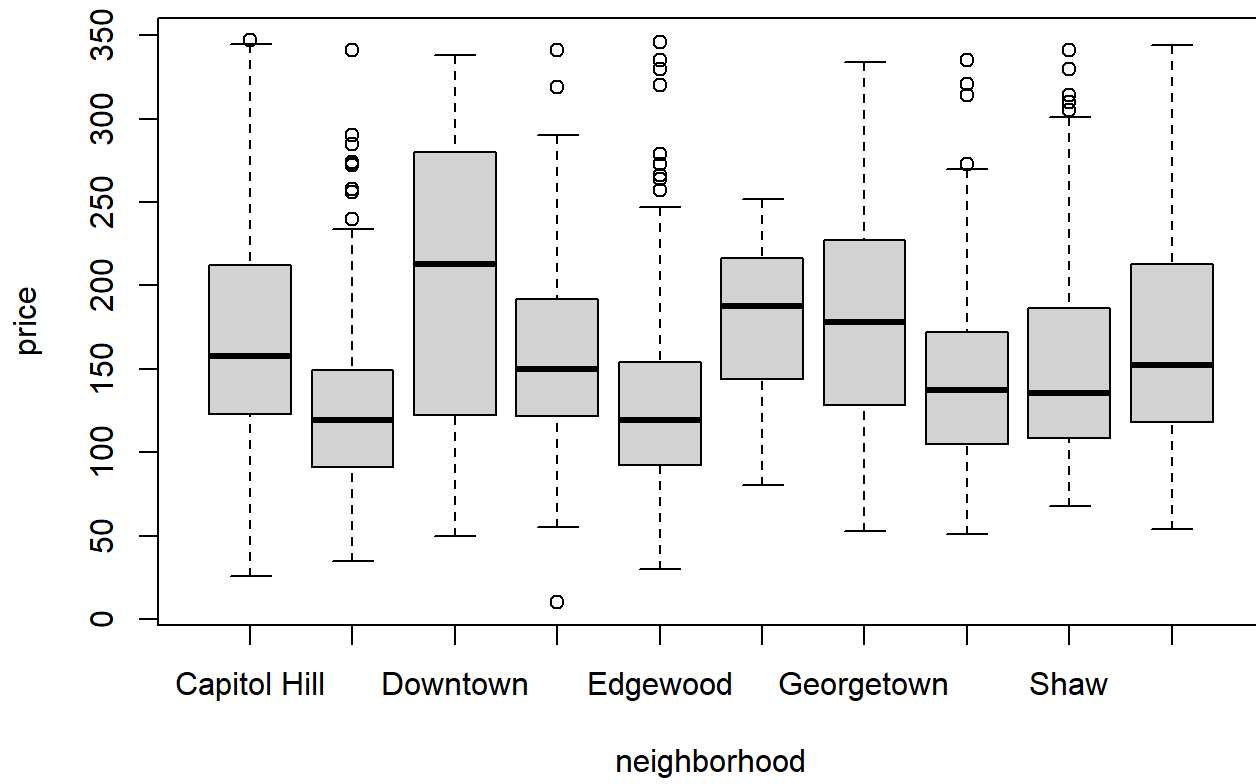
#Boxplot to compare price and bedrooms

```
boxplot(price~bedrooms, df_price)
```



#Boxplot to compare price and neighborhood

```
boxplot(price~neighborhood, df_price)
```



#Q6. What's the best simple linear regression model for the "price" of the listings based on R-squared and residual standard error?

#Compare/present the results of the tested models as a table in your report.

#Simple linear regression of price using neighborhood as predictor.

```
RegNeighborhood = lm(price~neighborhood, df_price)
```

#Simple linear regression of price using bedrooms as predictor.

```
RegBedrooms = lm(price~bedrooms, df_price)
```

#Simple linear regression of price using room type as predictor.

```
RegRoomType = lm(price~room_type, df_price)
```

#Simple linear regression of price using accommodates as predictor.

```
RegAccommodates = lm(price~accommodates, df_price)
```

#Simple linear regression of price using beds as predictor.

```
RegBeds = lm(price~beds, df_price)
```

#Simple linear regression of price using bathrooms as predictor.

```
RegBathrooms = lm(price~bathrooms, df_price)
```

#Simple linear regression of price using minimum nights as predictor.

```
RegMinNights = lm(price~min_nights, df_price)
```

#Simple linear regression of price using total reviews as predictor.

```
RegReviews = lm(price~total_reviews, df_price)
```

#Simple linear regression of price using host acceptance rate as predictor.

```
RegHostAcceptRate = lm(price~host_acceptance_rate, df_price)
```

#Simple linear regression of price using Superhost as predictor.

```
RegSuperhost = lm(price~superhost, df_price)
```

#Simple linear regression of price using host total listings as predictor.

```
RegHostListings = lm(price~host_total_listings, df_price)
```

#Simple linear regression of price using average rating as predictor.

```
RegAvgRating = lm(price~avg_rating, df_price)
```

#Compare the regression models of R-squared and residual standard error.

#Summaries

```
sumNeighborhood = summary(RegNeighborhood)
```

```
sumBedrooms = summary(RegBedrooms)
```

```
sumRoomType = summary(RegRoomType)
```

```
sumAccommodates = summary(RegAccommodates)
```

```
sumBeds = summary(RegBeds)
```

```
sumBathrooms = summary(RegBathrooms)
```

```
sumMinNights = summary(RegMinNights)
```

```
sumReviews = summary(RegReviews)
```

```
sumHostAcceptRate = summary(RegHostAcceptRate)
```

```

sumSuperhost = summary(RegSuperhost)
sumHostListings = summary(RegHostListings)
sumAvgRating = summary(RegAvgRating)

#Comparison table for all the models.

modelComp = data.frame(
  Model = c("Req with Neighborhood", "Reg with Bedrooms", "Reg with Room Type",
            "Reg with Accommodates", "Reg with Beds", "Reg with Bathrooms",
            "Reg with Min Nights", "Reg with Total Reviews", "Reg with Host Acceptance Rate",
            "Reg with Superhost", "Reg with Host Total Listings", "Reg with Average Rating"),
  SER = c(sumNeighborhood$sigma, sumBedrooms$sigma, sumRoomType$sigma,
          sumAccommodates$sigma, sumBeds$sigma, sumBathrooms$sigma,
          sumMinNights$sigma, sumReviews$sigma, sumHostAcceptRate$sigma,
          sumSuperhost$sigma, sumHostListings$sigma, sumAvgRating$sigma),
  R2 = c(sumNeighborhood$r.squared, sumBedrooms$r.squared, sumRoomType$r.squared,
         sumAccommodates$r.squared, sumBeds$r.squared, sumBathrooms$r.squared,
         sumMinNights$r.squared, sumReviews$r.squared, sumHostAcceptRate$r.squared,
         sumSuperhost$r.squared, sumHostListings$r.squared, sumAvgRating$r.squared)
)

modelComp

```

##	Model	SER	R2
## 1	Req with Neighborhood	61.25694	0.0963342077
## 2	Reg with Bedrooms	57.45585	0.2015308358
## 3	Reg with Room Type	60.89715	0.1044808891
## 4	Reg with Accommodates	56.91354	0.2165326508
## 5	Reg with Beds	61.01476	0.0995503403
## 6	Reg with Bathrooms	55.16945	0.2686162663
## 7	Reg with Min Nights	63.72875	0.0176632197
## 8	Reg with Total Reviews	63.78832	0.0158257530
## 9	Reg with Host Acceptance Rate	63.44086	0.0265182301
## 10	Reg with Superhost	64.25538	0.0013607687
## 11	Reg with Host Total Listings	62.63565	0.0510729702
## 12	Reg with Average Rating	64.28685	0.0003821928

```

#Model with lowest residual standard error
modelComp$Model[which.min(modelComp$SER)]

```

```
## [1] "Reg with Bathrooms"
```

```

#Model with highest residual standard error
modelComp$Model[which.max(modelComp$SER)]

```

```
## [1] "Reg with Average Rating"
```

```
#Q7:
# multiple linear model
# predictors: bedrooms + neighborhood + room_type + accommodates only analysis
filtReg = lm(price ~ bedrooms + neighborhood + room_type + accommodates,df_price)

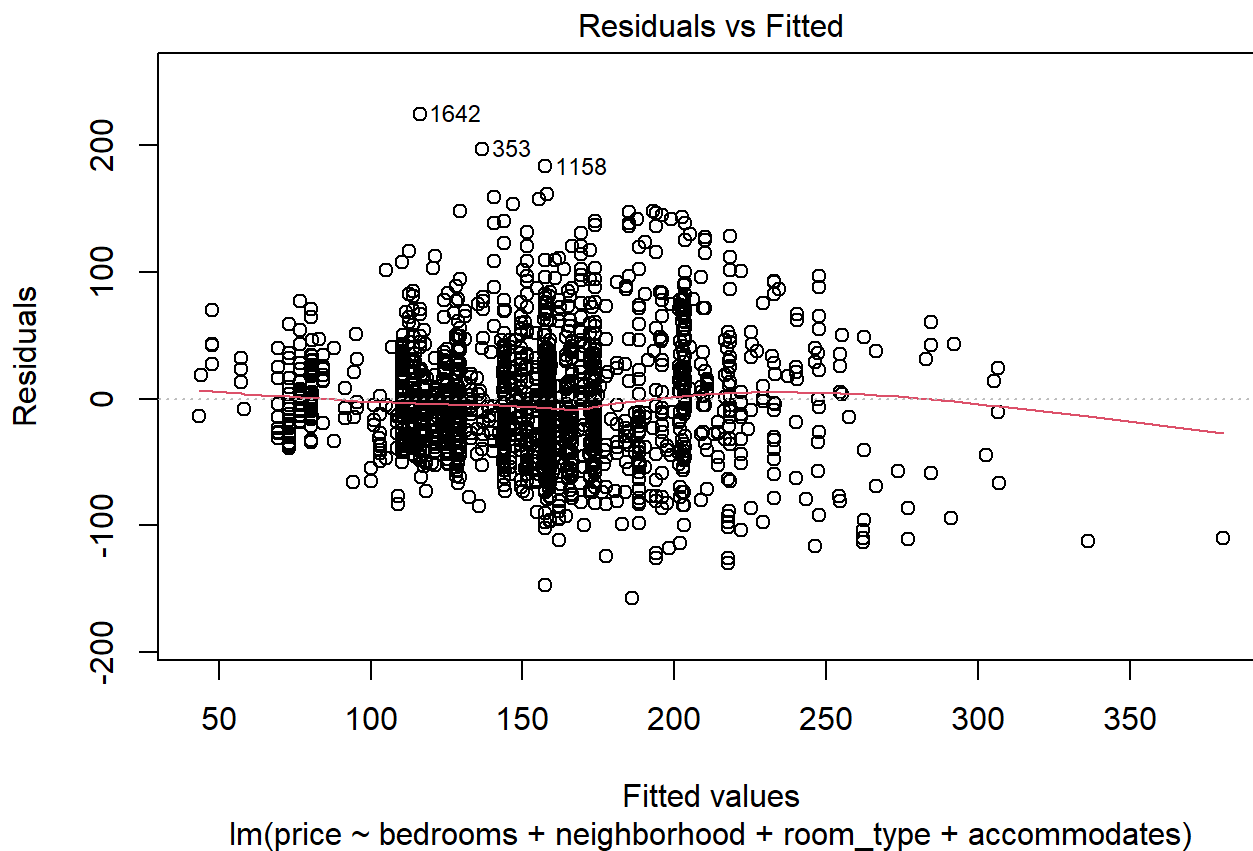
summary(filtReg)
```

```
##
## Call:
## lm(formula = price ~ bedrooms + neighborhood + room_type + accommodates,
##     data = df_price)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -157.915  -33.448   -6.635   27.190  224.755
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      114.678      4.068  28.191 < 2e-16 ***
## bedrooms           29.534      2.297  12.858 < 2e-16 ***
## neighborhoodColumbia Heights -48.806      4.456 -10.954 < 2e-16 ***
## neighborhoodDowntown       43.798      6.690   6.547 7.63e-11 ***
## neighborhoodDupont Circle   -1.475      4.530   -0.326  0.7447
## neighborhoodEdgewood      -45.157      4.191 -10.774 < 2e-16 ***
## neighborhoodFoggy Bottom     9.779     11.501   0.850  0.3953
## neighborhoodGeorgetown      11.113      6.449   1.723  0.0850 .
## neighborhoodKalamazoo Heights -13.091      6.278  -2.085  0.0372 *
## neighborhoodShaw            -9.369      5.016  -1.868  0.0619 .
## neighborhoodUnion Station   -14.970      3.717  -4.027 5.88e-05 ***
## room_typeHotel room        -24.337     12.842  -1.895  0.0582 .
## room_typePrivate room      -33.401      3.640  -9.177 < 2e-16 ***
## room_typeShared room       -94.360     13.545  -6.966 4.53e-12 ***
## room_typeUnknown           -2.705      8.547  -0.316  0.7517
## accommodates              7.401      1.065   6.950 5.07e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 49.8 on 1826 degrees of freedom
## Multiple R-squared:  0.4048, Adjusted R-squared:  0.3999
## F-statistic: 82.78 on 15 and 1826 DF, p-value: < 2.2e-16
```

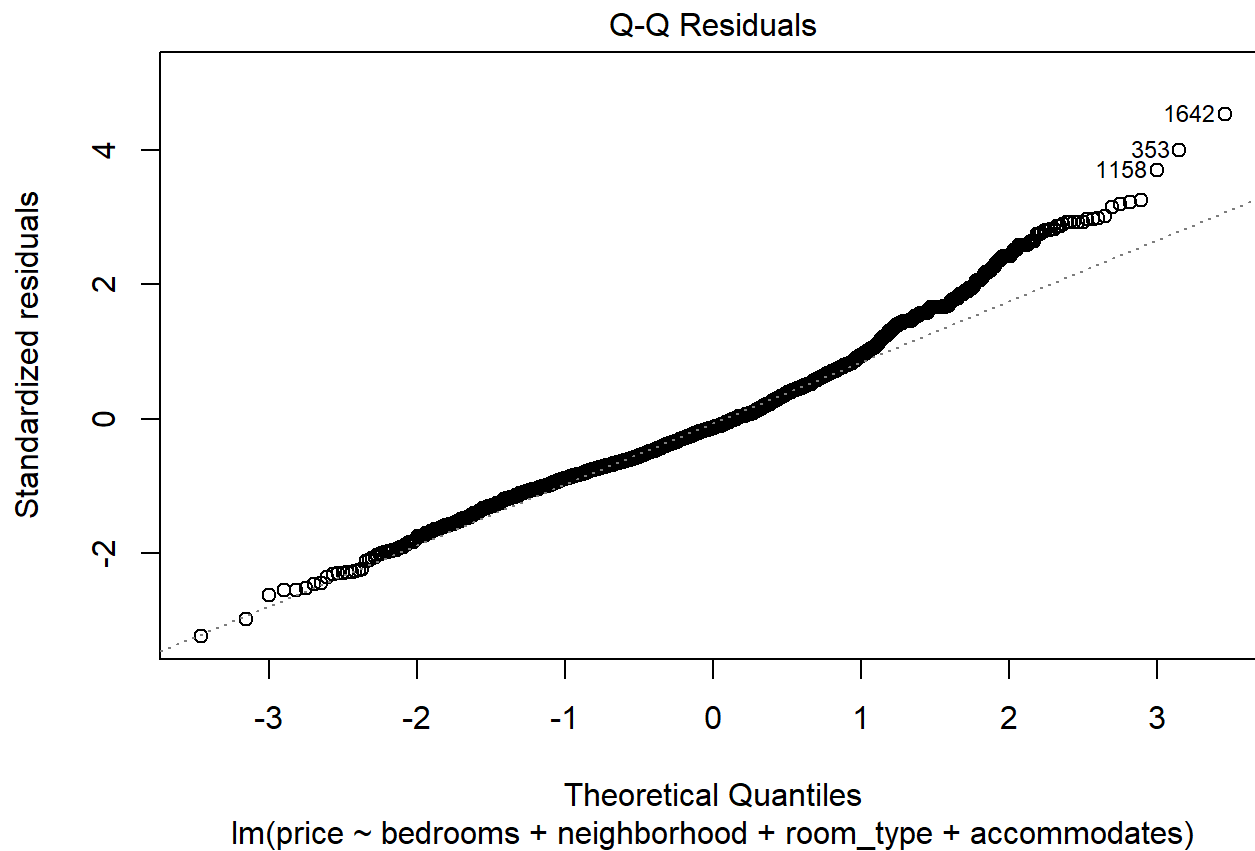
```
# coefficient table
coef = coef(summary(filtReg))
coef
```

##	Estimate	Std. Error	t value	Pr(> t)
## (Intercept)	114.677808	4.067920	28.1907705	1.827829e-145
## bedrooms	29.533668	2.296987	12.8575682	2.779478e-36
## neighborhoodColumbia Heights	-48.806178	4.455713	-10.9536169	4.356316e-27
## neighborhoodDowntown	43.798252	6.690221	6.5466079	7.626672e-11
## neighborhoodDupont Circle	-1.475209	4.530411	-0.3256236	7.447465e-01
## neighborhoodEdgewood	-45.157412	4.191256	-10.7741962	2.759777e-26
## neighborhoodFoggy Bottom	9.779139	11.501267	0.8502663	3.952885e-01
## neighborhoodGeorgetown	11.113385	6.448941	1.7232885	8.500566e-02
## neighborhoodKalorama Heights	-13.091115	6.278407	-2.0851013	3.719870e-02
## neighborhoodShaw	-9.368741	5.015586	-1.8679254	6.193253e-02
## neighborhoodUnion Station	-14.970486	3.717453	-4.0270818	5.878556e-05
## room_typeHotel room	-24.336809	12.842198	-1.8950658	5.824151e-02
## room_typePrivate room	-33.400972	3.639670	-9.1769222	1.163835e-19
## room_typeShared room	-94.359749	13.545352	-6.9662088	4.528111e-12
## room_typeUnknown	-2.705148	8.547334	-0.3164903	7.516666e-01
## accommodates	7.401429	1.064969	6.9498975	5.068598e-12

```
#Diagnostic plots
plot(filtReg, which = 1)
```



```
plot(filtReg, which = 2)
```



```
#Summary Of Data Model" sigma, R2, AdjR
ModelName = c("filtReg")
Sigma = c(summary(filtReg)$sigma)
RSqrd = c(summary(filtReg)$r.squared)
AdjRSqrd = c(summary(filtReg)$adj.r.squared)

SummaryTable = data.frame(ModelName,Sigma,RSqrd,AdjRSqrd)
SummaryTable
```

```
##   ModelName   Sigma   RSqrd AdjRSqrd
## 1   filtReg 49.7972 0.4047733 0.3998837
```



```
# predicted price
df_price$prdctPrce = predict(filtReg)

#Group, summarize, and keep top 5 groups
topFiveGroup = df_price %>%
  group_by(neighborhood, room_type) %>%
  summarize(
    numLstngs = n(),
    avgPred = mean(prdctPrce))%>%
  ungroup() %>%
  filter(numLstngs >= 5) %>%
  arrange(desc(avgPred)) %>%
  slice_head(n = 5)
```

```
## `summarise()` has grouped output by 'neighborhood'. You can override using the
## `.groups` argument.
```

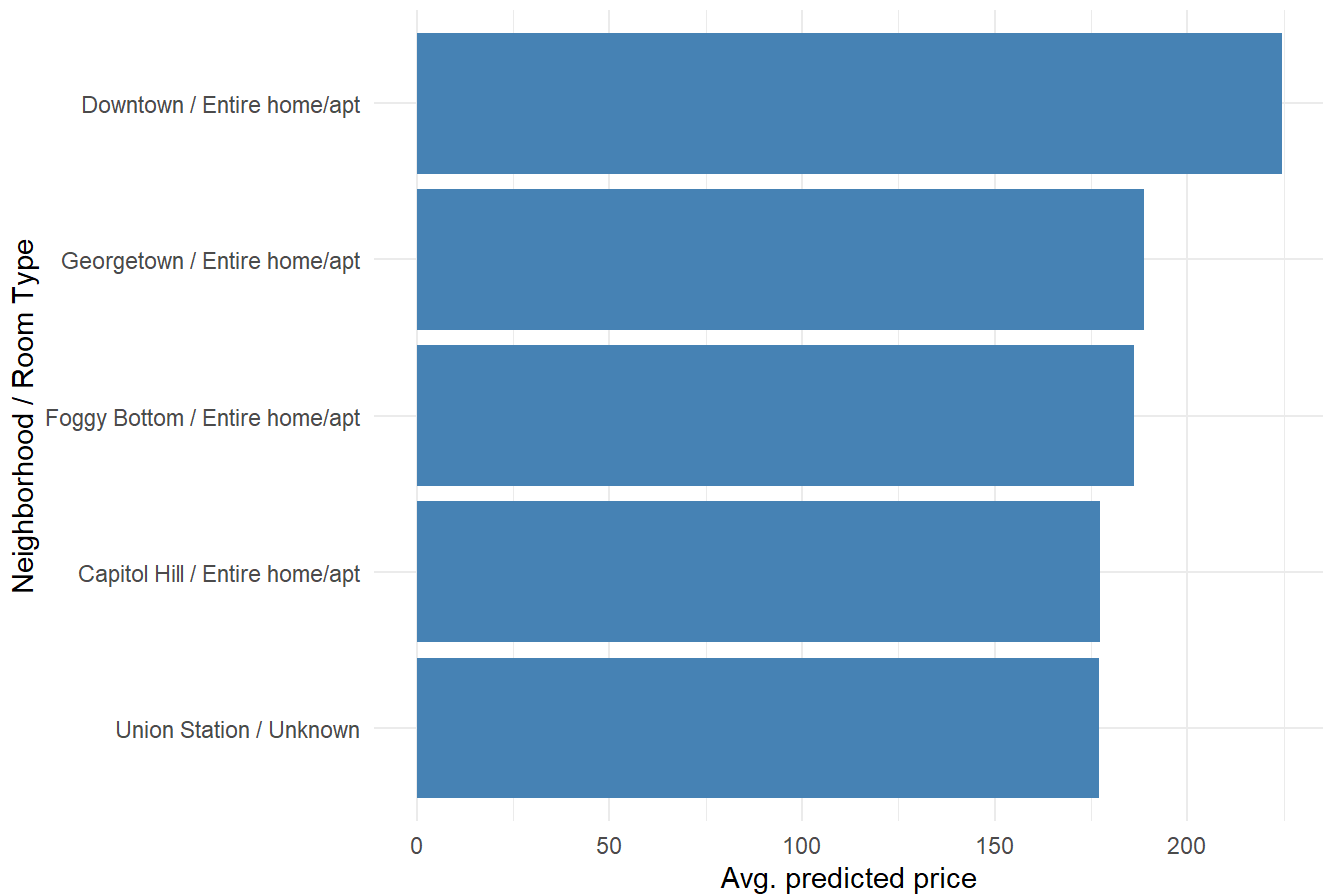
```
topFiveGroup
```

```
## # A tibble: 5 × 4
##   neighborhood room_type      numLstngs avgPred
##   <fct>         <fct>         <int>    <dbl>
## 1 Downtown     Entire home/apt         45    224.
## 2 Georgetown   Entire home/apt         64    189.
## 3 Foggy Bottom Entire home/apt         15    186.
## 4 Capitol Hill Entire home/apt        271    177.
## 5 Union Station Unknown           6    177.
```

```
#ggplot top 5
```

```
ggplot(topFiveGroup,
  aes(x = avgPred,
      y = reorder(paste(neighborhood, "/", room_type), avgPred))) +
  geom_col(fill = "steelblue") +
  labs(title = "Top 5 By Predicted Price (Avg.)",
    x = "Avg. predicted price", y = "Neighborhood / Room Type") +
  theme_minimal()
```

Top 5 By Predicted Price (Avg.)



#Q8: Ran correlations against bedrooms and accommodates. mild correlation. Still mulling this one over.

```
cor(df_price$bedrooms, df_price$accommodates)
```

```
## [1] 0.6860678
```

```

#
#
#
# ## Finding actionable insights:
#
# ## EXTRA:
#
# pb_reg = lm(price~beds , df)
# summary(pb_reg)
# plot(pb_reg)
# df[c(182,183,109),]
# df6 = df[-c(109,192,183),]
# pb_reg2 = lm(price~beds , df6)
# summary(pb_reg2)
#
#
# pa_reg= lm(price~accommodates, df)
# summary(pa_reg)
# plot(pa_reg)
# df[c(109,1098,1767),]
# pa_reg2 = lm(price~accommodates, df7)
# summary(pa_reg2)
# df7 = df[-c(109,192,183),]
#
# pb_reg = lm(price~bedrooms, df)
# summary(pb_reg)
# plot(pb_reg)
# df[c(327,503,1188,1524),]
# df8 = df[-c(327,503,1188,1524),]
# pb_reg2 =lm(price~bedrooms, df8)
# summary(pb_reg2)
# ## removing the outlier values from these only really increased the R^2 by around 0.02
#
# pn_reg = lm(price~neighborhood, df)
# summary(pn_reg)
# plot(pn_reg)
#
# prt_reg = lm(price~room_type, df)
# summary(prt_reg)
# plot(prt_reg)
#
# mult_reg = lm(price~bedrooms+accommodates+neighborhood, df)
# summary(mult_reg)
#
# df9 = df %>%
#   filter(neighborhood == "Capitol Hill" | neighborhood == "Edgewood" | neighborhood == "Union
# Station" |
#           neighborhood == "Dupont Circle" | neighborhood == "Columbia Heights")
#
# summary(lm(price~neighborhood , df9))
# ## due to the highly imbalanced dataset and poor regression
# # narrow into only entire homes since 1500 (over 80 percent of the dataset)

```

```
# df10 = df %>%
#   filter(room_type == "Entire home/apt")
#
#
# ggplot(df10 , aes(x = bedrooms, y = price)) +
#   geom_point() + geom_jitter() + geom_smooth(method = "lm" , se = FALSE) +
#   facet_wrap(~neighborhood, scales = "free_x")
#
# ggplot(df10 , aes(x = accommodates, y = price)) +
#   geom_point() + geom_jitter() + geom_smooth(method = "lm" , se = FALSE) +
#   facet_wrap(~neighborhood, scales = "free_x")
#
# df11 = df10 %>%
#   mutate(bedrooms_f = as.factor(bedrooms)) %>%
#   filter(neighborhood == "Capitol Hill")
#
#
# summary(lm(price~bedrooms+accommodates, df11))
#
# ggplot(df11, aes(x= accommodates, y = price,color = factor(bedrooms))) + geom_point() + facet_
wrap(~superhost , scales = "free_x")
#
# df11 %>%
#   select_if(is.numeric) %>%
#   cor()
#
#
#
#
#
#
#
#
#
#
#
#
#
#
#
#
#
#
#
# ## possibly create a new metric
#
# ggplot(df, aes(y = log(price), x=log(1+host_total_listings*host_acceptance_rate))) + geom_poin
t()
#
#
# ggplot(df, aes(y = price, x=0.5*accommodates + 0.5*bedrooms)) + geom_point()
#
#
# ## maybe explore avg rating
#
# ggplot(df, aes(y= price, x = avg_rating)) + geom_point()
#
#
```

```
#  
#  
#  
#  
#  
#  
#  
#
```